

particle-transport-framework

Generated by Doxygen 1.16.1

Chapter 1

particle-transport-framework

C++ framework for large-scale particle transport simulation and ocean flow data processing.

The framework is designed for long-running simulations based on gridded environmental velocity fields (e.g., HYCOM NetCDF data), with emphasis on performance, modular architecture, and reproducibility.

It provides building blocks for constructing simulation pipelines that operate on structured flow fields and track particle trajectories over time in complex spatial domains.

1.1 Features

- Modular architecture for simulation workflows and data processing pipelines
- Efficient handling and reorganization of gridded flow data (NetCDF, HYCOM)
- Spatial indexing using kd-trees (nanoflann) for fast nearest-neighbor queries
- Local geometric transformations and coordinate handling
- Interpolation and reconstruction of physical fields (Gaussian weighting, spline-based methods)
- Parallel processing using Intel TBB for efficient handling of large datasets and simulation workloads
- Support for long-duration simulations and repeated runs

1.2 Tech Stack

- C++
- CMake
- Boost
- NetCDF
- Intel TBB
- Linux

1.3 Status

The repository exposes the core structure and interfaces of the framework. Implementation is being actively cleaned, modularized, and documented.

Chapter 2

Hierarchical Index

2.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

BBox	??
pkd2::BlockHeader	??
Date	??
DateRange	??
FileHeaderPKD1	??
particle_tracker::Grid< DT >	??
std::hash<Date >	??
pkd2::Header	??
DateRange::iterator	??
particle_tracker::LonLatDistanceAdaptor< DataType, DatasetAdaptor >	??
MonthData	??
particle_tracker::MonthData	??
Neighbor< DataType >	??
particle_tracker::NeighborWithNeighbors< DataType >	??
particle_tracker::Neighbor< DataType >	??
particle_tracker::NeighborWithNeighbors< DataType >	??
particle_tracker::NeighborPrime< DataType >	??
NeighborsData< DataType, TimeType >	??
particle_tracker::NeighborsData< DataType, TimeType >	??
particle_tracker::NeighborWithNeighborsData< DataType, TimeType >	??
particle_tracker::NetCDFDataLoader< DataType, TimeType >	??
particle_tracker::NetCDFProcessor< DataType, TimeType >	??
PackedParticle	??
Particle< DT, TT >	??
particle_tracker::Particle< DT, TT >	??
ParticleState	??
particle_tracker::PointCloud< DataType >	??
PointCloud< DataType >	??
pkd2::Reader	??
ReconGuards	??
SeedInit	??
particle_tracker::TimeSpline< DataType, TimeType >	??
TimeSpline< DataType, TimeType >	??
particle_tracker::TimeStepData< DT, TT >	??
pkd2::TocEntry	??
Trajectory	??
particle_tracker::VariableDerivatives< DataType >	??
pkd2::Writer	??

Chapter 3

Class Index

3.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

BBox	??
pkd2::BlockHeader	??
Date	
Represents a calendar date (day/month/year) in the Gregorian calendar	??
DateRange	
Inclusive date range [startDate	??
FileHeaderPKD1	??
particle_tracker::Grid< DT >	??
std::hash<::Date >	??
pkd2::Header	??
DateRange::iterator	??
particle_tracker::LonLatDistanceAdaptor< DataType, DatasetAdaptor >	
Approximate metric for KD-tree pruning in (lon, lat)	??
MonthData	??
particle_tracker::MonthData	??
Neighbor< DataType >	
One spatial neighbor (grid node) used in interpolation	??
particle_tracker::Neighbor< DataType >	
One spatial neighbor (grid node) used in interpolation	??
particle_tracker::NeighborPrime< DataType >	
A "secondary neighbor" (neighbor of a primary neighbor)	??
NeighborsData< DataType, TimeType >	
Spatial neighbor query + Gaussian weighting + multi-variable interpolation	??
particle_tracker::NeighborsData< DataType, TimeType >	
Spatial neighbor query + Gaussian weighting + multi-variable interpolation	??
particle_tracker::NeighborWithNeighbors< DataType >	
A primary Neighbor plus a list of its secondary neighbors	??
particle_tracker::NeighborWithNeighborsData< DataType, TimeType >	
Two-level neighborhood builder + heuristic spatial derivative estimator	??
particle_tracker::NetCDFDataLoader< DataType, TimeType >	
Load one or more NetCDF files and store selected variables keyed by time	??
particle_tracker::NetCDFProcessor< DataType, TimeType >	
Convenience wrapper to load a single file and iterate over selected variables and times	??
PackedParticle	??
Particle< DT, TT >	??

particle_tracker::Particle< DT, TT >	??
ParticleState	??
particle_tracker::PointCloud< DataType >	
Minimal dataset adaptor for nanoflann storing (lon, lat) in a flat array	??
PointCloud< DataType >	
Minimal dataset adaptor for nanoflann storing (lon, lat) in a flat array	??
pkd2::Reader	??
ReconGuards	??
SeedInit	??
particle_tracker::TimeSpline< DataType, TimeType >	
Wrapper around a Boost cubic spline interpolator for uniformly spaced samples	??
TimeSpline< DataType, TimeType >	
Wrapper around a Boost cubic spline interpolator for uniformly spaced samples	??
particle_tracker::TimeStepData< DT, TT >	??
pkd2::TocEntry	??
Trajectory	??
particle_tracker::VariableDerivatives< DataType >	
Output container: value + time derivative + (heuristic) spatial derivatives	??
pkd2::Writer	??

Chapter 4

File Index

4.1 File List

Here is a list of all documented files with brief descriptions:

examples/hpc_particles.cpp	??
examples/hpc_trajectories.cpp	??
include/particle_tracker/data_prep_utils.hpp	??
include/particle_tracker/geography_utils.hpp	
Small geographic utilities for spherical (lon/lat) computations in radians	??
include/particle_tracker/interpolation_utils.hpp	
Time and spherical (lon/lat) interpolation utilities, including an experimental "neighbors-of-neighbors" spatial derivative estimator	??
include/particle_tracker/seed_loader_v2.hpp	??
include/particle_tracker/detail/datetime_utils.hpp	
Small date and date-range utilities (civil calendar, Gregorian rules)	??
include/particle_tracker/detail/ncdf_utils.hpp	
Header-only NetCDF-C++4 helpers for loading gridded variables (optionally time-dependent)	??
include/particle_tracker/fixed_point/fixed_point_core.hpp	??
include/particle_tracker/fixed_point/fixed_point_io_codec.hpp	??
include/particle_tracker/fixed_point/fixed_point_pkd1_io.hpp	??
include/particle_tracker/fixed_point/fixed_point_pkd2.hpp	??
include/particle_tracker/fixed_point/pk_config.hpp	??

Chapter 5

Class Documentation

5.1 BBox Struct Reference

Public Member Functions

- [BBox](#) (DataType lo_min, DataType lo_max, DataType la_min, DataType la_max)
- bool [contains](#) (DataType lon, DataType lat) const noexcept

Static Public Member Functions

- static DataType [norm_pi](#) (DataType L) noexcept

Public Attributes

- DataType [lon_min](#) {}
- DataType [lon_max](#) {}
- DataType [lat_min](#) {}
- DataType [lat_max](#) {}
- bool [wraps](#) {}

5.1.1 Detailed Description

Definition at line [84](#) of file [hpc_particles.cpp](#).

5.1.2 Constructor & Destructor Documentation

5.1.2.1 BBox()

```
BBox::BBox (
    DataType lo_min,
    DataType lo_max,
    DataType la_min,
    DataType la_max) [inline]
```

Definition at line [94](#) of file [hpc_particles.cpp](#).

5.1.3 Member Function Documentation

5.1.3.1 contains()

```
bool BBox::contains (  
    DataType lon,  
    DataType lat) const [inline], [noexcept]
```

Definition at line 105 of file [hpc_particles.cpp](#).

5.1.3.2 norm_pi()

```
DataType BBox::norm_pi (  
    DataType L) [inline], [static], [noexcept]
```

Definition at line 87 of file [hpc_particles.cpp](#).

5.1.4 Member Data Documentation

5.1.4.1 lat_max

```
DataType BBox::lat_max {}
```

Definition at line 85 of file [hpc_particles.cpp](#).

5.1.4.2 lat_min

```
DataType BBox::lat_min {}
```

Definition at line 85 of file [hpc_particles.cpp](#).

5.1.4.3 lon_max

```
DataType BBox::lon_max {}
```

Definition at line 85 of file [hpc_particles.cpp](#).

5.1.4.4 lon_min

```
DataType BBox::lon_min {}
```

Definition at line 85 of file [hpc_particles.cpp](#).

5.1.4.5 wraps

```
bool BBox::wraps {}
```

Definition at line 86 of file [hpc_particles.cpp](#).

The documentation for this struct was generated from the following file:

- [examples/hpc_particles.cpp](#)

5.2 pkd2::BlockHeader Struct Reference

Public Attributes

- uint64_t [seed_id](#)
- int32_t [start_index](#)
- uint16_t [stride](#)
- uint16_t [_pad](#) {0}
- uint32_t [n_points](#)
- int32_t [lon0_q](#)
- int32_t [lat0_q](#)

5.2.1 Detailed Description

Definition at line 78 of file [fixed_point_pkd2.hpp](#).

5.2.2 Member Data Documentation

5.2.2.1 _pad

```
uint16_t pkd2::BlockHeader::_pad {0}
```

Definition at line 82 of file [fixed_point_pkd2.hpp](#).

5.2.2.2 lat0_q

```
int32_t pkd2::BlockHeader::lat0_q
```

Definition at line 85 of file [fixed_point_pkd2.hpp](#).

5.2.2.3 lon0_q

```
int32_t pkd2::BlockHeader::lon0_q
```

Definition at line 84 of file [fixed_point_pkd2.hpp](#).

5.2.2.4 n_points

```
uint32_t pkd2::BlockHeader::n_points
```

Definition at line 83 of file [fixed_point_pkd2.hpp](#).

5.2.2.5 seed_id

```
uint64_t pkd2::BlockHeader::seed_id
```

Definition at line 79 of file [fixed_point_pkd2.hpp](#).

5.2.2.6 start_index

```
int32_t pkd2::BlockHeader::start_index
```

Definition at line 80 of file [fixed_point_pkd2.hpp](#).

5.2.2.7 stride

```
uint16_t pkd2::BlockHeader::stride
```

Definition at line 81 of file [fixed_point_pkd2.hpp](#).

The documentation for this struct was generated from the following file:

- [include/particle_tracker/fixed_point/fixed_point_pkd2.hpp](#)

5.3 Date Class Reference

Represents a calendar date (day/month/year) in the Gregorian calendar.

```
#include <datetime_utils.hpp>
```

Public Member Functions

- [Date](#) (int day, int month, int year)
Construct a [Date](#).
- void [increment](#) ()
- void [decrement](#) ()
- bool [operator<=](#) (const [Date](#) &other) const
- bool [operator<](#) (const [Date](#) &other) const
- bool [operator==](#) (const [Date](#) &other) const
- bool [operator>](#) (const [Date](#) &other) const
- bool [operator>=](#) (const [Date](#) &other) const
- bool [operator!=](#) (const [Date](#) &other) const
- int [getYear](#) () const
- int [getMonth](#) () const
- int [getDay](#) () const
- int [daysSince](#) (const [Date](#) &other) const
*Days difference: (*this) - other in days.*
- int [hoursSince](#) (const [Date](#) &other) const
- int [secondsSince](#) (const [Date](#) &other) const
- [Date](#) [addDays](#) (int delta) const
Return a new [Date](#) shifted by delta days.
- [Date](#) [addMonths](#) (int delta) const
Return a new [Date](#) shifted by delta months.
- [Date](#) [beginOfMonth](#) () const

Friends

- `std::ostream & operator<< (std::ostream &os, const Date &date)`

5.3.1 Detailed Description

Represents a calendar date (day/month/year) in the Gregorian calendar.

This is intentionally minimal:

- No time-of-day.
- No timezone or locale.

Arithmetic:

- `increment()/decrement()`: +/- 1 day with proper month/year rollover.
- `addDays(delta)`: returns a new [Date](#) shifted by delta days (uses repeated +/-1 steps). For large deltas you could add an optimized version, but in your simulation the heavy operations are typically in `secondsSince()` / `daysSince()`, which are $O(1)$.
- `addMonths(delta)`: shifts month, clamps day to last valid day of target month.

Comparison:

- Lexicographic by (year, month, day).

Safety:

- Constructor validates month/day ranges and throws if invalid.

Definition at line 44 of file [datetime_utils.hpp](#).

5.3.2 Constructor & Destructor Documentation

5.3.2.1 Date()

```
Date::Date (
    int day,
    int month,
    int year) [inline]
```

Construct a [Date](#).

Exceptions

<code>std::runtime_error</code>	if month/day are invalid.
---------------------------------	---------------------------

Definition at line 94 of file [datetime_utils.hpp](#).

Referenced by [addDays\(\)](#), [addMonths\(\)](#), and [daysSince\(\)](#).

5.3.3 Member Function Documentation

5.3.3.1 addDays()

```
Date Date::addDays (
    int delta) const [inline]
```

Return a new [Date](#) shifted by delta days.

Note: This uses repeated increment()/decrement(). It is simple and safe.

Definition at line 220 of file [datetime_utils.hpp](#).

References [Date\(\)](#).

5.3.3.2 addMonths()

```
Date Date::addMonths (
    int delta) const [inline]
```

Return a new [Date](#) shifted by delta months.

The day is clamped to the last valid day in the resulting month. Example: 31 Jan + 1 month -> 28 Feb (or 29 in leap years).

Definition at line 242 of file [datetime_utils.hpp](#).

References [Date\(\)](#).

5.3.3.3 beginOfMonth()

```
Date Date::beginOfMonth () const [inline]
```

Definition at line 261 of file [datetime_utils.hpp](#).

5.3.3.4 daysSince()

```
int Date::daysSince (
    const Date & other) const [inline]
```

Days difference: (*this) - other in days.

Returns:

- positive if this date is later than other
- negative if earlier
- zero if equal

Complexity: O(1).

Definition at line 193 of file [datetime_utils.hpp](#).

References [Date\(\)](#).

5.3.3.5 decrement()

```
void Date::decrement () [inline]
```

Definition at line 121 of file [datetime_utils.hpp](#).

5.3.3.6 getDay()

```
int Date::getDay () const [inline]
```

Definition at line 181 of file [datetime_utils.hpp](#).

5.3.3.7 getMonth()

```
int Date::getMonth () const [inline]
```

Definition at line 179 of file [datetime_utils.hpp](#).

5.3.3.8 getYear()

```
int Date::getYear () const [inline]
```

Definition at line 177 of file [datetime_utils.hpp](#).

5.3.3.9 hoursSince()

```
int Date::hoursSince (  
    const Date & other) const [inline]
```

Definition at line 203 of file [datetime_utils.hpp](#).

5.3.3.10 increment()

```
void Date::increment () [inline]
```

Definition at line 105 of file [datetime_utils.hpp](#).

5.3.3.11 operator"!="()

```
bool Date::operator!= (  
    const Date & other) const [inline]
```

Definition at line 168 of file [datetime_utils.hpp](#).

5.3.3.12 operator<()

```
bool Date::operator< (
    const Date & other) const [inline]
```

Definition at line 151 of file [datetime_utils.hpp](#).

5.3.3.13 operator<=()

```
bool Date::operator<= (
    const Date & other) const [inline]
```

Definition at line 143 of file [datetime_utils.hpp](#).

5.3.3.14 operator==(())

```
bool Date::operator==(
    const Date & other) const [inline]
```

Definition at line 159 of file [datetime_utils.hpp](#).

5.3.3.15 operator>()

```
bool Date::operator> (
    const Date & other) const [inline]
```

Definition at line 164 of file [datetime_utils.hpp](#).

5.3.3.16 operator>=()

```
bool Date::operator>= (
    const Date & other) const [inline]
```

Definition at line 166 of file [datetime_utils.hpp](#).

5.3.3.17 secondsSince()

```
int Date::secondsSince (
    const Date & other) const [inline]
```

Definition at line 209 of file [datetime_utils.hpp](#).

5.3.4 Friends And Related Symbol Documentation

5.3.4.1 operator<<

```
std::ostream & operator<< (  
    std::ostream & os,  
    const Date & date) [friend]
```

Definition at line 171 of file [datetime_utils.hpp](#).

The documentation for this class was generated from the following file:

- [include/particle_tracker/detail/datetime_utils.hpp](#)

5.4 DateRange Class Reference

Inclusive date range [startDate .

```
#include <datetime_utils.hpp>
```

Classes

- struct [iterator](#)

Public Member Functions

- [DateRange](#) (const [Date](#) &s, const [Date](#) &e)
- [iterator begin](#) () const
- [iterator end](#) () const
- int [length](#) () const

5.4.1 Detailed Description

Inclusive date range [startDate .

. endDate] with iterator support.

Example: for (const [Date](#)& d : [DateRange](#)([Date](#)(1,1,2000), [Date](#)(10,1,2000))) { ... }

Definition at line 273 of file [datetime_utils.hpp](#).

5.4.2 Constructor & Destructor Documentation

5.4.2.1 DateRange()

```
DateRange::DateRange (  
    const Date & s,  
    const Date & e) [inline]
```

Definition at line 279 of file [datetime_utils.hpp](#).

5.4.3 Member Function Documentation

5.4.3.1 begin()

```
iterator DateRange::begin () const [inline]
```

Definition at line 305 of file [datetime_utils.hpp](#).

5.4.3.2 end()

```
iterator DateRange::end () const [inline]
```

Definition at line 307 of file [datetime_utils.hpp](#).

5.4.3.3 length()

```
int DateRange::length () const [inline]
```

Definition at line 315 of file [datetime_utils.hpp](#).

The documentation for this class was generated from the following file:

- [include/particle_tracker/detail/datetime_utils.hpp](#)

5.5 FileHeaderPKD1 Struct Reference

Public Attributes

- `std::array< char, 8 > magic = {'P','K','D','1',0,0,0,0}`
- `uint32_t version = 1`
- `uint32_t pos_scale = fp_ticks_per_rad`
- `uint64_t time_tick = fp_ticks_per_sec`
- `uint64_t reserved0 = 0`
- `uint64_t reserved1 = 0`

5.5.1 Detailed Description

Definition at line 71 of file [fixed_point_core.hpp](#).

5.5.2 Member Data Documentation

5.5.2.1 magic

```
std::array<char,8> FileHeaderPKD1::magic = {'P','K','D','1',0,0,0,0}
```

Definition at line 73 of file [fixed_point_core.hpp](#).

5.5.2.2 pos_scale

```
uint32_t FileHeaderPKD1::pos_scale = fp_ticks_per_rad
```

Definition at line 75 of file [fixed_point_core.hpp](#).

5.5.2.3 reserved0

```
uint64_t FileHeaderPKD1::reserved0 = 0
```

Definition at line 77 of file [fixed_point_core.hpp](#).

5.5.2.4 reserved1

```
uint64_t FileHeaderPKD1::reserved1 = 0
```

Definition at line 78 of file [fixed_point_core.hpp](#).

5.5.2.5 time_tick

```
uint64_t FileHeaderPKD1::time_tick = fp_ticks_per_sec
```

Definition at line 76 of file [fixed_point_core.hpp](#).

5.5.2.6 version

```
uint32_t FileHeaderPKD1::version = 1
```

Definition at line 74 of file [fixed_point_core.hpp](#).

The documentation for this struct was generated from the following file:

- [include/particle_tracker/fixed_point/fixed_point_core.hpp](#)

5.6 particle_tracker::Grid< DT > Struct Template Reference

Public Attributes

- `std::vector< DT >` [lon_rad](#)
- `std::vector< DT >` [lat_rad](#)
- `std::vector< bool >` [in_the_sea](#)
- `std::size_t` [num_points](#)

5.6.1 Detailed Description

```
template<typename DT>
struct particle_tracker::Grid< DT >
```

Definition at line 57 of file [data_prep_utils.hpp](#).

5.6.2 Constructor & Destructor Documentation

5.6.2.1 Grid()

```
template<typename DT>
particle_tracker::Grid< DT >::Grid () [inline]
```

Definition at line 63 of file [data_prep_utils.hpp](#).

5.6.3 Member Data Documentation

5.6.3.1 in_the_sea

```
template<typename DT>
std::vector<bool> particle_tracker::Grid< DT >::in_the_sea
```

Definition at line 60 of file [data_prep_utils.hpp](#).

5.6.3.2 lat_rad

```
template<typename DT>
std::vector<DT> particle_tracker::Grid< DT >::lat_rad
```

Definition at line 59 of file [data_prep_utils.hpp](#).

5.6.3.3 lon_rad

```
template<typename DT>
std::vector<DT> particle_tracker::Grid< DT >::lon_rad
```

Definition at line 58 of file [data_prep_utils.hpp](#).

5.6.3.4 num_points

```
template<typename DT>
std::size_t particle_tracker::Grid< DT >::num_points
```

Definition at line 61 of file [data_prep_utils.hpp](#).

The documentation for this struct was generated from the following file:

- [include/particle_tracker/data_prep_utils.hpp](#)

5.7 std::hash<::Date > Struct Reference

Public Member Functions

- std::size_t [operator\(\)](#) (const ::Date &d) const noexcept

5.7.1 Detailed Description

Definition at line 45 of file [data_prep_utils.hpp](#).

5.7.2 Member Function Documentation

5.7.2.1 operator()()

```
std::size_t std::hash<::Date >::operator() (
    const ::Date & d) const [inline], [noexcept]
```

Definition at line 46 of file [data_prep_utils.hpp](#).

The documentation for this struct was generated from the following file:

- include/particle_tracker/data_prep_utils.hpp

5.8 pkd2::Header Struct Reference

Public Attributes

- char [magic](#) [8] = {'P', 'K', 'D', '2', 0, 0, 0, 0}
- uint32_t [version](#) = 3
- uint8_t [little](#) = 1
- uint8_t [_r1](#) {0}
- uint16_t [_r2](#) {0}
- uint32_t [pos_scale_ticks_per_rad](#) = 1'000'000
- int64_t [t0_epoch_s](#) = 0
- int32_t [dt_s](#) = 0
- uint32_t [_r3](#) {0}
- uint64_t [n_traj](#) = 0
- uint64_t [toc_offset](#) = 0

5.8.1 Detailed Description

Definition at line 63 of file [fixed_point_pkd2.hpp](#).

5.8.2 Member Data Documentation

5.8.2.1 _r1

```
uint8_t pkd2::Header::_r1 {0}
```

Definition at line 67 of file [fixed_point_pkd2.hpp](#).

5.8.2.2 _r2

```
uint16_t pkd2::Header::_r2 {0}
```

Definition at line 68 of file [fixed_point_pkd2.hpp](#).

5.8.2.3 _r3

```
uint32_t pkd2::Header::_r3 {0}
```

Definition at line 72 of file [fixed_point_pkd2.hpp](#).

5.8.2.4 dt_s

```
int32_t pkd2::Header::dt_s = 0
```

Definition at line 71 of file [fixed_point_pkd2.hpp](#).

5.8.2.5 little

```
uint8_t pkd2::Header::little = 1
```

Definition at line 66 of file [fixed_point_pkd2.hpp](#).

5.8.2.6 magic

```
char pkd2::Header::magic[8] = {'P', 'K', 'D', '2', 0, 0, 0, 0}
```

Definition at line 64 of file [fixed_point_pkd2.hpp](#).

5.8.2.7 n_traj

```
uint64_t pkd2::Header::n_traj = 0
```

Definition at line 73 of file [fixed_point_pkd2.hpp](#).

5.8.2.8 pos_scale_ticks_per_rad

```
uint32_t pkd2::Header::pos_scale_ticks_per_rad = 1'000'000
```

Definition at line 69 of file [fixed_point_pkd2.hpp](#).

5.8.2.9 t0_epoch_s

```
int64_t pkd2::Header::t0_epoch_s = 0
```

Definition at line 70 of file [fixed_point_pkd2.hpp](#).

5.8.2.10 toc_offset

```
uint64_t pkd2::Header::toc_offset = 0
```

Definition at line 74 of file [fixed_point_pkd2.hpp](#).

5.8.2.11 version

```
uint32_t pkd2::Header::version = 3
```

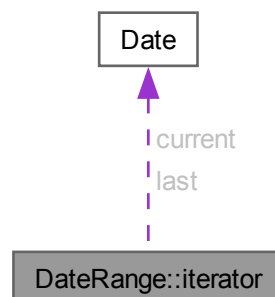
Definition at line 65 of file [fixed_point_pkd2.hpp](#).

The documentation for this struct was generated from the following file:

- include/particle_tracker/fixed_point/fixed_point_pkd2.hpp

5.9 DateRange::iterator Struct Reference

Collaboration diagram for DateRange::iterator:



Public Member Functions

- [iterator](#) (const [Date](#) &cur, const [Date](#) &last)
- const [Date](#) & [operator*](#) () const
- iterator & [operator++](#) ()
- bool [operator!=](#) (const iterator &other) const

Public Attributes

- [Date](#) current
- [Date](#) last

5.9.1 Detailed Description

Definition at line 285 of file [datetime_utils.hpp](#).

5.9.2 Constructor & Destructor Documentation

5.9.2.1 iterator()

```
DateRange::iterator::iterator (  
    const Date & cur,  
    const Date & last) [inline]
```

Definition at line 289 of file [datetime_utils.hpp](#).

5.9.3 Member Function Documentation

5.9.3.1 operator"!=()

```
bool DateRange::iterator::operator!= (  
    const iterator & other) const [inline]
```

Definition at line 299 of file [datetime_utils.hpp](#).

5.9.3.2 operator*()

```
const Date & DateRange::iterator::operator* () const [inline]
```

Definition at line 291 of file [datetime_utils.hpp](#).

5.9.3.3 operator++()

```
iterator & DateRange::iterator::operator++ () [inline]
```

Definition at line 293 of file [datetime_utils.hpp](#).

5.9.4 Member Data Documentation

5.9.4.1 current

`Date` `DateRange::iterator::current`

Definition at line 287 of file `datetime_utils.hpp`.

5.9.4.2 last

`Date` `DateRange::iterator::last`

Definition at line 287 of file `datetime_utils.hpp`.

The documentation for this struct was generated from the following file:

- `include/particle_tracker/detail/datetime_utils.hpp`

5.10 particle_tracker::LonLatDistanceAdaptor< DataType, DatasetAdaptor > Struct Template Reference

Approximate metric for KD-tree pruning in (lon, lat).

```
#include <interpolation_utils.hpp>
```

Public Types

- typedef `DataType` `ElementType`
- typedef `DataType` `DistanceType`

Public Member Functions

- `LonLatDistanceAdaptor` (`const DatasetAdaptor &data_`)
- `const DataType * getPoint` (`std::size_t idx`) `const`
- `DistanceType evalMetric` (`const DataType *a`, `const DataType *b`, `std::size_t`) `const`
Squared approximate distance between two explicit points.
- `DistanceType evalMetric` (`const DataType *a`, `std::size_t idx_b`, `std::size_t size`) `const`
Squared approximate distance between query point a and dataset point idx_b.
- `template<typename T, typename IndexType>`
`DistanceType accum_dist` (`const T a`, `const T b`, `IndexType dim`) `const`
Incremental squared contribution along a single dimension (KD-tree pruning).

Public Attributes

- `const DatasetAdaptor & data`

5.10.1 Detailed Description

```
template<typename DataType, class DatasetAdaptor>
struct particle_tracker::LonLatDistanceAdaptor< DataType, DatasetAdaptor >
```

Approximate metric for KD-tree pruning in (lon, lat).

Template Parameters

<i>DataType</i>	numeric type.
<i>DatasetAdaptor</i>	type providing <code>pts</code> in [lon,lat,lon,lat,...] layout.

5.10.1.0.1 What this metric is (and is not)

- This is **not** a great-circle distance.
- It is a cheap local approximation used *only* for KD-tree pruning:

$$d^2 \approx (dlat)^2 + (\cos(lat_avg) * dlon_wrapped)^2$$
- After KD-tree returns candidate neighbors, **true** great-circle distance is computed via `GCD()` /↔ `GCD_deriv()` and used to compute weights.

5.10.1.0.2 Dateline behavior

- Longitude differences are wrapped into $(-\pi, \pi]$ for continuity across the dateline.

Definition at line 298 of file [interpolation_utils.hpp](#).

5.10.2 Member Typedef Documentation

5.10.2.1 DistanceType

```
template<typename DataType, class DatasetAdaptor>
typedef DataType particle_tracker::LonLatDistanceAdaptor< DataType, DatasetAdaptor >::Distance↔
Type
```

Definition at line 300 of file [interpolation_utils.hpp](#).

5.10.2.2 ElementType

```
template<typename DataType, class DatasetAdaptor>
typedef DataType particle_tracker::LonLatDistanceAdaptor< DataType, DatasetAdaptor >::Element↔
Type
```

Definition at line 299 of file [interpolation_utils.hpp](#).

5.10.3 Constructor & Destructor Documentation

5.10.3.1 LonLatDistanceAdaptor()

```
template<typename DataType, class DatasetAdaptor>
particle_tracker::LonLatDistanceAdaptor< DataType, DatasetAdaptor >::LonLatDistanceAdaptor (
    const DatasetAdaptor & data_) [inline], [explicit]
```

Definition at line 304 of file [interpolation_utils.hpp](#).

5.10.4 Member Function Documentation

5.10.4.1 accum_dist()

```
template<typename DataType, class DatasetAdaptor>
template<typename T, typename IndexType>
DistanceType particle_tracker::LonLatDistanceAdaptor< DataType, DatasetAdaptor >::accum_dist (
    const T a,
    const T b,
    IndexType dim) const [inline]
```

Incremental squared contribution along a single dimension (KD-tree pruning).

Parameters

<i>a</i>	coordinate of query
<i>b</i>	coordinate of dataset
<i>dim</i>	dimension index (0 -> lon, 1 -> lat)

Definition at line 350 of file [interpolation_utils.hpp](#).

5.10.4.2 evalMetric() [1/2]

```
template<typename DataType, class DatasetAdaptor>
DistanceType particle_tracker::LonLatDistanceAdaptor< DataType, DatasetAdaptor >::evalMetric (
    const DataType * a,
    const DataType * b,
    std::size_t ) const [inline]
```

Squared approximate distance between two explicit points.

Parameters

<i>a</i>	pointer to {lon, lat}
<i>b</i>	pointer to {lon, lat}

Definition at line 316 of file [interpolation_utils.hpp](#).

Referenced by [evalMetric\(\)](#).

5.10.4.3 evalMetric() [2/2]

```
template<typename DataType, class DatasetAdaptor>
DistanceType particle_tracker::LonLatDistanceAdaptor< DataType, DatasetAdaptor >::evalMetric (
    const DataType * a,
    std::size_t idx_b,
    std::size_t size) const [inline]
```

Squared approximate distance between query point a and dataset point idx_b.

Definition at line 335 of file [interpolation_utils.hpp](#).

References [evalMetric\(\)](#), and [getPoint\(\)](#).

5.10.4.4 getPoint()

```
template<typename DataType, class DatasetAdaptor>
const DataType * particle_tracker::LonLatDistanceAdaptor< DataType, DatasetAdaptor >::getPoint
(
    std::size_t idx) const [inline]
```

Returns

pointer to (lon, lat) for dataset point idx.

Definition at line 307 of file [interpolation_utils.hpp](#).

Referenced by [evalMetric\(\)](#).

5.10.5 Member Data Documentation

5.10.5.1 data

```
template<typename DataType, class DatasetAdaptor>
const DatasetAdaptor& particle_tracker::LonLatDistanceAdaptor< DataType, DatasetAdaptor >::↔
data
```

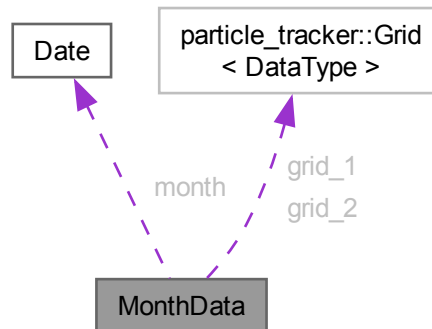
Definition at line 302 of file [interpolation_utils.hpp](#).

The documentation for this struct was generated from the following file:

- [include/particle_tracker/interpolation_utils.hpp](#)

5.11 MonthData Struct Reference

Collaboration diagram for MonthData:



Public Attributes

- [Date month](#) {1, 1, 2000}
- spline_map< DataType > [spl_1](#)
- spline_map< DataType > [spl_2](#)
- spline_map< DataType > [spl](#)
- Grid< DataType > [grid_1](#)
- Grid< DataType > [grid_2](#)
- std::vector< std::size_t > [sea_idx_1](#)
- std::vector< std::size_t > [sea_idx_2](#)
- std::vector< DataType > [sea_x](#)
- std::vector< DataType > [sea_y](#)
- PointCloud< DataType > [sea_cloud](#)
- PointCloud< DataType > [land_cloud](#)
- std::unique_ptr< KDTree< DataType > > [kd_sea](#) {nullptr}
- std::unique_ptr< KDTree< DataType > > [kd_land](#) {nullptr}

5.11.1 Detailed Description

Definition at line 606 of file [data_prep_utils.hpp](#).

5.11.2 Member Data Documentation

5.11.2.1 grid_1

Grid<DataType> particle_tracker::MonthData::grid_1

Definition at line 612 of file [data_prep_utils.hpp](#).

5.11.2.2 grid_2

```
Grid<DataType> particle_tracker::MonthData::grid_2
```

Definition at line 612 of file [data_prep_utils.hpp](#).

5.11.2.3 kd_land

```
std::unique_ptr<KDTree<DataType> > particle_tracker::MonthData::kd_land {nullptr}
```

Definition at line 617 of file [data_prep_utils.hpp](#).

5.11.2.4 kd_sea

```
std::unique_ptr<KDTree<DataType> > particle_tracker::MonthData::kd_sea {nullptr}
```

Definition at line 616 of file [data_prep_utils.hpp](#).

5.11.2.5 land_cloud

```
PointCloud<DataType> particle_tracker::MonthData::land_cloud
```

Definition at line 615 of file [data_prep_utils.hpp](#).

5.11.2.6 month

```
Date particle_tracker::MonthData::month {1, 1, 2000}
```

Definition at line 608 of file [data_prep_utils.hpp](#).

5.11.2.7 sea_cloud

```
PointCloud<DataType> particle_tracker::MonthData::sea_cloud
```

Definition at line 615 of file [data_prep_utils.hpp](#).

5.11.2.8 sea_idx_1

```
std::vector<std::size_t> particle_tracker::MonthData::sea_idx_1
```

Definition at line 613 of file [data_prep_utils.hpp](#).

5.11.2.9 sea_idx_2

```
std::vector<std::size_t> particle_tracker::MonthData::sea_idx_2
```

Definition at line 613 of file [data_prep_utils.hpp](#).

5.11.2.10 sea_x

```
std::vector<DataType> particle_tracker::MonthData::sea_x
```

Definition at line 614 of file [data_prep_utils.hpp](#).

5.11.2.11 sea_y

```
std::vector<DataType> particle_tracker::MonthData::sea_y
```

Definition at line 614 of file [data_prep_utils.hpp](#).

5.11.2.12 spl

```
spline_map<DataType> particle_tracker::MonthData::spl
```

Definition at line 611 of file [data_prep_utils.hpp](#).

5.11.2.13 spl_1

```
spline_map<DataType> particle_tracker::MonthData::spl_1
```

Definition at line 609 of file [data_prep_utils.hpp](#).

5.11.2.14 spl_2

```
spline_map<DataType> particle_tracker::MonthData::spl_2
```

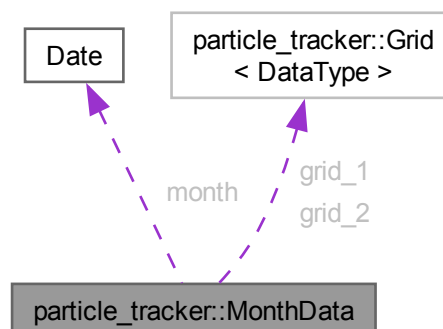
Definition at line 610 of file [data_prep_utils.hpp](#).

The documentation for this struct was generated from the following file:

- [include/particle_tracker/data_prep_utils.hpp](#)

5.12 particle_tracker::MonthData Struct Reference

Collaboration diagram for particle_tracker::MonthData:



Public Attributes

- [Date month](#) {1, 1, 2000}
- [spline_map](#)< [DataType](#) > [spl_1](#)
- [spline_map](#)< [DataType](#) > [spl_2](#)
- [spline_map](#)< [DataType](#) > [spl](#)
- [Grid](#)< [DataType](#) > [grid_1](#)
- [Grid](#)< [DataType](#) > [grid_2](#)
- [std::vector](#)< [std::size_t](#) > [sea_idx_1](#)
- [std::vector](#)< [std::size_t](#) > [sea_idx_2](#)
- [std::vector](#)< [DataType](#) > [sea_x](#)
- [std::vector](#)< [DataType](#) > [sea_y](#)
- [PointCloud](#)< [DataType](#) > [sea_cloud](#)
- [PointCloud](#)< [DataType](#) > [land_cloud](#)
- [std::unique_ptr](#)< [KDTree](#)< [DataType](#) > > [kd_sea](#) {nullptr}
- [std::unique_ptr](#)< [KDTree](#)< [DataType](#) > > [kd_land](#) {nullptr}

5.12.1 Detailed Description

Definition at line 606 of file [data_prep_utils.hpp](#).

5.12.2 Member Data Documentation

5.12.2.1 grid_1

```
Grid<DataType> particle_tracker::MonthData::grid_1
```

Definition at line 612 of file [data_prep_utils.hpp](#).

5.12.2.2 grid_2

```
Grid<DataType> particle_tracker::MonthData::grid_2
```

Definition at line 612 of file [data_prep_utils.hpp](#).

5.12.2.3 kd_land

```
std::unique_ptr<KDTree<DataType> > particle_tracker::MonthData::kd_land {nullptr}
```

Definition at line 617 of file [data_prep_utils.hpp](#).

5.12.2.4 kd_sea

```
std::unique_ptr<KDTree<DataType> > particle_tracker::MonthData::kd_sea {nullptr}
```

Definition at line 616 of file [data_prep_utils.hpp](#).

5.12.2.5 land_cloud

```
PointCloud<DataType> particle_tracker::MonthData::land_cloud
```

Definition at line 615 of file [data_prep_utils.hpp](#).

5.12.2.6 month

```
Date particle_tracker::MonthData::month {1, 1, 2000}
```

Definition at line 608 of file [data_prep_utils.hpp](#).

5.12.2.7 sea_cloud

```
PointCloud<DataType> particle_tracker::MonthData::sea_cloud
```

Definition at line 615 of file [data_prep_utils.hpp](#).

5.12.2.8 sea_idx_1

```
std::vector<std::size_t> particle_tracker::MonthData::sea_idx_1
```

Definition at line 613 of file [data_prep_utils.hpp](#).

5.12.2.9 sea_idx_2

```
std::vector<std::size_t> particle_tracker::MonthData::sea_idx_2
```

Definition at line 613 of file [data_prep_utils.hpp](#).

5.12.2.10 sea_x

```
std::vector<DataType> particle_tracker::MonthData::sea_x
```

Definition at line 614 of file [data_prep_utils.hpp](#).

5.12.2.11 sea_y

```
std::vector<DataType> particle_tracker::MonthData::sea_y
```

Definition at line 614 of file [data_prep_utils.hpp](#).

5.12.2.12 spl

```
spline_map<DataType> particle_tracker::MonthData::spl
```

Definition at line 611 of file [data_prep_utils.hpp](#).

5.12.2.13 spl_1

```
spline_map<DataType> particle_tracker::MonthData::spl_1
```

Definition at line 609 of file [data_prep_utils.hpp](#).

5.12.2.14 spl_2

```
spline_map<DataType> particle_tracker::MonthData::spl_2
```

Definition at line 610 of file [data_prep_utils.hpp](#).

The documentation for this struct was generated from the following file:

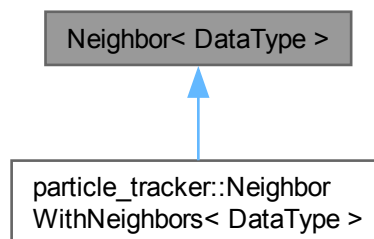
- [include/particle_tracker/data_prep_utils.hpp](#)

5.13 Neighbor< DataType > Struct Template Reference

One spatial neighbor (grid node) used in interpolation.

```
#include <interpolation_utils.hpp>
```

Inheritance diagram for Neighbor< DataType >:



Public Member Functions

- [Neighbor](#) (std::size_t index=0, DataType distance=DataType{}, DataType node_weight=DataType{})

Public Attributes

- std::size_t [ind](#) {}
- DataType [dist](#) {}
- DataType [weight](#) {}

5.13.1 Detailed Description

```
template<typename DataType>
struct Neighbor< DataType >
```

One spatial neighbor (grid node) used in interpolation.

Template Parameters

<i>DataType</i>	numeric type.
-----------------	---------------

Fields:

- `ind`: index into grid coordinate arrays (`xx_sparse/yy_sparse`) and per-node data arrays.
- `dist`: great-circle distance (from `GCD`), **not** KD-tree metric distance.
- `weight`: Gaussian weight, typically `exp(-(dist*dist)/shape)`.

Note

This is a lightweight, POD-like aggregate in practice (trivially copyable if `DataType` is), but it has a user-provided constructor, so it is **not** a "POD type" in the strict C++03 sense. In modern C++, the relevant traits are:

- `standard-layout`: yes (for typical `DataType`)
- `trivially copyable`: yes (for typical `DataType`)

Definition at line 417 of file [interpolation_utils.hpp](#).

5.13.2 Constructor & Destructor Documentation

5.13.2.1 Neighbor()

```
template<typename DataType>
particle_tracker::Neighbor< DataType >::Neighbor (
    std::size_t index = 0,
    DataType distance = DataType{},
    DataType node_weight = DataType{}) [inline]
```

Definition at line 422 of file [interpolation_utils.hpp](#).

5.13.3 Member Data Documentation

5.13.3.1 dist

```
template<typename DataType>
DataType particle_tracker::Neighbor< DataType >::dist {}
```

Definition at line 419 of file [interpolation_utils.hpp](#).

5.13.3.2 ind

```
template<typename DataType>
std::size_t particle_tracker::Neighbor< DataType >::ind {}
```

Definition at line 418 of file [interpolation_utils.hpp](#).

5.13.3.3 weight

```
template<typename DataType>
DataType particle_tracker::Neighbor< DataType >::weight {}
```

Definition at line 420 of file [interpolation_utils.hpp](#).

The documentation for this struct was generated from the following file:

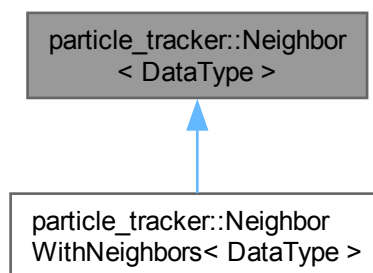
- [include/particle_tracker/interpolation_utils.hpp](#)

5.14 particle_tracker::Neighbor< DataType > Struct Template Reference

One spatial neighbor (grid node) used in interpolation.

```
#include <interpolation_utils.hpp>
```

Inheritance diagram for `particle_tracker::Neighbor< DataType >`:



Public Member Functions

- [Neighbor](#) (std::size_t index=0, DataType distance=DataType{}, DataType node_weight=DataType{})

Public Attributes

- std::size_t [ind](#) {}
- DataType [dist](#) {}
- DataType [weight](#) {}

5.14.1 Detailed Description

```
template<typename DataType>
struct particle_tracker::Neighbor< DataType >
```

One spatial neighbor (grid node) used in interpolation.

Template Parameters

<i>DataType</i>	numeric type.
-----------------	---------------

Fields:

- `ind`: index into grid coordinate arrays (`xx_sparse/yy_sparse`) and per-node data arrays.
- `dist`: great-circle distance (from `GCD`), **not** KD-tree metric distance.
- `weight`: Gaussian weight, typically `exp(-(dist*dist)/shape)`.

Note

This is a lightweight, POD-like aggregate in practice (trivially copyable if `DataType` is), but it has a user-provided constructor, so it is **not** a "POD type" in the strict C++03 sense. In modern C++, the relevant traits are:

- `standard-layout`: yes (for typical `DataType`)
- `trivially copyable`: yes (for typical `DataType`)

Definition at line 417 of file [interpolation_utils.hpp](#).

5.14.2 Constructor & Destructor Documentation

5.14.2.1 Neighbor()

```
template<typename DataType>
particle_tracker::Neighbor< DataType >::Neighbor (
    std::size_t index = 0,
    DataType distance = DataType{},
    DataType node_weight = DataType{}) [inline]
```

Definition at line 422 of file [interpolation_utils.hpp](#).

5.14.3 Member Data Documentation

5.14.3.1 dist

```
template<typename DataType>
DataType particle_tracker::Neighbor< DataType >::dist {}
```

Definition at line 419 of file [interpolation_utils.hpp](#).

5.14.3.2 ind

```
template<typename DataType>
std::size_t particle_tracker::Neighbor< DataType >::ind {}
```

Definition at line 418 of file [interpolation_utils.hpp](#).

5.14.3.3 weight

```
template<typename DataType>
DataType particle_tracker::Neighbor< DataType >::weight {}
```

Definition at line 420 of file [interpolation_utils.hpp](#).

The documentation for this struct was generated from the following file:

- [include/particle_tracker/interpolation_utils.hpp](#)

5.15 particle_tracker::NeighborPrime< DataType > Struct Template Reference

A "secondary neighbor" (neighbor of a primary neighbor).

```
#include <interpolation_utils.hpp>
```

Public Member Functions

- [NeighborPrime](#) (std::size_t index_prime=0, std::array< DataType, 3 > distance_prime={DataType{}, DataType{}, DataType{}}, DataType node_weight_prime=DataType{})

Public Attributes

- std::size_t [ind_prime](#) {}
- std::array< DataType, 3 > [dist_prime](#) {}
- DataType [weight_prime](#) {}

5.15.1 Detailed Description

```
template<typename DataType>
struct particle_tracker::NeighborPrime< DataType >
```

A "secondary neighbor" (neighbor of a primary neighbor).

Template Parameters

<i>DataType</i>	numeric type.
-----------------	---------------

Fields:

- `ind_prime`: index of the secondary grid node.
- `dist_prime`: array [d, ddx, ddy] returned by `GCD_deriv(primary, secondary)`.
 - `dist_prime[0]` = d : great-circle distance
 - `dist_prime[1]` = ddx : partial derivative of distance w.r.t. x-like coordinate
 - `dist_prime[2]` = ddy : partial derivative of distance w.r.t. y-like coordinate
- `weight_prime`: Gaussian weight $\exp(-(d*d)/shape)$ based on `dist_prime[0]`.

Definition at line 638 of file [interpolation_utils.hpp](#).

5.15.2 Constructor & Destructor Documentation

5.15.2.1 NeighborPrime()

```
template<typename DataType>
particle_tracker::NeighborPrime< DataType >::NeighborPrime (
    std::size_t index_prime = 0,
    std::array< DataType, 3 > distance_prime = {DataType{}, DataType{}, DataType{}},
    DataType node_weight_prime = DataType{}) [inline]
```

Definition at line 643 of file [interpolation_utils.hpp](#).

5.15.3 Member Data Documentation

5.15.3.1 dist_prime

```
template<typename DataType>
std::array<DataType, 3> particle_tracker::NeighborPrime< DataType >::dist_prime {}
```

Definition at line 640 of file [interpolation_utils.hpp](#).

5.15.3.2 ind_prime

```
template<typename DataType>
std::size_t particle_tracker::NeighborPrime< DataType >::ind_prime {}
```

Definition at line 639 of file [interpolation_utils.hpp](#).

5.15.3.3 weight_prime

```
template<typename DataType>
DataType particle_tracker::NeighborPrime< DataType >::weight_prime {}
```

Definition at line 641 of file [interpolation_utils.hpp](#).

The documentation for this struct was generated from the following file:

- include/particle_tracker/[interpolation_utils.hpp](#)

5.16 NeighborsData< DataType, TimeType > Class Template Reference

Spatial neighbor query + Gaussian weighting + multi-variable interpolation.

```
#include <interpolation_utils.hpp>
```

Public Member Functions

- [NeighborsData](#) (const [KDTree](#)< DataType > &idx, const std::vector< DataType > &xxs, const std::vector< DataType > &yys, DataType r2, DataType shape_param)
- void [computeNeighbors](#) (std::vector< [Neighbor](#)< DataType > > &neighbors, DataType x, DataType y)
Compute neighbor list around query point (x,y).
- void [updateNeighborWeights](#) (std::vector< [Neighbor](#)< DataType > > &neighbors, DataType x, DataType y)
Recompute dist/weight for an existing neighbor list without changing indices.
- std::map< std::string, DataType > [interpolateVariables](#) (TimeType t, const std::vector< [Neighbor](#)< DataType > > &neighbors, const std::map< std::string, std::vector< [TimeSpline](#)< DataType, TimeType > > > &variable_splines, const std::vector< std::string > &variable_names)
Weighted spatial interpolation for multiple variables at time t.
- std::map< std::string, DataType > [calculateVariables](#) (TimeType t, std::vector< [Neighbor](#)< DataType > > &neighbors, const std::map< std::string, std::vector< [TimeSpline](#)< DataType, TimeType > > > &variable_splines, const std::vector< std::string > &variable_names, DataType x, DataType y)
Convenience: computeNeighbors + interpolateVariables.
- std::map< std::string, DataType > [calculateVariablesUpdate](#) (TimeType t, std::vector< [Neighbor](#)< DataType > > &neighbors, const std::map< std::string, std::vector< [TimeSpline](#)< DataType, TimeType > > > &variable_splines, const std::vector< std::string > &variable_names, DataType x, DataType y)
Convenience: updateNeighborWeights + interpolateVariables.

5.16.1 Detailed Description

```
template<typename DataType, typename TimeType = double>
class NeighborsData< DataType, TimeType >
```

Spatial neighbor query + Gaussian weighting + multi-variable interpolation.

Template Parameters

<i>DataType</i>	numeric type.
<i>TimeType</i>	time type.

This class holds references to:

- a pre-built [KDTree](#)
- grid longitudes and latitudes (`xx_sparse`, `yy_sparse`) and parameters:
- `rad2` : squared KD-tree search radius (approximate metric)
- `shape` : Gaussian length scale for weights in terms of **great-circle distance**

5.16.1.0.1 Dateline policy

- Query longitude is wrapped into $(-\pi, \pi]$ before search.
- Grid longitudes are wrapped before calling `GCD()` to avoid discontinuities if your `GCD()` implementation expects normalized longitudes.

Definition at line [450](#) of file [interpolation_utils.hpp](#).

5.16.2 Constructor & Destructor Documentation**5.16.2.1 NeighborsData()**

```
template<typename DataType, typename TimeType = double>
particle_tracker::NeighborsData< DataType, TimeType >::NeighborsData (
    const KDTree< DataType > & idx,
    const std::vector< DataType > & xxs,
    const std::vector< DataType > & yys,
    DataType r2,
    DataType shape_param) [inline]
```

Definition at line [459](#) of file [interpolation_utils.hpp](#).

5.16.3 Member Function Documentation**5.16.3.1 calculateVariables()**

```
template<typename DataType, typename TimeType = double>
std::map< std::string, DataType > particle_tracker::NeighborsData< DataType, TimeType >::↔
calculateVariables (
    TimeType t,
    std::vector< Neighbor< DataType > > & neighbors,
    const std::map< std::string, std::vector< TimeSpline< DataType, TimeType > > >
& variable_splines,
    const std::vector< std::string > & variable_names,
    DataType x,
    DataType y) [inline]
```

Convenience: `computeNeighbors + interpolateVariables`.

Definition at line [581](#) of file [interpolation_utils.hpp](#).

5.16.3.2 calculateVariablesUpdate()

```
template<typename DataType, typename TimeType = double>
std::map< std::string, DataType > particle_tracker::NeighborsData< DataType, TimeType >::↔
calculateVariablesUpdate (
    TimeType t,
    std::vector< Neighbor< DataType > > & neighbors,
    const std::map< std::string, std::vector< TimeSpline< DataType, TimeType > > >
& variable_splines,
    const std::vector< std::string > & variable_names,
    DataType x,
    DataType y) [inline]
```

Convenience: updateNeighborWeights + interpolateVariables.

Definition at line 593 of file [interpolation_utils.hpp](#).

5.16.3.3 computeNeighbors()

```
template<typename DataType, typename TimeType = double>
void particle_tracker::NeighborsData< DataType, TimeType >::computeNeighbors (
    std::vector< Neighbor< DataType > > & neighbors,
    DataType x,
    DataType y) [inline]
```

Compute neighbor list around query point (x,y).

- Uses KD-tree radius search with rad2 (approx metric).
- Computes true great-circle distance via GCD for each candidate.
- Computes Gaussian weights from true distance.

Parameters

<i>neighbors</i>	output vector cleared and filled.
<i>x</i>	query longitude (radians)
<i>y</i>	query latitude (radians)

Definition at line 476 of file [interpolation_utils.hpp](#).

5.16.3.4 interpolateVariables()

```
template<typename DataType, typename TimeType = double>
std::map< std::string, DataType > particle_tracker::NeighborsData< DataType, TimeType >::↔
interpolateVariables (
    TimeType t,
    const std::vector< Neighbor< DataType > > & neighbors,
    const std::map< std::string, std::vector< TimeSpline< DataType, TimeType > > >
& variable_splines,
    const std::vector< std::string > & variable_names) [inline]
```

Weighted spatial interpolation for multiple variables at time t .

Parameters

t	evaluation time
<i>neighbors</i>	neighbor list produced by <code>computeNeighbors/updateNeighborWeights</code>
<i>variable_splines</i>	<code>map(var_name -> spline-per-gridpoint)</code>
<i>variable_names</i>	variables to evaluate

Returns

`map(var_name -> interpolated value)`

Definition at line 539 of file [interpolation_utils.hpp](#).

5.16.3.5 `updateNeighborWeights()`

```
template<typename DataType, typename TimeType = double>
void particle_tracker::NeighborsData< DataType, TimeType >::updateNeighborWeights (
    std::vector< Neighbor< DataType > > & neighbors,
    DataType x,
    DataType y) [inline]
```

Recompute dist/weight for an existing neighbor list without changing indices.

Use this only if the query point moved *slightly* the neighbor identity may no longer be optimal. For larger moves, call [computeNeighbors](#) again.

Definition at line 515 of file [interpolation_utils.hpp](#).

The documentation for this class was generated from the following file:

- [include/particle_tracker/interpolation_utils.hpp](#)

5.17 `particle_tracker::NeighborsData< DataType, TimeType >` Class Template Reference

Spatial neighbor query + Gaussian weighting + multi-variable interpolation.

```
#include <interpolation_utils.hpp>
```

Public Member Functions

- [NeighborsData](#) (const [KDTree](#)< DataType > &idx, const std::vector< DataType > &xxs, const std::vector< DataType > &yys, DataType r2, DataType shape_param)
- void [computeNeighbors](#) (std::vector< [Neighbor](#)< DataType > > &neighbors, DataType x, DataType y)
Compute neighbor list around query point (x,y).
- void [updateNeighborWeights](#) (std::vector< [Neighbor](#)< DataType > > &neighbors, DataType x, DataType y)
Recompute dist/weight for an existing neighbor list without changing indices.
- std::map< std::string, DataType > [interpolateVariables](#) (TimeType t, const std::vector< [Neighbor](#)< DataType > > &neighbors, const std::map< std::string, std::vector< [TimeSpline](#)< DataType, TimeType > > > &variable_splines, const std::vector< std::string > &variable_names)
Weighted spatial interpolation for multiple variables at time t.
- std::map< std::string, DataType > [calculateVariables](#) (TimeType t, std::vector< [Neighbor](#)< DataType > > &neighbors, const std::map< std::string, std::vector< [TimeSpline](#)< DataType, TimeType > > > &variable_splines, const std::vector< std::string > &variable_names, DataType x, DataType y)
Convenience: computeNeighbors + interpolateVariables.
- std::map< std::string, DataType > [calculateVariablesUpdate](#) (TimeType t, std::vector< [Neighbor](#)< DataType > > &neighbors, const std::map< std::string, std::vector< [TimeSpline](#)< DataType, TimeType > > > &variable_splines, const std::vector< std::string > &variable_names, DataType x, DataType y)
Convenience: updateNeighborWeights + interpolateVariables.

5.17.1 Detailed Description

template<typename DataType, typename TimeType = double>
class particle_tracker::NeighborsData< DataType, TimeType >

Spatial neighbor query + Gaussian weighting + multi-variable interpolation.

Template Parameters

<i>DataType</i>	numeric type.
<i>TimeType</i>	time type.

This class holds references to:

- a pre-built [KDTree](#)
- grid longitudes and latitudes (xx_sparse, yy_sparse) and parameters:
- `rad2` : squared KD-tree search radius (approximate metric)
- `shape` : Gaussian length scale for weights in terms of **great-circle distance**

5.17.1.0.1 Dateline policy

- Query longitude is wrapped into $(-\pi, \pi]$ before search.
- [Grid](#) longitudes are wrapped before calling [GCD\(\)](#) to avoid discontinuities if your [GCD\(\)](#) implementation expects normalized longitudes.

Definition at line 450 of file [interpolation_utils.hpp](#).

5.17.2 Constructor & Destructor Documentation

5.17.2.1 `NeighborsData()`

```
template<typename DataType, typename TimeType = double>
particle_tracker::NeighborsData< DataType, TimeType >::NeighborsData (
    const KDTree< DataType > & idx,
    const std::vector< DataType > & xxs,
    const std::vector< DataType > & yys,
    DataType r2,
    DataType shape_param) [inline]
```

Definition at line 459 of file [interpolation_utils.hpp](#).

5.17.3 Member Function Documentation

5.17.3.1 `calculateVariables()`

```
template<typename DataType, typename TimeType = double>
std::map< std::string, DataType > particle_tracker::NeighborsData< DataType, TimeType >::↔
calculateVariables (
    TimeType t,
    std::vector< Neighbor< DataType > > & neighbors,
    const std::map< std::string, std::vector< TSpline< DataType, TimeType > > >
& variable_splines,
    const std::vector< std::string > & variable_names,
    DataType x,
    DataType y) [inline]
```

Convenience: `computeNeighbors` + `interpolateVariables`.

Definition at line 581 of file [interpolation_utils.hpp](#).

References [computeNeighbors\(\)](#), and [interpolateVariables\(\)](#).

5.17.3.2 `calculateVariablesUpdate()`

```
template<typename DataType, typename TimeType = double>
std::map< std::string, DataType > particle_tracker::NeighborsData< DataType, TimeType >::↔
calculateVariablesUpdate (
    TimeType t,
    std::vector< Neighbor< DataType > > & neighbors,
    const std::map< std::string, std::vector< TSpline< DataType, TimeType > > >
& variable_splines,
    const std::vector< std::string > & variable_names,
    DataType x,
    DataType y) [inline]
```

Convenience: `updateNeighborWeights` + `interpolateVariables`.

Definition at line 593 of file [interpolation_utils.hpp](#).

References [interpolateVariables\(\)](#), and [updateNeighborWeights\(\)](#).

5.17.3.3 computeNeighbors()

```
template<typename DataType, typename TimeType = double>
void particle\_tracker::NeighborsData< DataType, TimeType >::computeNeighbors (
    std::vector< Neighbor< DataType > > & neighbors,
    DataType x,
    DataType y) [inline]
```

Compute neighbor list around query point (x,y).

- Uses KD-tree radius search with `rad2` (approx metric).
- Computes true great-circle distance via `GCD` for each candidate.
- Computes Gaussian weights from true distance.

Parameters

<i>neighbors</i>	output vector cleared and filled.
<i>x</i>	query longitude (radians)
<i>y</i>	query latitude (radians)

Definition at line 476 of file [interpolation_utils.hpp](#).

Referenced by [calculateVariables\(\)](#).

5.17.3.4 interpolateVariables()

```
template<typename DataType, typename TimeType = double>
std::map< std::string, DataType > particle\_tracker::NeighborsData< DataType, TimeType >::↔
interpolateVariables (
    TimeType t,
    const std::vector< Neighbor< DataType > > & neighbors,
    const std::map< std::string, std::vector< TimeSpline< DataType, TimeType > > >
& variable_splines,
    const std::vector< std::string > & variable_names) [inline]
```

Weighted spatial interpolation for multiple variables at time t.

Parameters

<i>t</i>	evaluation time
<i>neighbors</i>	neighbor list produced by <code>computeNeighbors/updateNeighborWeights</code>
<i>variable_splines</i>	map(var_name -> spline-per-gridpoint)
<i>variable_names</i>	variables to evaluate

Returns

map(var_name -> interpolated value)

Definition at line 539 of file [interpolation_utils.hpp](#).

Referenced by [calculateVariables\(\)](#), and [calculateVariablesUpdate\(\)](#).

5.17.3.5 updateNeighborWeights()

```
template<typename DataType, typename TimeType = double>
void particle_tracker::NeighborsData< DataType, TimeType >::updateNeighborWeights (
    std::vector< Neighbor< DataType > > & neighbors,
    DataType x,
    DataType y) [inline]
```

Recompute dist/weight for an existing neighbor list without changing indices.

Use this only if the query point moved *slightly* the neighbor identity may no longer be optimal. For larger moves, call [computeNeighbors](#) again.

Definition at line 515 of file [interpolation_utils.hpp](#).

Referenced by [calculateVariablesUpdate\(\)](#).

The documentation for this class was generated from the following file:

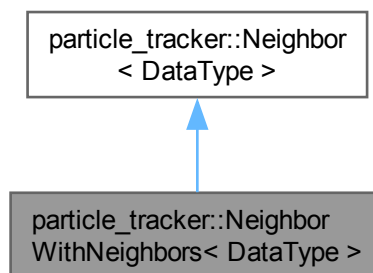
- include/particle_tracker/[interpolation_utils.hpp](#)

5.18 particle_tracker::NeighborWithNeighbors< DataType > Struct Template Reference

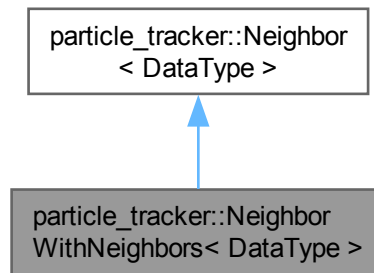
A primary [Neighbor](#) plus a list of its secondary neighbors.

```
#include <interpolation_utils.hpp>
```

Inheritance diagram for particle_tracker::NeighborWithNeighbors< DataType >:



Collaboration diagram for `particle_tracker::NeighborWithNeighbors< DataType >`:



Public Member Functions

- [NeighborWithNeighbors](#) (`std::size_t index`, `DataType distance`, `DataType node_weight`, `const std::vector< NeighborPrime< DataType > > &nbr_prime`)

Public Member Functions inherited from [particle_tracker::Neighbor< DataType >](#)

- [Neighbor](#) (`std::size_t index=0`, `DataType distance=DataType{}`, `DataType node_weight=DataType{}`)

Public Attributes

- `std::vector< NeighborPrime< DataType > >` [neighbors_prime](#)

Public Attributes inherited from [particle_tracker::Neighbor< DataType >](#)

- `std::size_t` [ind](#) {}
- `DataType` [dist](#) {}
- `DataType` [weight](#) {}

5.18.1 Detailed Description

```
template<typename DataType>
struct particle_tracker::NeighborWithNeighbors< DataType >
```

A primary [Neighbor](#) plus a list of its secondary neighbors.

Used for the experimental derivative estimator.

Definition at line 658 of file [interpolation_utils.hpp](#).

5.18.2 Constructor & Destructor Documentation

5.18.2.1 NeighborWithNeighbors()

```
template<typename DataType>
particle_tracker::NeighborWithNeighbors< DataType >::NeighborWithNeighbors (
    std::size_t index,
    DataType distance,
    DataType node_weight,
    const std::vector< NeighborPrime< DataType > > & nbr_prime) [inline]
```

Definition at line 663 of file [interpolation_utils.hpp](#).

5.18.3 Member Data Documentation

5.18.3.1 neighbors_prime

```
template<typename DataType>
std::vector<NeighborPrime<DataType> > particle_tracker::NeighborWithNeighbors< DataType
>::neighbors_prime
```

Definition at line 659 of file [interpolation_utils.hpp](#).

The documentation for this struct was generated from the following file:

- [include/particle_tracker/interpolation_utils.hpp](#)

5.19 particle_tracker::NeighborWithNeighborsData< DataType, TimeType > Class Template Reference

Two-level neighborhood builder + heuristic spatial derivative estimator.

```
#include <interpolation_utils.hpp>
```

Public Member Functions

- [NeighborWithNeighborsData](#) (const [KDTree](#)< DataType > &idx, const std::vector< DataType > &xxs, const std::vector< DataType > &yys, DataType r2, DataType shape_param)
- void [computeNeighbors](#) (std::vector< [NeighborWithNeighbors](#)< DataType > > &neighbors, DataType x, DataType y)

Build primary+secondary neighborhoods around (x,y).
- void [updateNeighborWeights](#) (std::vector< [NeighborWithNeighbors](#)< DataType > > &neighbors, DataType x, DataType y)

Update only primary (query->primary) distances and weights for an existing neighbor list.
- std::map< std::string, [VariableDerivatives](#)< DataType > > [interpolateDerivatives](#) (TimeType t, const std::vector< [NeighborWithNeighbors](#)< DataType > > &neighbors, const std::map< std::string, std::vector< [TimeSpline](#)< DataType, TimeType > > > &variable_splines, const std::vector< std::string > &variable_names, DataType shape) const

Interpolate value/time-derivative and estimate spatial derivatives for requested variables.

- `std::map< std::string, VariableDerivatives< DataType > > calculateDerivatives` (TimeType t, `std::vector< NeighborWithNeighbors< DataType > >` &neighbors, `const std::map< std::string, std::vector< TimeSpline< DataType, TimeType > > >` &variable_splines, `const std::vector< std::string >` &variable_names, DataType x, DataType y, DataType shape)

Convenience: computeNeighbors + interpolateDerivatives.

- `std::map< std::string, VariableDerivatives< DataType > > calculateDerivativesUpdate` (TimeType t, `std::vector< NeighborWithNeighbors< DataType > >` &neighbors, `const std::map< std::string, std::vector< TimeSpline< DataType, TimeType > > >` &variable_splines, `const std::vector< std::string >` &variable_names, DataType x, DataType y, DataType shape)

Convenience: updateNeighborWeights + interpolateDerivatives.

5.19.1 Detailed Description

```
template<typename DataType, typename TimeType = double>
class particle_tracker::NeighborWithNeighborsData< DataType, TimeType >
```

Two-level neighborhood builder + heuristic spatial derivative estimator.

Template Parameters

<i>DataType</i>	numeric type.
<i>TimeType</i>	time type.

5.19.1.1 Two-level neighborhood

1) Find primary neighbors around the query point (x,y). 2) For each primary neighbor i, find secondary neighbors around grid point i.

5.19.1.2 Derivative estimator (heuristic)

- For each primary neighbor i:
 - For each secondary neighbor j:
 - * $dv = v_j - v_i$
 - * accumulate using $weight_prime * (d * ddx * dv)$ and $(d * ddy * dv)$
- Normalize per-primary-neighbor by sum of $weight_prime$ (den_prime)
- Average over primary neighbors with primary weights
- Apply final scaling factor ($2/shape$)

5.19.1.3 Dateline policy

- Query longitude x is wrapped into $(-\pi, \pi]$.
- [Grid](#) longitudes are wrapped before calling `GCD` and `GCD_deriv`.

5.19.1.4 Loud failure behavior

- If secondary neighborhood search returns empty for a primary neighbor, the primary neighbor is **skipped** (because derivatives cannot be formed). This changes the primary set. This is intentional and explicit; it prevents silent garbage derivatives.

Definition at line 718 of file [interpolation_utils.hpp](#).

5.19.2 Constructor & Destructor Documentation

5.19.2.1 `NeighborWithNeighborsData()`

```
template<typename DataType, typename TimeType = double>
particle_tracker::NeighborWithNeighborsData< DataType, TimeType >::NeighborWithNeighborsData (
    const KDTree< DataType > & idx,
    const std::vector< DataType > & xxs,
    const std::vector< DataType > & yys,
    DataType r2,
    DataType shape_param) [inline]
```

Definition at line 727 of file [interpolation_utils.hpp](#).

5.19.3 Member Function Documentation

5.19.3.1 `calculateDerivatives()`

```
template<typename DataType, typename TimeType = double>
std::map< std::string, VariableDerivatives< DataType > > particle_tracker::NeighborWithNeighborsData<
DataType, TimeType >::calculateDerivatives (
    TimeType t,
    std::vector< NeighborWithNeighbors< DataType > > & neighbors,
    const std::map< std::string, std::vector< TimeSpline< DataType, TimeType > > >
& variable_splines,
    const std::vector< std::string > & variable_names,
    DataType x,
    DataType y,
    DataType shape) [inline]
```

Convenience: `computeNeighbors` + `interpolateDerivatives`.

Definition at line 971 of file [interpolation_utils.hpp](#).

References [computeNeighbors\(\)](#), and [interpolateDerivatives\(\)](#).

5.19.3.2 calculateDerivativesUpdate()

```
template<typename DataType, typename TimeType = double>
std::map< std::string, VariableDerivatives< DataType > > particle_tracker::NeighborWithNeighborsData<
DataType, TimeType >::calculateDerivativesUpdate (
    TimeType t,
    std::vector< NeighborWithNeighbors< DataType > > & neighbors,
    const std::map< std::string, std::vector< TimeSpline< DataType, TimeType > > >
& variable_splines,
    const std::vector< std::string > & variable_names,
    DataType x,
    DataType y,
    DataType shape) [inline]
```

Convenience: updateNeighborWeights + interpolateDerivatives.

Definition at line 986 of file [interpolation_utils.hpp](#).

References [interpolateDerivatives\(\)](#), and [updateNeighborWeights\(\)](#).

5.19.3.3 computeNeighbors()

```
template<typename DataType, typename TimeType = double>
void particle_tracker::NeighborWithNeighborsData< DataType, TimeType >::computeNeighbors (
    std::vector< NeighborWithNeighbors< DataType > > & neighbors,
    DataType x,
    DataType y) [inline]
```

Build primary+secondary neighborhoods around (x,y).

Parameters

<i>neighbors</i>	output vector cleared and filled
<i>x</i>	query longitude (radians)
<i>y</i>	query latitude (radians)

Definition at line 741 of file [interpolation_utils.hpp](#).

Referenced by [calculateDerivatives\(\)](#).

5.19.3.4 interpolateDerivatives()

```
template<typename DataType, typename TimeType = double>
std::map< std::string, VariableDerivatives< DataType > > particle_tracker::NeighborWithNeighborsData<
DataType, TimeType >::interpolateDerivatives (
    TimeType t,
    const std::vector< NeighborWithNeighbors< DataType > > & neighbors,
    const std::map< std::string, std::vector< TimeSpline< DataType, TimeType > > >
& variable_splines,
    const std::vector< std::string > & variable_names,
    DataType shape) const [inline]
```

Interpolate value/time-derivative and estimate spatial derivatives for requested variables.

Parameters

<i>t</i>	evaluation time
<i>neighbors</i>	two-level neighborhood list
<i>variable_splines</i>	map(var_name -> spline-per-gridpoint)
<i>variable_names</i>	which variables to compute
<i>shape</i>	normalization length scale used in final scaling (2/shape)

Returns

map(var_name -> [VariableDerivatives](#))

Warning

The spatial derivatives are heuristic and must be validated.

Definition at line 848 of file [interpolation_utils.hpp](#).

Referenced by [calculateDerivatives\(\)](#), and [calculateDerivativesUpdate\(\)](#).

5.19.3.5 `updateNeighborWeights()`

```
template<typename DataType, typename TimeType = double>
void particle_tracker::NeighborWithNeighborsData< DataType, TimeType >::updateNeighborWeights
(
    std::vector< NeighborWithNeighbors< DataType > > & neighbors,
    DataType x,
    DataType y) [inline]
```

Update only primary (query->primary) distances and weights for an existing neighbor list.

Secondary neighborhoods and their weights are not recomputed.

Definition at line 821 of file [interpolation_utils.hpp](#).

Referenced by [calculateDerivativesUpdate\(\)](#).

The documentation for this class was generated from the following file:

- include/particle_tracker/[interpolation_utils.hpp](#)

5.20 `particle_tracker::NetCDFDataLoader< DataType, TimeType >` Class Template Reference

Load one or more NetCDF files and store selected variables keyed by time.

```
#include <ncdf_utils.hpp>
```

Public Types

- using [DataMap](#) = std::map<TimeType, std::vector<DataType>>

Public Member Functions

- [NetCDFDataLoader](#) (const std::vector< std::string > &file_paths, const std::vector< std::string > &variable_names)
- void [loadData](#) ()
- const DataMap & [getVariableData](#) (const std::string &variable_name) const
- const std::vector< std::size_t > & [getVariableShape](#) (const std::string &variable_name) const
- const std::string & [getVariableType](#) (const std::string &variable_name) const
- const std::vector< TimeType > & [getTimeValues](#) (const std::string &file_path) const
- std::vector< DataType > [getVariableDataAtIndex](#) (const std::string &var_name, std::size_t index) const
- netCDF::NcVar [getNetCDFVariable](#) (const std::string &variable_name) const

Return a netCDF variable handle (for metadata/attributes).

5.20.1 Detailed Description

```
template<typename DataType, typename TimeType = double>
class particle_tracker::NetCDFDataLoader< DataType, TimeType >
```

Load one or more NetCDF files and store selected variables keyed by time.

Template Parameters

<i>DataType</i>	numeric output type (float/double recommended)
<i>TimeType</i>	time type (double by default; often integer seconds are stored after quantization)

Data are stored as: data_[var_name][time_value] -> std::vector<DataType>

Each vector contains the flattened variable slice for that time.

5.20.1.0.1 Error policy

- Per-variable failures do not abort loadData(): they warn (optional) and continue.
- Accessors throw if requested data were not loaded.
- In strict mode (PARTICLE_TRACKER_NCDF_STRICT=1), certain structural issues throw.

Definition at line 127 of file [ncdf_utils.hpp](#).

5.20.2 Member Typedef Documentation

5.20.2.1 DataMap

```
template<typename DataType, typename TimeType = double>
using particle_tracker::NetCDFDataLoader< DataType, TimeType >::DataMap = std::map<TimeType,
std::vector<DataType>>
```

Definition at line 130 of file [ncdf_utils.hpp](#).

5.20.3 Constructor & Destructor Documentation

5.20.3.1 NetCDFDataLoader()

```
template<typename DataType, typename TimeType = double>
particle_tracker::NetCDFDataLoader< DataType, TimeType >::NetCDFDataLoader (
    const std::vector< std::string > & file_paths,
    const std::vector< std::string > & variable_names) [inline]
```

Definition at line 132 of file [ncdf_utils.hpp](#).

5.20.4 Member Function Documentation

5.20.4.1 getNetCDFVariable()

```
template<typename DataType, typename TimeType = double>
netCDF::NcVar particle_tracker::NetCDFDataLoader< DataType, TimeType >::getNetCDFVariable (
    const std::string & variable_name) const [inline]
```

Return a netCDF variable handle (for metadata/attributes).

Note:

- This opens the first file from `file_paths_` and returns a variable from it.
- If per-file metadata are needed, open that file explicitly in caller code.

Definition at line 202 of file [ncdf_utils.hpp](#).

5.20.4.2 getTimeValues()

```
template<typename DataType, typename TimeType = double>
const std::vector< TimeType > & particle_tracker::NetCDFDataLoader< DataType, TimeType >::←
getTimeValues (
    const std::string & file_path) const [inline]
```

Definition at line 172 of file [ncdf_utils.hpp](#).

5.20.4.3 getVariableData()

```
template<typename DataType, typename TimeType = double>
const DataMap & particle_tracker::NetCDFDataLoader< DataType, TimeType >::getVariableData (
    const std::string & variable_name) const [inline]
```

Definition at line 145 of file [ncdf_utils.hpp](#).

5.20.4.4 `getVariableDataAtIndex()`

```
template<typename DataType, typename TimeType = double>
std::vector< DataType > particle\_tracker::NetCDFDataLoader< DataType, TimeType >::getVariableDataAtIndex (
    const std::string & var_name,
    std::size_t index) const [inline]
```

Definition at line 181 of file [ncdf_utils.hpp](#).

5.20.4.5 `getVariableShape()`

```
template<typename DataType, typename TimeType = double>
const std::vector< std::size_t > & particle\_tracker::NetCDFDataLoader< DataType, TimeType >::getVariableShape (
    const std::string & variable_name) const [inline]
```

Definition at line 154 of file [ncdf_utils.hpp](#).

5.20.4.6 `getVariableType()`

```
template<typename DataType, typename TimeType = double>
const std::string & particle\_tracker::NetCDFDataLoader< DataType, TimeType >::getVariableType (
    const std::string & variable_name) const [inline]
```

Definition at line 163 of file [ncdf_utils.hpp](#).

5.20.4.7 `loadData()`

```
template<typename DataType, typename TimeType = double>
void particle\_tracker::NetCDFDataLoader< DataType, TimeType >::loadData () [inline]
```

Definition at line 137 of file [ncdf_utils.hpp](#).

The documentation for this class was generated from the following file:

- [include/particle_tracker/detail/ncdf_utils.hpp](#)

5.21 `particle_tracker::NetCDFProcessor< DataType, TimeType >` Class Template Reference

Convenience wrapper to load a single file and iterate over selected variables and times.

```
#include <ncdf_utils.hpp>
```

Public Types

- using [DataMap](#) = std::map<TimeType, std::vector<DataType>>
- using [VariableProcessingFunc](#)

Public Member Functions

- [NetCDFProcessor](#) (const std::string &file_path, const std::vector< std::string > &variable_names)
- void [processVariables](#) (const VariableProcessingFunc &processFunc)
- netCDF::NcVar [getNetCDFVariable](#) (const std::string &variable_name) const
- const std::vector< std::size_t > & [getVariableShape](#) (const std::string &variable_name) const
- const std::string & [getVariableType](#) (const std::string &variable_name) const
- const std::vector< TimeType > & [getTimeValues](#) () const
- const [NetCDFDataLoader](#)< DataType, TimeType >::DataMap & [getVariableData](#) (const std::string &variable_name) const
- std::vector< DataType > [getVariableDataAtTime](#) (const std::string &var_name, TimeType time_value)
- const std::vector< DataType > & [getVariableDataAtTimeRef](#) (const std::string &var_name, TimeType time_value) const

5.21.1 Detailed Description

```
template<typename DataType, typename TimeType = double>
class particle_tracker::NetCDFProcessor< DataType, TimeType >
```

Convenience wrapper to load a single file and iterate over selected variables and times.

Definition at line 550 of file [ncdf_utils.hpp](#).

5.21.2 Member Typedef Documentation

5.21.2.1 DataMap

```
template<typename DataType, typename TimeType = double>
using particle_tracker::NetCDFProcessor< DataType, TimeType >::DataMap = std::map<TimeType,
std::vector<DataType>>
```

Definition at line 553 of file [ncdf_utils.hpp](#).

5.21.2.2 VariableProcessingFunc

```
template<typename DataType, typename TimeType = double>
using particle_tracker::NetCDFProcessor< DataType, TimeType >::VariableProcessingFunc
```

Initial value:

```
std::function<void(const std::string&, TimeType, const std::vector<DataType>&)>
```

Definition at line 554 of file [ncdf_utils.hpp](#).

5.21.3 Constructor & Destructor Documentation

5.21.3.1 NetCDFProcessor()

```
template<typename DataType, typename TimeType = double>
particle_tracker::NetCDFProcessor< DataType, TimeType >::NetCDFProcessor (
    const std::string & file_path,
    const std::vector< std::string > & variable_names) [inline]
```

Definition at line 557 of file [ncdf_utils.hpp](#).

5.21.4 Member Function Documentation

5.21.4.1 getNetCDFVariable()

```
template<typename DataType, typename TimeType = double>
netCDF::NcVar particle_tracker::NetCDFProcessor< DataType, TimeType >::getNetCDFVariable (
    const std::string & variable_name) const [inline]
```

Definition at line 594 of file [ncdf_utils.hpp](#).

5.21.4.2 getTimeValues()

```
template<typename DataType, typename TimeType = double>
const std::vector< TimeType > & particle_tracker::NetCDFProcessor< DataType, TimeType >::↔
getTimeValues () const [inline]
```

Definition at line 609 of file [ncdf_utils.hpp](#).

5.21.4.3 getVariableData()

```
template<typename DataType, typename TimeType = double>
const NetCDFDataLoader< DataType, TimeType >::DataMap & particle_tracker::NetCDFProcessor<
DataType, TimeType >::getVariableData (
    const std::string & variable_name) const [inline]
```

Definition at line 615 of file [ncdf_utils.hpp](#).

5.21.4.4 getVariableDataAtTime()

```
template<typename DataType, typename TimeType = double>
std::vector< DataType > particle_tracker::NetCDFProcessor< DataType, TimeType >::getVariable↔
DataAtTime (
    const std::string & var_name,
    TimeType time_value) [inline]
```

Definition at line 626 of file [ncdf_utils.hpp](#).

5.21.4.5 `getVariableDataAtTimeRef()`

```
template<typename DataType, typename TimeType = double>
const std::vector< DataType > & particle\_tracker::NetCDFProcessor< DataType, TimeType >::getVariableDataAtTimeRef (
    const std::string & var_name,
    TimeType time_value) const [inline]
```

Definition at line 640 of file [ncdf_utils.hpp](#).

5.21.4.6 `getVariableShape()`

```
template<typename DataType, typename TimeType = double>
const std::vector< std::size_t > & particle\_tracker::NetCDFProcessor< DataType, TimeType >::getVariableShape (
    const std::string & variable_name) const [inline]
```

Definition at line 599 of file [ncdf_utils.hpp](#).

5.21.4.7 `getVariableType()`

```
template<typename DataType, typename TimeType = double>
const std::string & particle\_tracker::NetCDFProcessor< DataType, TimeType >::getVariableType (
    const std::string & variable_name) const [inline]
```

Definition at line 604 of file [ncdf_utils.hpp](#).

5.21.4.8 `processVariables()`

```
template<typename DataType, typename TimeType = double>
void particle\_tracker::NetCDFProcessor< DataType, TimeType >::processVariables (
    const VariableProcessingFunc & processFunc) [inline]
```

Definition at line 574 of file [ncdf_utils.hpp](#).

The documentation for this class was generated from the following file:

- [include/particle_tracker/detail/ncdf_utils.hpp](#)

5.22 PackedParticle Struct Reference

Public Attributes

- `int32_t lon0_q`
- `int32_t lat0_q`
- `int32_t lon1_q`
- `int32_t lat1_q`
- `int64_t t0_s`
- `int64_t t1_s`
- `uint8_t flags`

5.22.1 Detailed Description

Definition at line 58 of file [fixed_point_core.hpp](#).

5.22.2 Member Data Documentation

5.22.2.1 flags

```
uint8_t PackedParticle::flags
```

Definition at line 66 of file [fixed_point_core.hpp](#).

5.22.2.2 lat0_q

```
int32_t PackedParticle::lat0_q
```

Definition at line 61 of file [fixed_point_core.hpp](#).

5.22.2.3 lat1_q

```
int32_t PackedParticle::lat1_q
```

Definition at line 63 of file [fixed_point_core.hpp](#).

5.22.2.4 lon0_q

```
int32_t PackedParticle::lon0_q
```

Definition at line 60 of file [fixed_point_core.hpp](#).

5.22.2.5 lon1_q

```
int32_t PackedParticle::lon1_q
```

Definition at line 62 of file [fixed_point_core.hpp](#).

5.22.2.6 t0_s

```
int64_t PackedParticle::t0_s
```

Definition at line 64 of file [fixed_point_core.hpp](#).

5.22.2.7 t1_s

```
int64_t PackedParticle::t1_s
```

Definition at line 65 of file [fixed_point_core.hpp](#).

The documentation for this struct was generated from the following file:

- include/particle_tracker/fixed_point/fixed_point_core.hpp

5.23 Particle< DT, TT > Struct Template Reference

Public Attributes

- DT [init_lon](#)
- DT [init_lat](#)
- TT [init_time](#)
- DT [lon](#)
- DT [lat](#)
- TT [time](#)
- EndReason [reason](#)

5.23.1 Detailed Description

```
template<typename DT, typename TT>
struct Particle< DT, TT >
```

Definition at line 874 of file [data_prep_utils.hpp](#).

5.23.2 Member Data Documentation

5.23.2.1 init_lat

```
template<typename DT, typename TT>
DT particle\_tracker::Particle< DT, TT >::init_lat
```

Definition at line 876 of file [data_prep_utils.hpp](#).

5.23.2.2 init_lon

```
template<typename DT, typename TT>
DT particle\_tracker::Particle< DT, TT >::init_lon
```

Definition at line 875 of file [data_prep_utils.hpp](#).

5.23.2.3 init_time

```
template<typename DT, typename TT>  
TT particle_tracker::Particle< DT, TT >::init_time
```

Definition at line 877 of file [data_prep_utils.hpp](#).

5.23.2.4 lat

```
template<typename DT, typename TT>  
DT particle_tracker::Particle< DT, TT >::lat
```

Definition at line 879 of file [data_prep_utils.hpp](#).

5.23.2.5 lon

```
template<typename DT, typename TT>  
DT particle_tracker::Particle< DT, TT >::lon
```

Definition at line 878 of file [data_prep_utils.hpp](#).

5.23.2.6 reason

```
template<typename DT, typename TT>  
EndReason particle_tracker::Particle< DT, TT >::reason
```

Definition at line 881 of file [data_prep_utils.hpp](#).

5.23.2.7 time

```
template<typename DT, typename TT>  
TT particle_tracker::Particle< DT, TT >::time
```

Definition at line 880 of file [data_prep_utils.hpp](#).

The documentation for this struct was generated from the following file:

- [include/particle_tracker/data_prep_utils.hpp](#)

5.24 particle_tracker::Particle< DT, TT > Struct Template Reference

Public Attributes

- DT [init_lon](#)
- DT [init_lat](#)
- TT [init_time](#)
- DT [lon](#)
- DT [lat](#)
- TT [time](#)
- EndReason [reason](#)

5.24.1 Detailed Description

```
template<typename DT, typename TT>
struct particle_tracker::Particle< DT, TT >
```

Definition at line 874 of file [data_prep_utils.hpp](#).

5.24.2 Member Data Documentation

5.24.2.1 init_lat

```
template<typename DT, typename TT>
DT particle_tracker::Particle< DT, TT >::init_lat
```

Definition at line 876 of file [data_prep_utils.hpp](#).

5.24.2.2 init_lon

```
template<typename DT, typename TT>
DT particle_tracker::Particle< DT, TT >::init_lon
```

Definition at line 875 of file [data_prep_utils.hpp](#).

5.24.2.3 init_time

```
template<typename DT, typename TT>
TT particle_tracker::Particle< DT, TT >::init_time
```

Definition at line 877 of file [data_prep_utils.hpp](#).

5.24.2.4 lat

```
template<typename DT, typename TT>
DT particle_tracker::Particle< DT, TT >::lat
```

Definition at line 879 of file [data_prep_utils.hpp](#).

5.24.2.5 lon

```
template<typename DT, typename TT>
DT particle_tracker::Particle< DT, TT >::lon
```

Definition at line 878 of file [data_prep_utils.hpp](#).

5.24.2.6 reason

```
template<typename DT, typename TT>
EndReason particle_tracker::Particle< DT, TT >::reason
```

Definition at line 881 of file [data_prep_utils.hpp](#).

5.24.2.7 time

```
template<typename DT, typename TT>
TT particle_tracker::Particle< DT, TT >::time
```

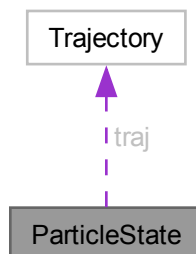
Definition at line 880 of file [data_prep_utils.hpp](#).

The documentation for this struct was generated from the following file:

- [include/particle_tracker/data_prep_utils.hpp](#)

5.25 ParticleState Struct Reference

Collaboration diagram for ParticleState:



Public Attributes

- uint64_t [seed_id](#) = 0
- DataType [lon](#) = 0
- DataType [lat](#) = 0
- int64_t [init_idx](#) = 0
- int64_t [time_idx](#) = 0
- bool [has_stop](#) = false
- int64_t [stop_idx](#) = 0
- DataType [stop_lon](#) = 0
- DataType [stop_lat](#) = 0
- EndReason [reason](#) = EndReason::none
- [Trajectory](#) [traj](#)

5.25.1 Detailed Description

Definition at line 129 of file [hpc_trajectories.cpp](#).

5.25.2 Member Data Documentation

5.25.2.1 has_stop

```
bool ParticleState::has_stop = false
```

Definition at line 136 of file [hpc_trajectories.cpp](#).

5.25.2.2 init_idx

```
int64_t ParticleState::init_idx = 0
```

Definition at line 133 of file [hpc_trajectories.cpp](#).

5.25.2.3 lat

```
DataType ParticleState::lat = 0
```

Definition at line 131 of file [hpc_trajectories.cpp](#).

5.25.2.4 lon

```
DataType ParticleState::lon = 0
```

Definition at line 131 of file [hpc_trajectories.cpp](#).

5.25.2.5 reason

```
EndReason ParticleState::reason = EndReason::none
```

Definition at line 140 of file [hpc_trajectories.cpp](#).

5.25.2.6 seed_id

```
uint64_t ParticleState::seed_id = 0
```

Definition at line 130 of file [hpc_trajectories.cpp](#).

5.25.2.7 stop_idx

```
int64_t ParticleState::stop_idx = 0
```

Definition at line 137 of file [hpc_trajectories.cpp](#).

5.25.2.8 stop_lat

```
DataType ParticleState::stop_lat = 0
```

Definition at line 138 of file [hpc_trajectories.cpp](#).

5.25.2.9 stop_lon

```
DataType ParticleState::stop_lon = 0
```

Definition at line 138 of file [hpc_trajectories.cpp](#).

5.25.2.10 time_idx

```
int64_t ParticleState::time_idx = 0
```

Definition at line 134 of file [hpc_trajectories.cpp](#).

5.25.2.11 traj

```
Trajectory ParticleState::traj
```

Definition at line 141 of file [hpc_trajectories.cpp](#).

The documentation for this struct was generated from the following file:

- [examples/hpc_trajectories.cpp](#)

5.26 particle_tracker::PointCloud< DataType > Struct Template Reference

Minimal dataset adaptor for nanoflann storing (lon, lat) in a flat array.

```
#include <interpolation_utils.hpp>
```

Public Member Functions

- `std::size_t kdtree_get_point_count ()` const
- `DataType kdtree_get_pt (const std::size_t idx, const std::size_t dim)` const
Coordinate accessor for nanoflann.
- `const DataType * data ()` const
Direct access to raw contiguous data.
- `template<class BBOX>`
`bool kdtree_get_bbox (BBOX &)` const
Bounding box hook for nanoflann.

Public Attributes

- `std::vector< DataType > pts`
Flat storage: [lon0, lat0, lon1, lat1, ...]. Must have even length.

5.26.1 Detailed Description

```
template<typename DataType>
struct particle_tracker::PointCloud< DataType >
```

Minimal dataset adaptor for nanoflann storing (lon, lat) in a flat array.

Template Parameters

<i>DataType</i>	numeric type (float/double).
-----------------	------------------------------

Points are stored as:

```
pts = [lon0, lat0, lon1, lat1, ..., lon(N-1), lat(N-1)]
```

Warning

`pts.size()` must be even (2 values per point).

Definition at line 249 of file [interpolation_utils.hpp](#).

5.26.2 Member Function Documentation

5.26.2.1 data()

```
template<typename DataType>
const DataType * particle_tracker::PointCloud< DataType >::data () const [inline]
```

Direct access to raw contiguous data.

Definition at line 268 of file [interpolation_utils.hpp](#).

References [pts](#).

5.26.2.2 kdtree_get_bbox()

```
template<typename DataType>
template<class BBOX>
bool particle_tracker::PointCloud< DataType >::kdtree_get_bbox (
    BBOX & ) const [inline]
```

Bounding box hook for nanoflann.

Returns

false (no precomputed bounding box is provided)

Definition at line 274 of file [interpolation_utils.hpp](#).

5.26.2.3 kdtree_get_point_count()

```
template<typename DataType>
std::size_t particle_tracker::PointCloud< DataType >::kdtree_get_point_count () const [inline]
```

Returns

number of points stored (pts.size()/2).

Definition at line 254 of file [interpolation_utils.hpp](#).

References [pts](#).

5.26.2.4 kdtree_get_pt()

```
template<typename DataType>
DataType particle_tracker::PointCloud< DataType >::kdtree_get_pt (
    const std::size_t idx,
    const std::size_t dim) const [inline]
```

Coordinate accessor for nanoflann.

Parameters

<i>idx</i>	Point index [0, point_count)
<i>dim</i>	Coordinate index (0 -> lon, 1 -> lat)

Returns

coordinate value

Definition at line 262 of file [interpolation_utils.hpp](#).

References [pts](#).

5.26.3 Member Data Documentation

5.26.3.1 pts

```
template<typename DataType>
std::vector<DataType> particle\_tracker::PointCloud< DataType >::pts
```

Flat storage: [lon0, lat0, lon1, lat1, ...]. Must have even length.

Definition at line 251 of file [interpolation_utils.hpp](#).

Referenced by [data\(\)](#), [kdtree_get_point_count\(\)](#), and [kdtree_get_pt\(\)](#).

The documentation for this struct was generated from the following file:

- include/particle_tracker/[interpolation_utils.hpp](#)

5.27 PointCloud< DataType > Struct Template Reference

Minimal dataset adaptor for nanoflann storing (lon, lat) in a flat array.

```
#include <interpolation_utils.hpp>
```

Public Member Functions

- `std::size_t kdtree\_get\_point\_count () const`
- `DataType kdtree\_get\_pt (const std::size_t idx, const std::size_t dim) const`
Coordinate accessor for nanoflann.
- `const DataType * data () const`
Direct access to raw contiguous data.
- `template<class BBOX>`
`bool kdtree\_get\_bbox (BBOX &) const`
Bounding box hook for nanoflann.

Public Attributes

- `std::vector< DataType > pts`
Flat storage: [lon0, lat0, lon1, lat1, ...]. Must have even length.

5.27.1 Detailed Description

```
template<typename DataType>
struct PointCloud< DataType >
```

Minimal dataset adaptor for nanoflann storing (lon, lat) in a flat array.

Template Parameters

<i>DataType</i>	numeric type (float/double).
-----------------	------------------------------

Points are stored as:

```
pts = [lon0, lat0, lon1, lat1, ..., lon(N-1), lat(N-1)]
```

Warning

`pts.size()` must be even (2 values per point).

Definition at line 249 of file [interpolation_utils.hpp](#).

5.27.2 Member Function Documentation

5.27.2.1 data()

```
template<typename DataType>
const DataType * particle\_tracker::PointCloud< DataType >::data () const [inline]
```

Direct access to raw contiguous data.

Definition at line 268 of file [interpolation_utils.hpp](#).

5.27.2.2 kdtree_get_bbox()

```
template<typename DataType>
template<class BBOX>
bool particle\_tracker::PointCloud< DataType >::kdtree_get_bbox (
    BBOX & ) const [inline]
```

Bounding box hook for nanoflann.

Returns

false (no precomputed bounding box is provided)

Definition at line 274 of file [interpolation_utils.hpp](#).

5.27.2.3 kdtree_get_point_count()

```
template<typename DataType>
std::size_t particle_tracker::PointCloud< DataType >::kdtree_get_point_count () const [inline]
```

Returns

number of points stored (pts.size()/2).

Definition at line 254 of file [interpolation_utils.hpp](#).

5.27.2.4 kdtree_get_pt()

```
template<typename DataType>
DataType particle_tracker::PointCloud< DataType >::kdtree_get_pt (
    const std::size_t idx,
    const std::size_t dim) const [inline]
```

Coordinate accessor for nanoflann.

Parameters

<i>idx</i>	Point index [0, point_count)
<i>dim</i>	Coordinate index (0 -> lon, 1 -> lat)

Returns

coordinate value

Definition at line 262 of file [interpolation_utils.hpp](#).

5.27.3 Member Data Documentation

5.27.3.1 pts

```
template<typename DataType>
std::vector<DataType> particle_tracker::PointCloud< DataType >::pts
```

Flat storage: [lon0, lat0, lon1, lat1, ...]. Must have even length.

Definition at line 251 of file [interpolation_utils.hpp](#).

The documentation for this struct was generated from the following file:

- include/particle_tracker/[interpolation_utils.hpp](#)

5.28 pkd2::Reader Class Reference

Public Member Functions

- [Reader](#) (const std::string &path)
- bool [has](#) (uint64_t seed_id) const
- void [read_traj](#) (uint64_t seed_id, std::vector< double > &lon_r, std::vector< double > &lat_r, std::vector< int64_t > &t_s) const
- const [Header](#) & [header](#) () const
- const std::vector< [TocEntry](#) > & [toc](#) () const

5.28.1 Detailed Description

Definition at line 360 of file [fixed_point_pkd2.hpp](#).

5.28.2 Constructor & Destructor Documentation

5.28.2.1 Reader()

```
pkd2::Reader::Reader (
    const std::string & path) [inline], [explicit]
```

Definition at line 362 of file [fixed_point_pkd2.hpp](#).

5.28.2.2 ~Reader()

```
pkd2::Reader::~Reader () [inline]
```

Definition at line 379 of file [fixed_point_pkd2.hpp](#).

5.28.3 Member Function Documentation

5.28.3.1 has()

```
bool pkd2::Reader::has (
    uint64_t seed_id) const [inline]
```

Definition at line 384 of file [fixed_point_pkd2.hpp](#).

5.28.3.2 header()

```
const Header & pkd2::Reader::header () const [inline]
```

Definition at line 449 of file [fixed_point_pkd2.hpp](#).

5.28.3.3 read_traj()

```
void pkd2::Reader::read_traj (
    uint64_t seed_id,
    std::vector< double > & lon_r,
    std::vector< double > & lat_r,
    std::vector< int64_t > & t_s) const [inline]
```

Definition at line 389 of file [fixed_point_pkd2.hpp](#).

5.28.3.4 toc()

```
const std::vector< TocEntry > & pkd2::Reader::toc () const [inline]
```

Definition at line 450 of file [fixed_point_pkd2.hpp](#).

The documentation for this class was generated from the following file:

- [include/particle_tracker/fixed_point/fixed_point_pkd2.hpp](#)

5.29 ReconGuards Struct Reference

Public Attributes

- DataType [max_current_mps](#) = 3.
- int [max_empty_neighbors](#) = 10
- DataType [step_slack](#) = 3.

5.29.1 Detailed Description

Definition at line 266 of file [hpc_trajectories.cpp](#).

5.29.2 Member Data Documentation

5.29.2.1 max_current_mps

```
DataType ReconGuards::max_current_mps = 3.
```

Definition at line 267 of file [hpc_trajectories.cpp](#).

5.29.2.2 max_empty_neighbors

```
int ReconGuards::max_empty_neighbors = 10
```

Definition at line 268 of file [hpc_trajectories.cpp](#).

5.29.2.3 step_slack

```
DataType ReconGuards::step_slack = 3.
```

Definition at line 269 of file [hpc_trajectories.cpp](#).

The documentation for this struct was generated from the following file:

- [examples/hpc_trajectories.cpp](#)

5.30 SeedInit Struct Reference

Public Attributes

- uint64_t [seed_id](#) {}
- double [lon_rad](#) {}
- double [lat_rad](#) {}
- int64_t [t_start_s](#) {}
- double [lon_start_rad](#) {}
- double [lat_start_rad](#) {}
- double [lon_stop_rad](#) {}
- double [lat_stop_rad](#) {}
- int64_t [t_stop_s](#) {}
- bool [has_stop](#) {false}

5.30.1 Detailed Description

Definition at line 35 of file [seed_loader_v2.hpp](#).

5.30.2 Member Data Documentation

5.30.2.1 has_stop

```
bool SeedInit::has_stop {false}
```

Definition at line 52 of file [seed_loader_v2.hpp](#).

5.30.2.2 lat_rad

```
double SeedInit::lat_rad {}
```

Definition at line 41 of file [seed_loader_v2.hpp](#).

5.30.2.3 lat_start_rad

```
double SeedInit::lat_start_rad {}
```

Definition at line 46 of file [seed_loader_v2.hpp](#).

5.30.2.4 lat_stop_rad

```
double SeedInit::lat_stop_rad {}
```

Definition at line 50 of file [seed_loader_v2.hpp](#).

5.30.2.5 lon_rad

```
double SeedInit::lon_rad {}
```

Definition at line 40 of file [seed_loader_v2.hpp](#).

5.30.2.6 lon_start_rad

```
double SeedInit::lon_start_rad {}
```

Definition at line 45 of file [seed_loader_v2.hpp](#).

5.30.2.7 lon_stop_rad

```
double SeedInit::lon_stop_rad {}
```

Definition at line 49 of file [seed_loader_v2.hpp](#).

5.30.2.8 seed_id

```
uint64_t SeedInit::seed_id {}
```

Definition at line 37 of file [seed_loader_v2.hpp](#).

5.30.2.9 t_start_s

```
int64_t SeedInit::t_start_s {}
```

Definition at line 42 of file [seed_loader_v2.hpp](#).

5.30.2.10 t_stop_s

```
int64_t SeedInit::t_stop_s {}
```

Definition at line 51 of file [seed_loader_v2.hpp](#).

The documentation for this struct was generated from the following file:

- [include/particle_tracker/seed_loader_v2.hpp](#)

5.31 particle_tracker::TimeSpline< DataType, TimeType > Class Template Reference

Wrapper around a Boost cubic spline interpolator for uniformly spaced samples.

```
#include <interpolation_utils.hpp>
```

Public Member Functions

- void [createSpline](#) (TimeType start_time, TimeType time_step)
Build the spline from the current [values](#).

Public Attributes

- std::vector< DataType > [values](#)
Sample values at uniform time spacing.

5.31.1 Detailed Description

```
template<typename DataType, typename TimeType = double>
class particle_tracker::TimeSpline< DataType, TimeType >
```

Wrapper around a Boost cubic spline interpolator for uniformly spaced samples.

Template Parameters

<i>DataType</i>	Value type of the samples (e.g. float, double).
<i>TimeType</i>	Time type used for evaluation (typically double).

5.31.1.0.1 Usage

- Fill [values](#) with uniformly spaced samples.
- Call `createSpline(start_time, time_step)`.
- Evaluate with `spline(t)` and time-derivative with `spline.prime(t)` if supported.

5.31.1.0.2 Notes

- This class stores the data and the spline object.
- The underlying Boost spline type depends on what Boost provides:
 - `boost::math::interpolators::cardinal_cubic_b_spline` (preferred when available)
 - `boost::math::cubic_b_spline` (fallback)

Definition at line 188 of file [interpolation_utils.hpp](#).

5.31.2 Member Function Documentation

5.31.2.1 createSpline()

```
template<typename DataType, typename TimeType = double>
void particle_tracker::TimeSpline< DataType, TimeType >::createSpline (
    TimeType start_time,
    TimeType time_step) [inline]
```

Build the spline from the current [values](#).

Parameters

<i>start_time</i>	Time of the first sample.
<i>time_step</i>	Uniform time step between samples.

Exceptions

<i>std::runtime_error</i>	if values is empty.
---------------------------	-------------------------------------

Definition at line 210 of file [interpolation_utils.hpp](#).

References [values](#).

5.31.3 Member Data Documentation

5.31.3.1 values

```
template<typename DataType, typename TimeType = double>
std::vector<DataType> particle_tracker::TimeSpline< DataType, TimeType >::values
```

Sample values at uniform time spacing.

Definition at line 191 of file [interpolation_utils.hpp](#).

Referenced by [createSpline\(\)](#).

The documentation for this class was generated from the following file:

- include/particle_tracker/[interpolation_utils.hpp](#)

5.32 TimeSpline< DataType, TimeType > Class Template Reference

Wrapper around a Boost cubic spline interpolator for uniformly spaced samples.

```
#include <interpolation_utils.hpp>
```

Public Member Functions

- void [createSpline](#) (TimeType start_time, TimeType time_step)
Build the spline from the current [values](#).

Public Attributes

- std::vector< DataType > [values](#)
Sample values at uniform time spacing.

5.32.1 Detailed Description

```
template<typename DataType, typename TimeType = double>
class TimeSpline< DataType, TimeType >
```

Wrapper around a Boost cubic spline interpolator for uniformly spaced samples.

Template Parameters

<i>DataType</i>	Value type of the samples (e.g. float, double).
<i>TimeType</i>	Time type used for evaluation (typically double).

5.32.1.0.1 Usage

- Fill [values](#) with uniformly spaced samples.
- Call `createSpline(start_time, time_step)`.
- Evaluate with `spline(t)` and time-derivative with `spline.prime(t)` if supported.

5.32.1.0.2 Notes

- This class stores the data and the spline object.
- The underlying Boost spline type depends on what Boost provides:
 - `boost::math::interpolators::cardinal_cubic_b_spline` (preferred when available)
 - `boost::math::cubic_b_spline` (fallback)

Definition at line 188 of file [interpolation_utils.hpp](#).

5.32.2 Member Function Documentation

5.32.2.1 createSpline()

```
template<typename DataType, typename TimeType = double>
void particle_tracker::TimeSpline< DataType, TimeType >::createSpline (
    TimeType start_time,
    TimeType time_step) [inline]
```

Build the spline from the current [values](#).

Parameters

<i>start_time</i>	Time of the first sample.
<i>time_step</i>	Uniform time step between samples.

Exceptions

<i>std::runtime_error</i>	if values is empty.
---------------------------	-------------------------------------

Definition at line 210 of file [interpolation_utils.hpp](#).

5.32.3 Member Data Documentation

5.32.3.1 values

```
template<typename DataType, typename TimeType = double>
std::vector<DataType> particle_tracker::TimeSpline< DataType, TimeType >::values
```

Sample values at uniform time spacing.

Definition at line 191 of file [interpolation_utils.hpp](#).

The documentation for this class was generated from the following file:

- include/particle_tracker/[interpolation_utils.hpp](#)

5.33 particle_tracker::TimeStepData< DT, TT > Struct Template Reference

Public Attributes

- TT [time_value](#)
- std::map< std::string, std::vector< DT > > [variable_data](#)

5.33.1 Detailed Description

```
template<typename DT, typename TT = double>
struct particle_tracker::TimeStepData< DT, TT >
```

Definition at line 67 of file [data_prep_utils.hpp](#).

5.33.2 Member Data Documentation

5.33.2.1 time_value

```
template<typename DT, typename TT = double>
TT particle_tracker::TimeStepData< DT, TT >::time_value
```

Definition at line 68 of file [data_prep_utils.hpp](#).

5.33.2.2 variable_data

```
template<typename DT, typename TT = double>
std::map<std::string, std::vector<DT> > particle_tracker::TimeStepData< DT, TT >::variable↵
_data
```

Definition at line 69 of file [data_prep_utils.hpp](#).

The documentation for this struct was generated from the following file:

- [include/particle_tracker/data_prep_utils.hpp](#)

5.34 pkd2::TocEntry Struct Reference

Public Attributes

- uint64_t [seed_id](#)
- uint64_t [file_offset](#)
- uint32_t [comp_size](#)
- uint32_t [raw_size](#)
- uint32_t [n_points](#)
- int32_t [start_index](#)
- uint16_t [stride](#)
- uint16_t [_pad](#) {0}

5.34.1 Detailed Description

Definition at line 90 of file [fixed_point_pkd2.hpp](#).

5.34.2 Member Data Documentation

5.34.2.1 _pad

```
uint16_t pkd2::TocEntry::_pad {0}
```

Definition at line 98 of file [fixed_point_pkd2.hpp](#).

5.34.2.2 comp_size

```
uint32_t pkd2::TocEntry::comp_size
```

Definition at line 93 of file [fixed_point_pkd2.hpp](#).

5.34.2.3 file_offset

```
uint64_t pkd2::TocEntry::file_offset
```

Definition at line 92 of file [fixed_point_pkd2.hpp](#).

5.34.2.4 n_points

```
uint32_t pkd2::TocEntry::n_points
```

Definition at line 95 of file [fixed_point_pkd2.hpp](#).

5.34.2.5 raw_size

```
uint32_t pkd2::TocEntry::raw_size
```

Definition at line 94 of file [fixed_point_pkd2.hpp](#).

5.34.2.6 seed_id

```
uint64_t pkd2::TocEntry::seed_id
```

Definition at line 91 of file [fixed_point_pkd2.hpp](#).

5.34.2.7 start_index

```
int32_t pkd2::TocEntry::start_index
```

Definition at line 96 of file [fixed_point_pkd2.hpp](#).

5.34.2.8 stride

```
uint16_t pkd2::TocEntry::stride
```

Definition at line 97 of file [fixed_point_pkd2.hpp](#).

The documentation for this struct was generated from the following file:

- [include/particle_tracker/fixed_point/fixed_point_pkd2.hpp](#)

5.35 Trajectory Struct Reference

Public Member Functions

- void [append](#) (float lon_r, float lat_r, float u_ms, float v_ms, int32_t idx)
- void [flush_to](#) ([pkd2::Writer](#) &w_pos, [pkd2::Writer](#) &w_vel, uint32_t stride=1)

Public Attributes

- uint64_t [seed_id](#) = 0
- std::vector< float > [lon](#)
- std::vector< float > [lat](#)
- std::vector< float > [u](#)
- std::vector< float > [v](#)
- int32_t [first_idx](#) = std::numeric_limits<int32_t>::max()

5.35.1 Detailed Description

Definition at line 83 of file [hpc_trajectories.cpp](#).

5.35.2 Member Function Documentation

5.35.2.1 append()

```
void Trajectory::append (
    float lon_r,
    float lat_r,
    float u_ms,
    float v_ms,
    int32_t idx) [inline]
```

Definition at line 91 of file [hpc_trajectories.cpp](#).

5.35.2.2 flush_to()

```
void Trajectory::flush_to (
    pkd2::Writer & w_pos,
    pkd2::Writer & w_vel,
    uint32_t stride = 1) [inline]
```

Definition at line 101 of file [hpc_trajectories.cpp](#).

5.35.3 Member Data Documentation

5.35.3.1 first_idx

```
int32_t Trajectory::first_idx = std::numeric_limits<int32_t>::max()
```

Definition at line 89 of file [hpc_trajectories.cpp](#).

5.35.3.2 lat

```
std::vector<float> Trajectory::lat
```

Definition at line 86 of file [hpc_trajectories.cpp](#).

5.35.3.3 lon

```
std::vector<float> Trajectory::lon
```

Definition at line 85 of file [hpc_trajectories.cpp](#).

5.35.3.4 seed_id

```
uint64_t Trajectory::seed_id = 0
```

Definition at line 84 of file [hpc_trajectories.cpp](#).

5.35.3.5 u

```
std::vector<float> Trajectory::u
```

Definition at line 87 of file [hpc_trajectories.cpp](#).

5.35.3.6 v

```
std::vector<float> Trajectory::v
```

Definition at line 88 of file [hpc_trajectories.cpp](#).

The documentation for this struct was generated from the following file:

- [examples/hpc_trajectories.cpp](#)

5.36 particle_tracker::VariableDerivatives< DataType > Struct Template Reference

Output container: value + time derivative + (heuristic) spatial derivatives.

```
#include <interpolation_utils.hpp>
```

Public Attributes

- DataType [value](#) {}
- DataType [time_derivative](#) {}
- DataType [spatial_derivative_x](#) {}
- DataType [spatial_derivative_y](#) {}

5.36.1 Detailed Description

```
template<typename DataType>
struct particle_tracker::VariableDerivatives< DataType >
```

Output container: value + time derivative + (heuristic) spatial derivatives.

Template Parameters

<i>DataType</i>	numeric type.
-----------------	---------------

Definition at line [677](#) of file [interpolation_utils.hpp](#).

5.36.2 Member Data Documentation

5.36.2.1 spatial_derivative_x

```
template<typename DataType>
DataType particle\_tracker::VariableDerivatives< DataType >::spatial\_derivative\_x {}
```

Definition at line [680](#) of file [interpolation_utils.hpp](#).

5.36.2.2 spatial_derivative_y

```
template<typename DataType>
DataType particle\_tracker::VariableDerivatives< DataType >::spatial\_derivative\_y {}
```

Definition at line [681](#) of file [interpolation_utils.hpp](#).

5.36.2.3 time_derivative

```
template<typename DataType>
DataType particle_tracker::VariableDerivatives< DataType >::time_derivative {}
```

Definition at line 679 of file [interpolation_utils.hpp](#).

5.36.2.4 value

```
template<typename DataType>
DataType particle_tracker::VariableDerivatives< DataType >::value {}
```

Definition at line 678 of file [interpolation_utils.hpp](#).

The documentation for this struct was generated from the following file:

- [include/particle_tracker/interpolation_utils.hpp](#)

5.37 pkd2::Writer Class Reference

Public Member Functions

- [Writer](#) (const std::string &path, int64_t t0_epoch_s, int32_t dt_s, uint32_t pos_scale=1 '000 '000, int compression_level=kDefaultLevel)
- template<class Real>
void [add_traj_rad](#) (uint64_t seed_id, const Real *lon_rad, const Real *lat_rad, uint32_t n_points, int32_t start_index, uint32_t stride=1)
- void [close](#) ()

5.37.1 Detailed Description

Definition at line 103 of file [fixed_point_pkd2.hpp](#).

5.37.2 Constructor & Destructor Documentation

5.37.2.1 Writer()

```
pkd2::Writer::Writer (
    const std::string & path,
    int64_t t0_epoch_s,
    int32_t dt_s,
    uint32_t pos_scale = 1'000'000,
    int compression_level = kDefaultLevel) [inline]
```

Definition at line 106 of file [fixed_point_pkd2.hpp](#).

5.37.2.2 ~Writer()

```
pkd2::Writer::~~Writer () [inline]
```

Definition at line 126 of file [fixed_point_pkd2.hpp](#).

5.37.3 Member Function Documentation

5.37.3.1 add_traj_rad()

```
template<class Real>
void pkd2::Writer::add_traj_rad (
    uint64_t seed_id,
    const Real * lon_rad,
    const Real * lat_rad,
    uint32_t n_points,
    int32_t start_index,
    uint32_t stride = 1) [inline]
```

Definition at line 136 of file [fixed_point_pkd2.hpp](#).

5.37.3.2 close()

```
void pkd2::Writer::close () [inline]
```

Definition at line 165 of file [fixed_point_pkd2.hpp](#).

The documentation for this class was generated from the following file:

- [include/particle_tracker/fixed_point/fixed_point_pkd2.hpp](#)

Chapter 6

File Documentation

6.1 hpc_particles.cpp

```
00001 #include "oneapi/tbb/blocked_range.h"
00002 #include "oneapi/tbb/global_control.h"
00003 #include "oneapi/tbb/parallel_for.h" // TBB parallel_for
00004 #include "particle_tracker/data_prep_utils.hpp" // Provides grid, time series, splines, etc.
00005 #include "particle_tracker/detail/datetime_utils.hpp"
00006 #include <algorithm>
00007 #include <atomic>
00008 #include <cassert>
00009 #include <chrono>
00010 #include <cmath>
00011 #include <cstdint>
00012 #include <cstdlib>
00013 #include <ctime> // for std::time_t, std::ctime
00014 #include <filesystem>
00015 #include <format>
00016 #include <fstream>
00017 #include <iomanip>
00018 #include <iostream> // for std::cerr
00019 #include <iterator>
00020 #include <map>
00021 #include <memory>
00022 #include <random>
00023 #include <regex>
00024 #include <set>
00025 #include <sstream>
00026 #include <unordered_map>
00027
00028 #include "particle_tracker/fixed_point/fixed_point_core.hpp" // the fixed-point pack/unpack
00029 #include "particle_tracker/fixed_point/fixed_point_io_codec.hpp" // the gzip helpers:
00030     write_records_gzip/read_records_gzip
00031
00031 using particle_tracker::DataType;
00032 using particle_tracker::dateFromSecondsSince2000;
00033 using particle_tracker::EndReason;
00034 using particle_tracker::formatDate;
00035 using particle_tracker::inverseTransform;
00036 using particle_tracker::KDTree;
00037 using particle_tracker::loadOneMonth;
00038 using particle_tracker::MonthData;
00039 using particle_tracker::Neighbor;
00040 using particle_tracker::NeighborsData;
00041 using particle_tracker::parseDate;
00042 using particle_tracker::Particle;
00043 using particle_tracker::point;
00044 using particle_tracker::PointCloud;
00045 using particle_tracker::secondsSince2000;
00046 using particle_tracker::splitArgs;
00047 using particle_tracker::TimeSpline;
00048 using particle_tracker::TimeType;
00049
00050 // -----
00051 //   Globals |
00052 // -----
00053
00054 // inline constexpr double number_pi = std::numbers::pi_v<double>;
00055 static const TimeType life_time_seconds = 2 * 365 * 24 * 3600.; // 2 yr
00056 static const TimeType grace_seconds = 7 * 24 * 3600.; // ~7 days
```

```

00057 static PointCloud<DataType> initial_band_cloud;
00058 static std::unique_ptr<KDTree<DataType>> kd_initial_band;
00059 static DataType stay_initial_band_thresh_squared;
00060
00061 // container keyed by release month
00062 using DayKey = Date;
00063 using ParticlePacket = std::vector<Particle<DataType, TimeType>>;
00064 static std::unordered_map<DayKey, ParticlePacket>
00065     all_particles; // new hash specialization is needed here
00066 static std::unordered_map<DayKey, std::uint64_t> released_total;
00067 static std::unordered_map<DayKey, std::uint64_t> landed_total;
00068 static std::unordered_map<DayKey, std::uint64_t> landed_east;
00069 static std::unordered_map<DayKey, std::uint64_t> landed_west;
00070 static std::unordered_map<DayKey, std::uint64_t>
00071     boring_total; // never left origin band (killed at grace)
00072 static std::unordered_map<DayKey, std::uint64_t>
00073     neverland_total; // left origin but never landed
00074
00075 // Random number generator
00076 static thread_local std::mt19937 gen(std::random_device{}());
00077
00078 // Helper: uniform random shift in [-max,max]
00079 template <class DataType> inline DataType randomShift(DataType max) {
00080     std::uniform_real_distribution<DataType> dis(-max, max);
00081     return dis(gen);
00082 }
00083
00084 struct BBox {
00085     DataType lon_min{}, lon_max{}, lat_min{}, lat_max{};
00086     bool wraps{};
00087     static inline DataType norm_pi(DataType L) noexcept {
00088         while (L <= -number_pi)
00089             L += 2 * number_pi;
00090         while (L > number_pi)
00091             L -= 2 * number_pi;
00092         return L;
00093     };
00094     BBox(DataType lo_min, DataType lo_max, DataType la_min, DataType la_max) {
00095         // normalize tiny numeric noise
00096         const DataType eps = static_cast<DataType>(1e-12);
00097         lon_min = norm_pi(lo_min);
00098         lon_max = norm_pi(lo_max);
00099         lat_min = la_min;
00100         lat_max = la_max;
00101         if (std::abs(lon_min - lon_max) < eps)
00102             lon_max = lon_min; // degenerate ok
00103         wraps = (lon_min > lon_max);
00104     }
00105     bool contains(DataType lon, DataType lat) const noexcept {
00106         if (lat < lat_min || lat > lat_max)
00107             return false;
00108         lon = norm_pi(lon); // make robust to any input range
00109         if (!wraps)
00110             return lon >= lon_min && lon <= lon_max;
00111         return lon >= lon_min || lon <= lon_max;
00112     }
00113 };
00114
00115 // =====
00116 // ONE-TIME INITIAL POSITION GENERATOR + DAILY SPAWNER
00117 // =====
00118
00119 // Global cache of initial positions (reused every day)
00120 static std::vector<point<DataType>> init_positions;
00121
00122 // Condition that the point is in the narrow band near the coast
00123 inline bool inNearCoastBand(DataType lon, DataType lat, const MonthData &md,
00124     DataType sea_tol_squared, DataType land_min_squared,
00125     DataType land_max_squared) {
00126     if (!md.kd_sea || !md.kd_land)
00127         throw std::runtime_error("KD trees not built");
00128
00129     DataType q[2] = {lon, lat};
00130
00131     // nearest SEA point check
00132     {
00133         size_t idx;
00134         DataType dist_squared;
00135         nanoflann::KNNResultSet<DataType> rs(1);
00136         rs.init(&idx, &dist_squared);
00137         md.kd_sea->findNeighbors(rs, q, nanoflann::SearchParameters());
00138         if (dist_squared > sea_tol_squared)
00139             return false;
00140     }
00141
00142     // band relative to LAND
00143     {

```

```

00144     size_t idx;
00145     DataType dist_squared;
00146     nanoflann::KNNResultSet<DataType> rs(1);
00147     rs.init(&idx, &dist_squared);
00148     md.kd_land->findNeighbors(rs, q, nanoflann::SearchParameters());
00149     if (dist_squared < land_min_squared || dist_squared > land_max_squared)
00150         return false;
00151 }
00152
00153 return true;
00154 }
00155
00156 std::vector<point<DataType>>
00157 buildCoastalBand(const MonthData &md, const BBox &box, DataType sea_tol_squared,
00158                 DataType land_min_squared, DataType land_max_squared) {
00159     std::vector<point<DataType>> band;
00160
00161     const std::size_t N = md.sea_cloud.kdtree_get_point_count();
00162
00163     band.reserve(N / 20);
00164
00165     for (size_t i = 0; i < N; i++) {
00166         DataType lon = md.sea_cloud.kdtree_get_pt(i, 0);
00167         DataType lat = md.sea_cloud.kdtree_get_pt(i, 1);
00168         if (!box.contains(lon, lat))
00169             continue;
00170
00171         if (inNearCoastBand(lon, lat, md, sea_tol_squared, land_min_squared,
00172                             land_max_squared))
00173             band.push_back({lon, lat});
00174     }
00175
00176     if (band.empty())
00177         std::cerr << "[WARN] buildCoastalBand(): empty band\n";
00178
00179     return band;
00180 }
00181
00182 void buildInitialPositions(std::size_t N, const MonthData &md_ref,
00183                           const BBox &box, DataType sea_tol, DataType land_min,
00184                           DataType land_max, DataType jitter) {
00185     if (!init_positions.empty())
00186         return; // already built
00187
00188     const DataType sea_tol_squared = sea_tol * sea_tol;
00189     const DataType land_min_squared = land_min * land_min;
00190     const DataType land_max_squared = land_max * land_max;
00191
00192     auto band = buildCoastalBand(md_ref, box, sea_tol_squared, land_min_squared,
00193                                 land_max_squared);
00194
00195     if (band.empty()) {
00196         std::cerr << "[ERROR] no valid band points for initial seeding\n";
00197         return;
00198     }
00199
00200     // ----- build coastal-band KD-tree -----
00201     initial_band_cloud.pts.clear();
00202     initial_band_cloud.pts.reserve(band.size() * 2);
00203     for (auto &p : band) {
00204         initial_band_cloud.pts.push_back(p[0]);
00205         initial_band_cloud.pts.push_back(p[1]);
00206     }
00207     auto kd_params = nanoflann::KDTreeSingleIndexAdaptorParams(10);
00208     kd_initial_band =
00209         std::make_unique<KDTree<DataType>>(2, initial_band_cloud, kd_params);
00210     kd_initial_band->buildIndex();
00211
00212     // Set "still-in-band" threshold once (example: 10 km)
00213     DataType stay_initial_band_rad = 10000. / 6371000.;
00214     stay_initial_band_thresh_squared =
00215         stay_initial_band_rad * stay_initial_band_rad;
00216
00217     std::uniform_int_distribution<std::size_t> pick(0, band.size() - 1);
00218
00219     init_positions.reserve(N);
00220
00221     std::size_t attempts = 0;
00222     const std::size_t max_attempts = N * 20; // heuristic
00223
00224     for (std::size_t accepted = 0; accepted < N && attempts < max_attempts;
00225         attempts++) {
00226         auto wrap_pi = [] (DataType L) {
00227             // wrap to (-pi, pi]
00228             while (L <= -number_pi)
00229                 L += 2 * number_pi;
00230             while (L > number_pi)
00231                 L -= 2 * number_pi;

```

```

00231         return L;
00232     };
00233     const auto &s = band[pick(gen)];
00234     DataType lon = wrap_pi(s[0] + randomShift(jitter));
00235     DataType lat = s[1] + randomShift(jitter);
00236
00237     if (inNearCoastBand(lon, lat, md_ref, sea_tol_squared, land_min_squared,
00238         land_max_squared)) {
00239         init_positions.push_back({lon, lat});
00240         ++accepted;
00241     }
00242 }
00243
00244 if (init_positions.size() != N) {
00245     std::cerr << "Failed to generate " << N << " points (got "
00246         << init_positions.size()
00247         << "). Loosen thresholds or enlarge jitter/band.\n";
00248 }
00249 if (init_positions.empty()) {
00250     throw std::runtime_error("Failed to seed initial positions (band empty or "
00251         "thresholds too strict).");
00252 }
00253 }
00254
00255 void spawn(const Date &day, ParticlePacket &packet, TimeType release_time) {
00256     // count once per day
00257     released_total[day] += static_cast<std::uint64_t>(init_positions.size());
00258
00259     packet.reserve(packet.size() + init_positions.size());
00260     for (auto const &xy : init_positions) {
00261         packet.push_back(Particle<DataType, TimeType>{xy[0], xy[1], release_time,
00262             xy[0], xy[1], release_time,
00263             EndReason::none});
00264     }
00265 }
00266
00267 // -----
00268 // Distance-to-land check (uses squared threshold)
00269 // -----
00270 inline bool nearLand(const point<DataType> &p, const MonthData &md,
00271     DataType thresh_squared) {
00272     if (!md.kd_land)
00273         throw std::runtime_error("kd_land not built");
00274     size_t idx;
00275     DataType d_squared;
00276     nanoflann::KNNResultSet<DataType> rs(1);
00277     rs.init(&idx, &d_squared);
00278     md.kd_land->findNeighbors(rs, p.data(), nanoflann::SearchParameters());
00279     return d_squared <= thresh_squared;
00280 }
00281
00282 inline bool stillInitialBand(const point<DataType> &p) {
00283     if (!kd_initial_band)
00284         return false; // safety
00285     size_t idx;
00286     DataType d_squared;
00287     nanoflann::KNNResultSet<DataType> rs(1);
00288     rs.init(&idx, &d_squared);
00289     kd_initial_band->findNeighbors(rs, p.data(), nanoflann::SearchParameters());
00290     return d_squared <= stay_initial_band_thresh_squared;
00291 }
00292
00293 // -----
00294 // Pick which month's data to use for absolute time t
00295 // -----
00296 inline const MonthData &pickMonth(TimeType t, const MonthData &older_m,
00297     const MonthData &newer_m) {
00298     Date d = dateFromSecondsSince2000(t);
00299     return (d.beginOfMonth() == older_m.month) ? older_m : newer_m;
00300 }
00301
00302 inline NeighborsData<DataType, TimeType> &
00303 pickNeighbors(TimeType t, const MonthData &older_m, const MonthData &newer_m,
00304     NeighborsData<DataType, TimeType> &nb_old,
00305     NeighborsData<DataType, TimeType> &nb_new) {
00306     Date d = dateFromSecondsSince2000(t);
00307     return (d.beginOfMonth() == older_m.month) ? nb_old : nb_new;
00308 }
00309
00310 // -----
00311 // Interpolate U,V using NeighborsData
00312 // -----
00313 // inline void interpolateUV(TimeType t,
00314 //     DataType lon,DataType lat,
00315 //     const std::vector<std::string>& vars, //
00316 //     {"water_u","water_v"} const MonthData& md,
00317 //     NeighborsData<DataType,TimeType>& nb,

```

```

00318 //                                     DataType& u,DataType& v)
00319 //{
00320 //     std::vector<Neighbor<DataType>> neigh;
00321 //     nb.computeNeighbors(neigh,lon,lat);
00322 //     if (neigh.empty()){u=v=0;return;}
00323 //
00324 //     auto vals=nb.interpolateVariables(t,neigh,md.spl,vars);
00325 //     u=vals.at(vars[0]);
00326 //     v=vals.at(vars[1]);
00327 //}
00328
00329 inline void
00330 interpolateUV(TimeType t, DataType lon, DataType lat,
00331               const std::vector<std::string> &vars, // {"water_u","water_v"}
00332               const MonthData &md, NeighborsData<DataType, TimeType> &nb,
00333               std::vector<Neighbor<DataType>> &neigh, // <-- scratch
00334               DataType &u, DataType &v) {
00335     neigh.clear(); // reuse capacity
00336     nb.computeNeighbors(neigh, lon, lat);
00337     if (neigh.empty()) {
00338         u = v = 0;
00339         return;
00340     }
00341
00342     auto vals = nb.interpolateVariables(t, neigh, md.spl, vars);
00343     u = vals.at(vars[0]);
00344     v = vals.at(vars[1]);
00345 }
00346
00347 // -----
00348 // propagateWindow(): back-propagate all alive particles over [win_beg, win_end]
00349 // using explicit Euler, spline interpolation & kd-trees.
00350 // Stops on: land hit, lifetime expired, window boundary.
00351 // -----
00352 void propagateWindow(const Date &win_beg, const Date &win_end,
00353                     const MonthData &older_m, const MonthData &newer_m,
00354                     const std::vector<std::string> &data_vars,
00355                     TimeType dt_seconds, DataType land_thresh_rad,
00356                     TimeType life_time_seconds) {
00357     const TimeType t_beg = secondsSince2000(win_beg); // inclusive (earliest)
00358     // const TimeType t_end=secondsSince2000(win_end); // inclusive (latest)
00359     const TimeType dt_sec = dt_seconds;
00360
00361     DataType search_radius = static_cast<DataType>(.08 * 1.1 / 180. * number_pi);
00362     DataType shape_param = search_radius * search_radius / (1.1 * 1.1);
00363
00364     // Build neighbor structures for both buffers (no static: rebuild each window)
00365     auto nb_old = makeNeighborsData(older_m, search_radius, shape_param);
00366     auto nb_new = makeNeighborsData(newer_m, search_radius, shape_param);
00367
00368     const DataType land_thresh_squared = land_thresh_rad * land_thresh_rad;
00369
00370     std::atomic<uint64_t> a_past_grace{0}, c_allow{0}, c_beach{0};
00371
00372     // Iterate through *all* release-month buckets
00373     for (auto &bucket : all_particles) {
00374         auto &vec = bucket.second;
00375
00376         if (!older_m.kd_sea || !newer_m.kd_sea)
00377             throw std::runtime_error("NeighborsData: kd_sea missing");
00378
00379         oneapi::tbb::parallel_for(
00380             oneapi::tbb::blocked_range<std::size_t>(0, vec.size(), 4096), // 32768
00381             [&](const oneapi::tbb::blocked_range<std::size_t> &r) {
00382                 std::vector<Neighbor<DataType>> neigh;
00383                 neigh.reserve(8); // typical neighbor count (4/8/16/32/64/128)
00384
00385                 for (std::size_t i = r.begin(); i != r.end(); i++) {
00386                     auto &p = vec[i];
00387                     // skip dead particles
00388                     if (p.reason != EndReason::none)
00389                         continue;
00390
00391                     bool grace_done = false;
00392
00393                     // while (p.reason==EndReason::none && p.time>=t_beg)
00394                     //{
00395                     //     TimeType age=p.init_time-p.time;
00396
00397                     //     if
00398                     //     (age>=life_time_seconds){p.reason=EndReason::lifetime;break;}
00399
00400                     //     if (!grace_done && age>=grace_seconds)
00401                     //     {
00402                     //         if
00403                     //         (stillInitialBand({p.lon,p.lat})) {p.reason=EndReason::neverleft;break;}
00404                     //         grace_done=true;

```

```

00405         //      }
00406
00407         //      const MonthData& md=pickMonth(p.time,older_m,newer_m);
00408         //      auto& nb =pickNeighbors(p.time,older_m,newer_m,nb_old,nb_new);
00409
00410         //      // Only beach if (past grace) OR (we've left the initial band)
00411         //      bool allow_beach=(age>=grace_seconds) ||
00412         //      !stillInitialBand({p.lon,p.lat}); if (allow_beach &&
00413         //      nearLand({p.lon,p.lat},md,land_thresh_squared))
00414         //      {
00415         //          p.reason=EndReason::landed;
00416         //          break;
00417         //      }
00418
00419         //      DataType u,v;
00420         //      interpolateUV(p.time,p.lon,p.lat,data_vars,md,nb,u,v);
00421         //      DataType dx=-u*dt_sec,dy=-v*dt_sec;
00422         //      inverseTransform(dx,dy,p.lon,p.lat);
00423
00424         //      p.time-=dt_sec;
00425         //      if (p.time<t_beg) break;
00426         //  }
00427
00428         const bool started_past_grace =
00429             (p.init_time - p.time) >= grace_seconds;
00430         bool grace_counted =
00431             started_past_grace; // already past -> don't count a crossing
00432
00433         while (p.reason == EndReason::none && p.time >= t_beg) {
00434             TimeType age = p.init_time - p.time;
00435
00436             if (!grace_counted && age >= grace_seconds) {
00437                 a_past_grace.fetch_add(1, std::memory_order_relaxed);
00438                 grace_counted = true;
00439             }
00440
00441             if (age >= life_time_seconds) {
00442                 p.reason = EndReason::lifetime;
00443                 break;
00444             }
00445
00446             const MonthData &md = pickMonth(p.time, older_m, newer_m);
00447             auto &nb =
00448                 pickNeighbors(p.time, older_m, newer_m, nb_old, nb_new);
00449
00450             //[[maybe_unused]] bool in_band =stillInitialBand({p.lon,p.lat});
00451             bool past_grace = (age >= grace_seconds);
00452             // bool allow_beach=(past_grace || !in_band);
00453             bool allow_beach = past_grace;
00454             if (allow_beach)
00455                 c_allow.fetch_add(1, std::memory_order_relaxed);
00456
00457             if (allow_beach &&
00458                 nearLand({p.lon, p.lat}, md, land_thresh_squared)) {
00459                 c_beach.fetch_add(1, std::memory_order_relaxed);
00460                 p.reason = EndReason::landed;
00461                 break;
00462             }
00463
00464             // 2) (Optional) Only *mark* never-left at grace,
00465             //      but don't break here -- let it keep going further back in
00466             //      time.
00467             if (!grace_done && past_grace) {
00468                 // if (in_band)
00469                 //{
00470                 //      // p.reason = EndReason::neverleft; break;    // <- REMOVE
00471                 //      this early break
00472                 //      // If you want to count later, track a flag:
00473                 //      // p.flags |= EVER_IN_BAND_AT_GRACE;  (or keep a side map
00474                 //      // if you can't change Particle)
00475                 // }
00476                 grace_done = true;
00477             }
00478
00479             DataType u, v;
00480             interpolateUV(p.time, p.lon, p.lat, data_vars, md, nb, neigh, u,
00481                         v);
00482             DataType dx = -u * dt_sec, dy = -v * dt_sec;
00483             inverseTransform(dx, dy, p.lon, p.lat);
00484
00485             auto wrap_lon_inplace = [](DataType &L) {
00486                 while (L <= -number_pi)
00487                     L += 2 * number_pi;
00488                 while (L > number_pi)
00489                     L -= 2 * number_pi;
00490             };
00491             auto clamp_lat_inplace = [](DataType &L) {

```

```

00492         if (L < -number_pi / 2)
00493             L = -number_pi / 2;
00494         if (L > number_pi / 2)
00495             L = number_pi / 2;
00496     };
00497
00498     wrap_lon_inplace(p.lon);
00499     clamp_lat_inplace(p.lat);
00500
00501     p.time -= dt_sec;
00502     if (p.time < t_beg)
00503         break;
00504     }
00505 }
00506 });
00507 }
00508
00509 std::cerr << "window " << formatDate(win_beg)
00510             << ": past_grace=" << a_past_grace.load()
00511             << " allow_beach=" << c_allow.load()
00512             << " beached=" << c_beach.load() << "\n";
00513 }
00514
00515 inline PackedParticle packBinary(const Particle<DataType, TimeType> &p) {
00516     uint8_t flags = static_cast<uint8_t>(p.reason); // 0..3
00517
00518     return encode_record<DataType, TimeType>(p.init_lon, // was p.lon0
00519                                              p.init_lat, // was p.lat0
00520                                              p.init_time, // was p.t0
00521                                              p.lon, // was p.lon1
00522                                              p.lat, // was p.lat1
00523                                              p.time, // was p.t1
00524                                              flags);
00525 }
00526
00527 inline std::string dayTag(const Date &d) {
00528     std::string out = "trajectories_" + formatDate(d) + ".pkd.gz";
00529     return out;
00530 }
00531
00532 inline std::filesystem::path outPath(const std::filesystem::path &dir,
00533                                     const Date &d, int level) {
00534     const auto filename =
00535         std::format("trajectories_{}_{m}.pkd.gz", formatDate(d), level);
00536     return dir / filename;
00537 }
00538
00539 // -----
00540 // Main
00541 // -----
00542 int main(int argc, char *argv[]) try {
00543     if (argc != 12 && argc != 13) {
00544         std::cerr << "usage: " << argv[0]
00545                 << " REL_START REL_END LON_MIN LON_MAX LAT_MIN LAT_MAX LEVEL "
00546                 << "TIMESTEP PATTERN1 PATTERN2 OUT_DIR"
00547                 << " [[GRID_LON,GRID_LAT,DATA1,DATA2,...]]\n";
00548         return 1;
00549     }
00550
00551     // counter on evaluation time
00552     auto start = std::chrono::system_clock::now();
00553     std::time_t start_time = std::chrono::system_clock::to_time_t(start);
00554     std::cerr << "started computation at " << std::ctime(&start_time) << '\n';
00555
00556     // oneapi::tbb::global_control
00557     // control(oneapi::tbb::global_control::max_allowed_parallelism, 64); // Limit
00558     // to XX threads
00559
00560     /* release window ----- */
00561     Date start_release = parseDate(argv[1]);
00562     Date end_release = parseDate(argv[2]);
00563     // DateRange sim_range(start_release, end_release);
00564
00565     /* spatial filter ----- */
00566     DataType lon_min = std::stof(argv[3]) * number_pi / 180.;
00567     DataType lon_max = std::stof(argv[4]) * number_pi / 180.;
00568     DataType lat_min = std::stof(argv[5]) * number_pi / 180.;
00569     DataType lat_max = std::stof(argv[6]) * number_pi / 180.;
00570
00571     int level = std::stoi(argv[7]);
00572     int timestep = std::stoi(argv[8]);
00573     std::string pattern_1 = argv[9];
00574     std::string pattern_2 = argv[10];
00575     std::filesystem::path out_dir = argv[11];
00576     std::error_code ec;
00577     std::filesystem::create_directories(out_dir, ec);
00578     if (ec) {

```

```

00579     std::cerr << "Cannot create output dir '" << out_dir
00580               << "': " << ec.message() << "\n";
00581     return 1;
00582 }
00583
00584 // -----
00585 // variable names (default to water_u / water_v)
00586 // -----
00587 std::vector<std::string> data_variable_names = {"water_u", "water_v"};
00588 std::vector<std::string> grid_variable_names = {"lon", "lat", "water_u"};
00589
00590 if (argc == 13) {
00591     auto names = splitArgs(argv[12]); // lon,lat,data1,...
00592
00593     if (names.size() < 3) {
00594         std::cerr << "ERROR: variable list must contain lon_var,lat_var,at least "
00595                   << "one data_var\n";
00596         return 1;
00597     }
00598
00599     // replace grids
00600     grid_variable_names[0] = names[0]; // lon variable
00601     grid_variable_names[1] = names[1]; // lat variable
00602     grid_variable_names[2] = names[2]; // first data-var (for NaN mask)
00603
00604     // replace data list
00605     data_variable_names.assign(names.begin() + 2, names.end());
00606 }
00607
00608 // TimeType dt_hours=timestep; // in hours
00609 TimeType dt_seconds = static_cast<TimeType>(timestep) * 3600.;
00610 Date cur_month = end_release.beginOfMonth(); // e.g. 2004-01-01
00611
00612 // Basic sanity on CLI angles (degrees expected, convert to rad above)
00613 auto in_range = [](double x, double a, double b) { return x >= a && x <= b; };
00614
00615 if (!in_range(std::stod(argv[3]), -180., 180.) ||
00616     !in_range(std::stod(argv[4]), -180., 180.) ||
00617     !in_range(std::stod(argv[5]), -90., 90.) ||
00618     !in_range(std::stod(argv[6]), -90., 90.)) {
00619     std::cerr << "Longitude/latitude degrees out of range. "
00620               << "Use lon in [-180,180], lat in [-90,90].\n";
00621     return 1;
00622 }
00623
00624 if (timestep <= 0) {
00625     std::cerr << "TIMESTEP must be positive (hours).\n";
00626     return 1;
00627 }
00628
00629 MonthData buf[2];
00630 static int older = 0, newer = 1;
00631
00632 // load Month 0 and Month -1 (+padding handled inside)
00633 buf[older] =
00634     loadOneMonth(cur_month, level, pattern_1, pattern_2, grid_variable_names,
00635                 data_variable_names, dt_seconds);
00636 buildKdTrees(buf[older]);
00637
00638 buf[newer] =
00639     loadOneMonth(cur_month.addMonths(-1), level, pattern_1, pattern_2,
00640                 grid_variable_names, data_variable_names, dt_seconds);
00641 buildKdTrees(buf[newer]);
00642
00643 /* -----
00644 */
00645 /* MAIN MONTH-ADVANCING LOOP */
00646 /* -----
00647 */
00648
00649 BBox bbox{lon_min, lon_max, lat_min, lat_max};
00650
00651 DataType sea_tol = .2 * number_pi / 180.;
00652 DataType land_min = 2000. / 6371000.;
00653 DataType land_max = 50000. / 6371000.;
00654 DataType jitter = .0005;
00655 std::size_t N_per_day = 2'800'000; // 2'000'000
00656
00657 buildInitialPositions(N_per_day, buf[older], bbox, sea_tol, land_min,
00658                      land_max, jitter);
00659
00660 // const Date last_month=end_release.beginOfMonth(); // e.g.
00661 // 2004-12-01
00662 const Date first_month =
00663     start_release.addMonths(-24).beginOfMonth(); // earliest month
00664
00665 while (cur_month >= first_month) {

```



```

00666     /* 1. one-month simulation window [cur_month , cur_month+1) ----- */
00667     Date win_beg = cur_month;
00668     Date win_end = cur_month.addMonths(1).addDays(-1); // inclusive last day
00669
00670     for (Date day = win_beg; day <= win_end; day.increment()) {
00671         if (day < start_release || day > end_release)
00672             continue; // <-- guard
00673         TimeType rel_time = secondsSince2000(day);
00674
00675         // MonthKey key=day.beginOfMonth();
00676         DayKey key = day;
00677         ParticlePacket &pack = all_particles[key];
00678
00679         spawn(day, pack, rel_time);
00680     }
00681
00682     DataType land_thresh = (0.5) * number_pi / 180.; // 5000./6371000.;
00683     propagateWindow(win_beg, win_end, buf[older], buf[newer],
00684                     data_variable_names, dt_seconds, land_thresh,
00685                     life_time_seconds);
00686
00687     for (auto it = all_particles.begin(); it != all_particles.end(); ) {
00688         const DayKey day = it->first; // copy key (iterator may change)
00689         auto &vec = it->second;
00690
00691         std::vector<PackedParticle> recs;
00692         recs.reserve(vec.size());
00693
00694         vec.erase(std::remove_if(vec.begin(), vec.end(),
00695                                  [&](const auto &p) {
00696                                      if (p.reason == EndReason::none)
00697                                          return false;
00698
00699                                      switch (p.reason) {
00700                                          case EndReason::landed: {
00701                                              recs.push_back(packBinary(p));
00702                                              landed_total[day]++;
00703
00704                                              // classify by longitude at landing
00705                                              double lon_deg = p.lon * 180. / number_pi;
00706                                              double lon360 = std::fmod(lon_deg + 360.,
00707                                                                      360.); // [0,360)
00708
00709                                              if (lon360 >= 180.)
00710                                                  landed_west[day]++;
00711                                              else
00712                                                  landed_east[day]++;
00713                                              break;
00714                                          }
00715
00716                                          case EndReason::neverleft:
00717                                              boring_total[day]++;
00718                                              break;
00719
00720                                          case EndReason::lifetime:
00721                                              neverland_total[day]++;
00722                                              break;
00723
00724                                          default:
00725                                              break;
00726                                      }
00727                                      return true; // drop ended from RAM
00728                                  })),
00729                vec.end());
00730
00731         if (!recs.empty()) {
00732             const auto path = outPath(out_dir, day, level);
00733             std::error_code ec2;
00734             const bool append = std::filesystem::exists(path, ec2) && !ec2;
00735             write_records_gzip(path.string(), recs, released_total[day],
00736                               /*level=*/6, /*append=*/append);
00737             // optional: std::cerr<<"write "<<recs.size()<<" to "<<path<<(append ? "
00738             // (append)\n" : " (new)\n");
00739         }
00740
00741         if (vec.empty())
00742             it = all_particles.erase(it);
00743         else
00744             ++it;
00745     }
00746
00747     std::uint64_t L = 0, B = 0, N = 0, R = 0, E = 0, W = 0;
00748     for (Date day = win_beg; day <= win_end; day.increment()) {
00749         L += landed_total[day];
00750         B += boring_total[day];
00751         N += neverland_total[day];
00752         R += released_total[day];

```

```

00753     E += landed_east[day];
00754     W += landed_west[day];
00755 }
00756 std::cerr << "[" << formatDate(win_beg) << "-" << formatDate(win_end)
00757 << "]" << "released=" << R << " landed=" << L
00758 << " landed east =" << E << " landed west=" << W << '\n'
00759 << "neverleft=" << B << " lifetime=" << N << "\n";
00760
00761 /* 2. advance the ring buffer by one month ----- */
00762 cur_month = cur_month.addMonths(-1); // Feb → Mar → Apr ...
00763
00764 older ^= 1; // swap indices (0 1)
00765 newer ^= 1;
00766
00767 /* load the *new* Month+1 (two months ahead of 'older') */
00768 if (cur_month > first_month) {
00769     buf[newer] = MonthData{}; // drop old newer before loading fresh
00770     buf[newer] =
00771         loadOneMonth(cur_month.addMonths(-1), level, pattern_1, pattern_2,
00772                     grid_variable_names, data_variable_names, dt_seconds);
00773     buildKdTrees(buf[newer]);
00774 }
00775 }
00776
00777 for (auto &[day, vec] : all_particles)
00778     for (auto &p : vec)
00779         if (p.reason == EndReason::none)
00780             neverland_total[day]++;
00781
00782 all_particles.clear();
00783
00784 auto end = std::chrono::system_clock::now();
00785 std::chrono::duration<double> elapsed_seconds = end - start;
00786 std::time_t end_time = std::chrono::system_clock::to_time_t(end);
00787 std::cerr << '\n'
00788 << "finished computation at " << std::ctime(&end_time)
00789 << "elapsed time: " << elapsed_seconds.count() << "s\n";
00790
00791 return 0;
00792 } catch (const std::exception &e) {
00793     std::cerr << "FATAL: " << e.what() << "\n";
00794     return 2;
00795 } catch (...) {
00796     std::cerr << "FATAL: unknown exception\n";
00797     return 3;
00798 }

```

6.2 hpc_trajectories.cpp

```

00001 // hpc_trajectories.cpp
00002
00003 #include <algorithm>
00004 #include <atomic>
00005 #include <cassert>
00006 #include <cctype>
00007 #include <chrono>
00008 #include <cmath>
00009 #include <cstdint>
00010 #include <cstdlib>
00011 #include <ctime>
00012 #include <filesystem>
00013 #include <format>
00014 #include <fstream>
00015 #include <iomanip>
00016 #include <iostream>
00017 #include <iterator>
00018 #include <limits>
00019 #include <map>
00020 #include <memory>
00021 #include <random>
00022 #include <regex>
00023 #include <set>
00024 #include <sstream>
00025 #include <stdexcept>
00026 #include <string_view>
00027 #include <unordered_map>
00028 #include <vector>
00029
00030 #include "oneapi/tbb/blocked_range.h"
00031 #include "oneapi/tbb/enumerable_thread_specific.h"
00032 #include "oneapi/tbb/global_control.h"
00033 #include "oneapi/tbb/parallel_for.h"
00034 #include "particle_tracker/data_prep_utils.hpp"

```

```

00035 #include "particle_tracker/detail/datetime_utils.hpp"
00036 #include "particle_tracker/seed_loader_v2.hpp"
00037
00038 #include "particle_tracker/fixed_point/fixed_point_core.hpp"
00039 #include "particle_tracker/fixed_point/fixed_point_io_codec.hpp"
00040 #include "particle_tracker/fixed_point/fixed_point_pkd2.hpp"
00041
00042 using particle_tracker::DataType;
00043 using particle_tracker::dateFromSecondsSince2000;
00044 using particle_tracker::EndReason;
00045 using particle_tracker::formatDate;
00046 using particle_tracker::inverseTransform;
00047 using particle_tracker::KDTree;
00048 using particle_tracker::loadOneMonth;
00049 using particle_tracker::MonthData;
00050 using particle_tracker::Neighbor;
00051 using particle_tracker::NeighborsData;
00052 using particle_tracker::parseDate;
00053 using particle_tracker::Particle;
00054 using particle_tracker::point;
00055 using particle_tracker::PointCloud;
00056 using particle_tracker::secondsSince2000;
00057 using particle_tracker::splitArgs;
00058 using particle_tracker::TimeSpline;
00059 using particle_tracker::TimeType;
00060
00061 // inline constexpr double number_pi = std::numbers::pi_v<double>;
00062 static const TimeType life_limit_seconds = 2 * 365 * 24 * 3600.;
00063
00064 // ----- small helpers -----
00065 static inline std::string lower(std::string s) {
00066     std::transform(s.begin(), s.end(), s.begin(), [](unsigned char c) {
00067         return static_cast<char>(std::tolower(c));
00068     });
00069     return s;
00070 }
00071 static inline bool ends_with_ci(const std::string &s, const std::string &suf) {
00072     if (s.size() < suf.size())
00073         return false;
00074     return lower(s.substr(s.size() - suf.size())) == lower(suf);
00075 }
00076 static inline SeedPoint opposite_seed_point(SeedPoint p) {
00077     return (p == SeedPoint::First) ? SeedPoint::Last : SeedPoint::First;
00078 }
00079
00080 // -----
00081 // Trajectory storage
00082 // -----
00083 struct Trajectory {
00084     uint64_t seed_id = 0;
00085     std::vector<float> lon;
00086     std::vector<float> lat;
00087     std::vector<float> u;
00088     std::vector<float> v;
00089     int32_t first_idx = std::numeric_limits<int32_t>::max();
00090
00091     inline void append(float lon_r, float lat_r, float u_ms, float v_ms,
00092         int32_t idx) {
00093         lon.push_back(lon_r);
00094         lat.push_back(lat_r);
00095         u.push_back(u_ms);
00096         v.push_back(v_ms);
00097         if (idx < first_idx)
00098             first_idx = idx;
00099     }
00100
00101     inline void flush_to(pk2::Writer &w_pos, pk2::Writer &w_vel,
00102         uint32_t stride = 1) {
00103         if (lon.empty())
00104             return;
00105         std::reverse(lon.begin(), lon.end());
00106         std::reverse(lat.begin(), lat.end());
00107         std::reverse(u.begin(), u.end());
00108         std::reverse(v.begin(), v.end());
00109         // Positions
00110         w_pos.add_traj_rad<float>(seed_id, lon.data(), lat.data(),
00111             static_cast<uint32_t>(lon.size()), first_idx,
00112             stride);
00113         // Velocities
00114         // lon channel = u, lat channel = v (units: m/s).
00115         w_vel.add_traj_rad<float>(seed_id, u.data(), v.data(),
00116             static_cast<uint32_t>(u.size()), first_idx,
00117             stride);
00118         lon.clear();
00119         lat.clear();
00120         u.clear();
00121         v.clear();

```

```

00122     first_idx = std::numeric_limits<int32_t>::max();
00123 }
00124 };
00125
00126 // -----
00127 // Particle: integer time indexing + optional stop target
00128 // -----
00129 struct ParticleState {
00130     uint64_t seed_id = 0;
00131     DataType lon = 0, lat = 0;
00132
00133     int64_t init_idx = 0; // start index on dt grid
00134     int64_t time_idx = 0; // current index
00135
00136     bool has_stop = false;
00137     int64_t stop_idx = 0; // valid only if has_stop==true
00138     DataType stop_lon = 0, stop_lat = 0;
00139
00140     EndReason reason = EndReason::none;
00141     Trajectory traj;
00142 };
00143
00144 using DayKey = Date;
00145 using ParticlePacket = std::vector<ParticleState>;
00146 static std::unordered_map<DayKey, ParticlePacket> all_particles;
00147 static std::unordered_map<DayKey, std::uint64_t> released_total;
00148
00149 // -----
00150 // Epoch + snapping utilities
00151 // -----
00152 enum class SeedTimeEpoch { Unix, Since2000 };
00153 constexpr int64_t UNIX_TO_2000 = 946684800;
00154
00155 static inline int64_t snap_to_grid(int64_t t_s, int32_t dt_s) {
00156     const double x = static_cast<double>(t_s) / static_cast<double>(dt_s);
00157     return static_cast<int64_t>(std::llround(x)) * static_cast<int64_t>(dt_s);
00158 }
00159
00160 static inline int64_t sec_to_idx_exact(int64_t t_s, int32_t dt_s) {
00161     if (dt_s <= 0)
00162         throw std::runtime_error("dt_s <= 0");
00163     if (t_s % dt_s == 0)
00164         return t_s / dt_s;
00165     return static_cast<int64_t>(
00166         std::llround(static_cast<double>(t_s) / static_cast<double>(dt_s)));
00167 }
00168
00169 inline void normalize_seed_times_to_t2000(std::vector<SeedInit> &seeds,
00170     SeedTimeEpoch epoch) {
00171     if (epoch == SeedTimeEpoch::Since2000)
00172         return;
00173     for (auto &s : seeds) {
00174         s.t_start_s -= UNIX_TO_2000;
00175         if (s.has_stop)
00176             s.t_stop_s -= UNIX_TO_2000;
00177     }
00178 }
00179
00180 // IMPORTANT: do NOT "fix" ordering by swapping only times.
00181 // We only snap times here.
00182 inline void snap_seed_times_to_grid(std::vector<SeedInit> &seeds,
00183     int32_t dt_s) {
00184     for (auto &s : seeds) {
00185         s.t_start_s = snap_to_grid(s.t_start_s, dt_s);
00186         if (s.has_stop)
00187             s.t_stop_s = snap_to_grid(s.t_stop_s, dt_s);
00188     }
00189 }
00190
00191 // -----
00192 // Enqueue seeds (sets integer indices and stop target)
00193 // -----
00194 static inline void enqueue_seeds(const std::vector<SeedInit> &seeds,
00195     int32_t dt_s) {
00196     for (const auto &s : seeds) {
00197         const Date d = dateFromSecondsSince2000(static_cast<TimeType>(s.t_start_s));
00198         auto &vec = all_particles[d];
00199
00200         ParticleState ps;
00201         ps.seed_id = s.seed_id;
00202
00203         // Start point = lon_start_rad/lat_start_rad by our record-building logic in
00204         // main()
00205         ps.lon = static_cast<DataType>(s.lon_start_rad);
00206         ps.lat = static_cast<DataType>(s.lat_start_rad);
00207
00208         ps.init_idx = sec_to_idx_exact(s.t_start_s, dt_s);

```

```

00209     ps.time_idx = ps.init_idx;
00210
00211     ps.has_stop = s.has_stop;
00212     if (s.has_stop) {
00213         ps.stop_lon = static_cast<DataType>(s.lon_stop_rad);
00214         ps.stop_lat = static_cast<DataType>(s.lat_stop_rad);
00215         ps.stop_idx = sec_to_idx_exact(s.t_stop_s, dt_s);
00216     } else {
00217         ps.stop_idx = 0; // unused
00218     }
00219
00220     ps.reason = EndReason::none;
00221     ps.traj.seed_id = s.seed_id;
00222
00223     vec.push_back(std::move(ps));
00224     released_total[d]++;
00225 }
00226 }
00227
00228 // Map month-start Date -> seeds
00229 using SeedsByMonth = std::unordered_map<Date, std::vector<SeedInit>;
00230
00231 static inline Date month_start_from_t2000(int64_t t2000_s) {
00232     return dateFromSecondsSince2000(static_cast<TimeType>(t2000_s))
00233         .beginOfMonth();
00234 }
00235
00236 static inline void bucketize_seeds(const std::vector<SeedInit> &seeds,
00237                                   SeedsByMonth &bym) {
00238     bym.reserve(seeds.size() / 32 + 8);
00239     for (const auto &s : seeds)
00240         bym[month_start_from_t2000(s.t_start_s)].push_back(s);
00241 }
00242
00243 static inline void activate_month(SeedsByMonth &bym, const Date &win_beg,
00244                                   const Date &win_end, int32_t dt_s) {
00245     const Date key = win_beg.beginOfMonth();
00246     auto it = bym.find(key);
00247     if (it == bym.end())
00248         return;
00249
00250     const int64_t tmin = static_cast<int64_t>(secondsSince2000(win_beg));
00251     const int64_t tmax =
00252         static_cast<int64_t>(secondsSince2000(win_end.addDays(1)));
00253
00254     std::vector<SeedInit> window_seeds;
00255     window_seeds.reserve(it->second.size());
00256     for (const auto &s : it->second)
00257         if (s.t_start_s >= tmin && s.t_start_s < tmax)
00258             window_seeds.push_back(s);
00259
00260     if (!window_seeds.empty())
00261         enqueue_seeds(window_seeds, dt_s);
00262
00263     bym.erase(it);
00264 }
00265
00266 struct ReconGuards {
00267     DataType max_current_mps = 3.;
00268     int max_empty_neighbors = 10;
00269     DataType step_slack = 3.;
00270 };
00271
00272 void propagateWindow(const Date &win_beg, const Date &win_end,
00273                    const MonthData &older_m, const MonthData &newer_m,
00274                    const std::vector<std::string> &data_vars,
00275                    TimeType dt_seconds, TimeType life_time_seconds,
00276                    pkd2::Writer &pkd_pos, pkd2::Writer &pkd_vel,
00277                    int32_t base_dt_s, const ReconGuards &G = ReconGuards{}) {
00278     (void)win_end;
00279     (void)dt_seconds;
00280
00281     if (data_vars.size() < 2)
00282         throw std::runtime_error(
00283             "propagateWindow: need at least 2 data_vars (u,v)");
00284     if (!older_m.kd_sea || !newer_m.kd_sea)
00285         throw std::runtime_error("propagateWindow: kd_sea missing");
00286     if (base_dt_s <= 0)
00287         throw std::runtime_error("propagateWindow: base_dt_s must be positive");
00288
00289     const TimeType dtT = static_cast<TimeType>(base_dt_s);
00290
00291     // Window lower bound in *index* (exact: win_beg is midnight, base_dt_s
00292     // divides 86400 for 3h)
00293     const int64_t t_beg_s = static_cast<int64_t>(secondsSince2000(win_beg));
00294     const int64_t t_beg_idx = t_beg_s / static_cast<int64_t>(base_dt_s);
00295

```

```

00296     const DataType search_radius =
00297         static_cast<DataType>(.08 * 1.1 / 180. * number_pi);
00298     const DataType shape_param = search_radius * search_radius / (1.1 * 1.1);
00299
00300     auto nb_old = makeNeighborsData(older_m, search_radius, shape_param);
00301     auto nb_new = makeNeighborsData(newer_m, search_radius, shape_param);
00302
00303     std::atomic<uint64_t> stop_life{0}, stop_no_nb{0}, stop_fast{0}, stop_step{0},
00304         stop_seed{0};
00305
00306     oneapi::tbb::enumerable_thread_specific<std::vector<Trajectory>> done_tls;
00307
00308     for (auto &bucket : all_particles) {
00309         auto &vec = bucket.second;
00310
00311         oneapi::tbb::parallel_for(
00312             oneapi::tbb::blocked_range<std::size_t>(0, vec.size(), 4096),
00313             [&](const oneapi::tbb::blocked_range<std::size_t> &r) {
00314                 std::vector<Neighbor<DataType>> neigh;
00315                 neigh.reserve(16);
00316
00317                 for (std::size_t i = r.begin(); i != r.end(); i++) {
00318                     auto &p = vec[i];
00319                     if (p.reason != EndReason::none)
00320                         continue;
00321
00322                     p.traj.seed_id = p.seed_id;
00323
00324                     int empty_nb_streak = 0;
00325                     int fast_streak = 0;
00326
00327                     // Stop if we hit window bound OR stop target (seed0) OR lifetime
00328                     while (p.reason == EndReason::none && p.time_idx >= t_beg_idx &&
00329                         (!p.has_stop || p.time_idx >= p.stop_idx)) {
00330                         // If exactly at seed0 time, snap to seed0 position, record, and
00331                         // finish.
00332                         if (p.has_stop && p.time_idx == p.stop_idx) {
00333                             p.lon = p.stop_lon;
00334                             p.lat = p.stop_lat;
00335
00336                             if (p.time_idx < std::numeric_limits<int32_t>::min() ||
00337                                 p.time_idx > std::numeric_limits<int32_t>::max()) {
00338                                 p.reason = EndReason::lifetime;
00339                                 ++stop_life;
00340                                 break;
00341                             }
00342
00343                             p.traj.append(static_cast<float>(p.lon),
00344                                 static_cast<float>(p.lat),
00345                                 std::numeric_limits<float>::quiet_NaN(),
00346                                 std::numeric_limits<float>::quiet_NaN(),
00347                                 static_cast<int32_t>(p.time_idx));
00348
00349                             p.reason = EndReason::lifetime; // finished
00350                             ++stop_seed0;
00351                             break;
00352                         }
00353
00354                         if (p.time_idx < std::numeric_limits<int32_t>::min() ||
00355                             p.time_idx > std::numeric_limits<int32_t>::max()) {
00356                             p.reason = EndReason::lifetime;
00357                             ++stop_life;
00358                             break;
00359                         }
00360
00361                         // exact lifetime in index space
00362                         const int64_t age_idx = (p.init_idx - p.time_idx);
00363                         const TimeType age_s = static_cast<TimeType>(age_idx) * dtT;
00364                         if (age_s >= life_time_seconds) {
00365                             p.reason = EndReason::lifetime;
00366                             ++stop_life;
00367                             break;
00368                         }
00369
00370                         // absolute time for interpolation
00371                         const TimeType t_s = static_cast<TimeType>(p.time_idx) * dtT;
00372
00373                         const Date dcur = dateFromSecondsSince2000(t_s);
00374                         const bool use_older = (dcur.beginOfMonth() == older_m.month);
00375                         const MonthData &md = use_older ? older_m : newer_m;
00376                         auto &nb = use_older ? nb_old : nb_new;
00377
00378                         neigh.clear();
00379                         nb.computeNeighbors(neigh, p.lon, p.lat);
00380
00381                         if (neigh.empty()) {
00382                             // record position, but velocity unknown

```

```

00383         p.traj.append(static_cast<float>(p.lon),
00384                       static_cast<float>(p.lat),
00385                       std::numeric_limits<float>::quiet_NaN(),
00386                       std::numeric_limits<float>::quiet_NaN(),
00387                       static_cast<int32_t>(p.time_idx));
00388
00389         if (++empty_nb_streak > G.max_empty_neighbors) {
00390             p.reason = EndReason::lifetime;
00391             stop_no_nb++;
00392             break;
00393         }
00394         --p.time_idx;
00395         continue;
00396     }
00397     empty_nb_streak = 0;
00398
00399     auto vals =
00400         nb.interpolateVariables(t_s, neigh, md.spl, data_vars);
00401     DataType u = vals.at(data_vars[0]);
00402     DataType v = vals.at(data_vars[1]);
00403
00404     p.traj.append(static_cast<float>(p.lon),
00405                 static_cast<float>(p.lat), static_cast<float>(u),
00406                 static_cast<float>(v),
00407                 static_cast<int32_t>(p.time_idx));
00408
00409     const double spd =
00410         std::sqrt(double(u) * double(u) + double(v) * double(v));
00411     if (spd > G.max_current_mps) {
00412         if (++fast_streak > 4) {
00413             p.reason = EndReason::lifetime;
00414             ++stop_fast;
00415             break;
00416         }
00417     } else
00418         fast_streak = 0;
00419
00420     const DataType dt = static_cast<DataType>(base_dt_s);
00421     DataType dx = -u * dt;
00422     DataType dy = -v * dt;
00423
00424     {
00425         const double max_step_km = G.max_current_mps *
00426             double(base_dt_s) * G.step_slack /
00427             1000.;
00428         const double dr_km = std::sqrt(double(dx) * double(dx) +
00429             double(dy) * double(dy)) /
00430             1000.;
00431         if (dr_km > max_step_km) {
00432             p.reason = EndReason::lifetime;
00433             ++stop_step;
00434             break;
00435         }
00436     }
00437
00438     inverseTransform(dx, dy, p.lon, p.lat);
00439
00440     while (p.lon <= -number_pi)
00441         p.lon += 2 * number_pi;
00442     while (p.lon > number_pi)
00443         p.lon -= 2 * number_pi;
00444     if (p.lat < -number_pi / 2)
00445         p.lat = -number_pi / 2;
00446     if (p.lat > number_pi / 2)
00447         p.lat = number_pi / 2;
00448
00449     --p.time_idx;
00450 }
00451
00452 if (p.reason != EndReason::none) {
00453     done_tls.local().push_back(std::move(p.traj));
00454     p.traj = Trajectory{};
00455     p.traj.seed_id = p.seed_id;
00456 }
00457 }
00458 });
00459 }
00460
00461 // flush finished trajectories serially (Writer not thread-safe)
00462 for (auto &local : done_tls) {
00463     for (auto &tr : local)
00464         tr.flush_to(pk_d_pos, pk_d_vel, /*stride=*/1);
00465     local.clear();
00466 }
00467
00468 std::cerr << std::format(
00469     "window {} | lifetime={} no_nb={} fast={} step={} seed0={}\n",

```

```

00470         formatDate(win_beg), stop_life.load(), stop_no_nb.load(),
00471         stop_fast.load(), stop_step.load(), stop_seed0.load());
00472     }
00473
00474     // -----
00475     // Main
00476     // -----
00477     int main(int argc, char *argv[]) try {
00478         const auto wall_start = std::chrono::system_clock::now();
00479         std::time_t start_time = std::chrono::system_clock::to_time_t(wall_start);
00480         std::cerr << "started computation at " << std::ctime(&start_time) << '\n';
00481
00482         if (argc < 9) {
00483             std::cerr
00484                 << "usage: " << argv[0] << "\n"
00485                 << "  REL_START REL_END LEVEL TIMESTEP_H PATTERN1 PATTERN2 OUT_DIR "
00486                 << "SEEDS_PATH\n"
00487                 << "  [SEED_EPOCH] [VARLIST]\n"
00488                 << "  [--keep-p p] [--cap N] [--seed S] [--seed-point first|last]\n";
00489             return 1;
00490         }
00491
00492         const Date start_release = parseDate(argv[1]);
00493         const Date end_release = parseDate(argv[2]);
00494         const int level = std::stoi(argv[3]);
00495         const int timestep_h = std::stoi(argv[4]);
00496         const std::string pattern_1 = argv[5];
00497         const std::string pattern_2 = argv[6];
00498         const std::filesystem::path out_dir = argv[7];
00499         const std::string seeds_path = argv[8];
00500
00501         if (timestep_h <= 0) {
00502             std::cerr << "TIMESTEP_H must be positive.\n";
00503             return 1;
00504         }
00505         const TimeType dt_seconds = static_cast<TimeType>(timestep_h) * 3600.0;
00506         const int32_t base_dt_s = static_cast<int32_t>(dt_seconds);
00507
00508         int argi = 9;
00509
00510         SeedTimeEpoch epoch = ends_with_ci(seeds_path, ".pkd2")
00511             ? SeedTimeEpoch::Since2000
00512             : SeedTimeEpoch::Unix;
00513
00514         std::vector<std::string> data_variable_names = {"water_u", "water_v"};
00515         std::vector<std::string> grid_variable_names = {"lon", "lat", "water_u"};
00516
00517         double keep_p = 1.0;
00518         std::size_t cap = std::numeric_limits<std::size_t>::max();
00519         uint32_t thin_seed = 0;
00520         bool use_thin_seed = false;
00521
00522         SeedPoint seed_point = SeedPoint::Last; // backtracking start point
00523
00524         auto is_flag = [](const char *s) -> bool {
00525             return s && std::string_view(s).rfind("--", 0) == 0;
00526         };
00527
00528         if (argi < argc && !is_flag(argv[argi])) {
00529             const std::string tok = lower(argv[argi]);
00530             if (tok == "unix" || tok == "t2000" || tok == "since2000") {
00531                 epoch = (tok == "unix") ? SeedTimeEpoch::Unix : SeedTimeEpoch::Since2000;
00532                 ++argi;
00533             }
00534         }
00535
00536         if (argi < argc && !is_flag(argv[argi])) {
00537             const std::string tok = argv[argi];
00538             if (tok.find(',') != std::string::npos) {
00539                 auto names = splitArgs(tok);
00540                 if (names.size() < 3) {
00541                     std::cerr << "ERROR: variable list must contain lon_var,lat_var,at "
00542                         << "least one data_var\n";
00543                     return 1;
00544                 }
00545                 grid_variable_names[0] = names[0];
00546                 grid_variable_names[1] = names[1];
00547                 grid_variable_names[2] = names[2];
00548                 data_variable_names.assign(names.begin() + 2, names.end());
00549                 ++argi;
00550             }
00551         }
00552
00553         for (; argi < argc; ++argi) {
00554             const std::string a = argv[argi];
00555             if (a == "--keep-p") {
00556                 if (argi + 1 >= argc) {

```



```

00557         std::cerr << "ERROR: --keep-p requires a value\n";
00558         return 1;
00559     }
00560     keep_p = std::stod(argv[++argi]);
00561 } else if (a == "--cap") {
00562     if (argi + 1 >= argc) {
00563         std::cerr << "ERROR: --cap requires a value\n";
00564         return 1;
00565     }
00566     cap = static_cast<std::size_t>(std::stoull(argv[++argi]));
00567 } else if (a == "--seed") {
00568     if (argi + 1 >= argc) {
00569         std::cerr << "ERROR: --seed requires a value\n";
00570         return 1;
00571     }
00572     thin_seed = static_cast<uint32_t>(std::stoul(argv[++argi]));
00573     use_thin_seed = true;
00574 } else if (a == "--seed-point") {
00575     if (argi + 1 >= argc) {
00576         std::cerr << "ERROR: --seed-point requires first|last\n";
00577         return 1;
00578     }
00579     const std::string v = lower(argv[++argi]);
00580     if (v == "first")
00581         seed_point = SeedPoint::First;
00582     else if (v == "last")
00583         seed_point = SeedPoint::Last;
00584     else {
00585         std::cerr << "ERROR: --seed-point must be first|last\n";
00586         return 1;
00587     }
00588 } else {
00589     std::cerr << "WARN: ignoring unknown argument '" << a << "'\n";
00590 }
00591 }
00592
00593 if (!(keep_p >= 0.0 && keep_p <= 1.0)) {
00594     std::cerr << "ERROR: --keep-p must be in [0,1], got " << keep_p << "\n";
00595     return 1;
00596 }
00597
00598 // If PKD2: epoch must be Since2000 (t2000). If user passes unix, fail loudly.
00599 if (ends_with_ci(seeds_path, ".pkd2") && epoch == SeedTimeEpoch::Unix) {
00600     std::cerr << "FATAL: SEEDS_PATH is .pkd2; seed times are already t2000. Do "
00601         << "NOT pass 'unix'.\n";
00602     return 2;
00603 }
00604
00605 {
00606     std::error_code ec;
00607     std::filesystem::create_directories(out_dir, ec);
00608     if (ec) {
00609         std::cerr << "Cannot create output dir '" << out_dir
00610             << "': " << ec.message() << "\n";
00611         return 1;
00612     }
00613 }
00614
00615 const int64_t T0_EPOCH_S = 0;
00616
00617 const auto pkd_pos_path =
00618     (out_dir / std::format("trajectories_{}h.pkd2", timestep_h)).string();
00619 const auto pkd_vel_path =
00620     (out_dir / std::format("velocities_{}h.pkd2", timestep_h)).string();
00621
00622 pkd2::Writer pkd_pos(pkd_pos_path, T0_EPOCH_S, base_dt_s,
00623     /*pos_scale=*/1'000'000u, /*zlib=*/6);
00624 pkd2::Writer pkd_vel(pkd_vel_path, T0_EPOCH_S, base_dt_s,
00625     /*pos_scale=*/1'000'000u, /*zlib=*/6);
00626
00627 // ----- LOAD SEEDS (build explicit start/stop records) -----
00628 std::vector<SeedInit> seeds;
00629 {
00630     auto starts = load_seeds_auto(seeds_path, seed_point);
00631     if (starts.empty()) {
00632         std::cerr << "No seeds loaded from " << seeds_path << "\n";
00633         return 1;
00634     }
00635
00636     if (ends_with_ci(seeds_path, ".pkd2")) {
00637         auto stops = load_seeds_auto(seeds_path, opposite_seed_point(seed_point));
00638         if (stops.size() != starts.size())
00639             throw std::runtime_error(
00640                 "PKD2: starts.size() != stops.size() (corrupt TOC or mismatch)");
00641
00642         seeds.reserve(starts.size());
00643         for (size_t i = 0; i < starts.size(); ++i) {

```

```

00644         if (starts[i].seed_id != stops[i].seed_id)
00645             throw std::runtime_error(
00646                 "PKD2: seed_id mismatch between start/stop streams");
00647
00648         SeedInit r{};
00649         r.seed_id = starts[i].seed_id;
00650
00651         // start = user-chosen seed_point
00652         r.lon_start_rad = starts[i].lon_rad;
00653         r.lat_start_rad = starts[i].lat_rad;
00654         r.t_start_s = starts[i].t_start_s;
00655
00656         // stop = opposite point
00657         r.has_stop = true;
00658         r.lon_stop_rad = stops[i].lon_rad;
00659         r.lat_stop_rad = stops[i].lat_rad;
00660         r.t_stop_s = stops[i].t_start_s;
00661
00662         seeds.push_back(r);
00663     }
00664 } else {
00665     // CSV: start only
00666     seeds = std::move(starts);
00667     for (auto &s : seeds) {
00668         s.lon_start_rad = s.lon_rad;
00669         s.lat_start_rad = s.lat_rad;
00670         s.has_stop = false;
00671         s.t_stop_s = 0;
00672         s.lon_stop_rad = 0;
00673         s.lat_stop_rad = 0;
00674     }
00675 }
00676 }
00677
00678 normalize_seed_times_to_t2000(seeds, epoch);
00679 snap_seed_times_to_grid(seeds, base_dt_s);
00680
00681 auto minmax = std::minmax_element(
00682     seeds.begin(), seeds.end(),
00683     [](auto const &a, auto const &b) { return a.t_start_s < b.t_start_s; });
00684
00685 std::cerr << "SEEDS t_start_s min=" << minmax.first->t_start_s
00686             << " max=" << minmax.second->t_start_s << "\n";
00687 std::cerr << "SEEDS month min="
00688             << formatDate(month_start_from_t2000(minmax.first->t_start_s))
00689             << " max="
00690             << formatDate(month_start_from_t2000(minmax.second->t_start_s))
00691             << "\n";
00692
00693 // Critical invariant for BACKWARD integration:
00694 // start must be later-or-equal to stop.
00695 for (const auto &s : seeds) {
00696     if (s.has_stop && s.t_stop_s > s.t_start_s) {
00697         throw std::runtime_error("Seed ordering invalid for backtracking: "
00698                                 "t_stop_s > t_start_s for seed_id=" +
00699                                 std::to_string(s.seed_id) +
00700                                 ". Use --seed-point last for backtracking, or "
00701                                 "fix seed construction.");
00702     }
00703 }
00704
00705 // Thin (keeps PKD2 start/stop paired because it's one record)
00706 if (keep_p < 1. || cap != std::numeric_limits<std::size_t>::max()) {
00707     std::mt19937 rng;
00708     if (use_thin_seed)
00709         rng.seed(thin_seed);
00710     else
00711         rng.seed(std::random_device{}());
00712
00713     std::bernoulli_distribution keep(keep_p);
00714
00715     std::size_t w = 0;
00716     const std::size_t N = seeds.size();
00717     for (std::size_t i = 0; i < N && w < cap; i++) {
00718         if (keep_p >= 1.0 || keep(rng))
00719             seeds[w++] = std::move(seeds[i]);
00720     }
00721     seeds.resize(w);
00722
00723     std::cerr << "After thinning: " << seeds.size() << "\n";
00724 }
00725
00726 SeedsByMonth bym;
00727 bucketize_seeds(seeds, bym);
00728
00729 Date cur_month = end_release.beginOfMonth();
00730

```

```

00731     MonthData buf[2];
00732     static int older = 0, newer = 1;
00733
00734     buf[older] =
00735         loadOneMonth(cur_month, level, pattern_1, pattern_2, grid_variable_names,
00736             data_variable_names, dt_seconds);
00737     buildKdTrees(buf[older]);
00738
00739     buf[newer] =
00740         loadOneMonth(cur_month.addMonths(-1), level, pattern_1, pattern_2,
00741             grid_variable_names, data_variable_names, dt_seconds);
00742     buildKdTrees(buf[newer]);
00743
00744     // 24 months back (e.g. 1997-01 -> 1995-01)
00745     const Date first_month = start_release.addMonths(-24).beginOfMonth();
00746
00747     while (cur_month >= first_month) {
00748         const Date win_beg = cur_month;
00749         const Date win_end = cur_month.addMonths(1).addDays(-1);
00750
00751         activate_month(bym, win_beg, win_end, base_dt_s);
00752
00753         propagateWindow(win_beg, win_end, buf[older], buf[newer],
00754             data_variable_names, dt_seconds, life_limit_seconds,
00755             pkd_pos, pkd_vel, base_dt_s);
00756
00757         for (auto it = all_particles.begin(); it != all_particles.end(); ) {
00758             auto &vec = it->second;
00759             vec.erase(std::remove_if(
00760                 vec.begin(), vec.end(),
00761                 [](const auto &p) { return p.reason != EndReason::none; }),
00762                 vec.end());
00763             if (vec.empty())
00764                 it = all_particles.erase(it);
00765             else
00766                 ++it;
00767         }
00768
00769         cur_month = cur_month.addMonths(-1);
00770         older ^= 1;
00771         newer ^= 1;
00772
00773         if (cur_month > first_month) {
00774             buf[newer] = MonthData{};
00775             buf[newer] =
00776                 loadOneMonth(cur_month.addMonths(-1), level, pattern_1, pattern_2,
00777                     grid_variable_names, data_variable_names, dt_seconds);
00778             buildKdTrees(buf[newer]);
00779         }
00780     }
00781
00782     // flush survivors
00783     for (auto &bucket : all_particles)
00784         for (auto &p : bucket.second)
00785             p.traj.flush_to(pkd_pos, pkd_vel, /*stride=*/1);
00786
00787     pkd_pos.close();
00788     pkd_vel.close();
00789
00790     const auto wall_end = std::chrono::system_clock::now();
00791     std::chrono::duration<double> elapsed_seconds = wall_end - wall_start;
00792     std::time_t end_time = std::chrono::system_clock::to_time_t(wall_end);
00793     std::cerr << "\nfinished computation at " << std::ctime(&end_time)
00794         << "elapsed time: " << elapsed_seconds.count() << " s\n";
00795
00796     return 0;
00797 } catch (const std::exception &e) {
00798     std::cerr << "FATAL: " << e.what() << "\n";
00799     return 2;
00800 } catch (...) {
00801     std::cerr << "FATAL: unknown exception\n";
00802     return 3;
00803 }

```

6.3 data_prep_utils.hpp

```

00001 #ifndef DATA_PREP_UTILS_HPP
00002 #define DATA_PREP_UTILS_HPP
00003
00004 #include <algorithm>
00005 #include <cassert>
00006 #include <cmath>
00007 #include <cstdint>

```

```

00008 #include <cstdlib>
00009 #include <fstream>
00010 #include <functional>
00011 #include <iomanip>
00012 #include <iostream>
00013 #include <limits>
00014 #include <map>
00015 #include <memory>
00016 #include <numbers> // std::numbers::pi_v
00017 #include <regex>
00018 #include <set>
00019 #include <sstream>
00020 #include <stdexcept>
00021 #include <string>
00022 #include <vector>
00023
00024 #include <oneapi/tbb/parallel_for.h>
00025
00026 #include "particle_tracker/detail/ncdf_utils.hpp"
00027 #include "particle_tracker/fixed_point/fixed_point_core.hpp"
00028 #include "particle_tracker/interpolation_utils.hpp"
00029 #include <particle_tracker/detail/datetime_utils.hpp>
00030
00031 // Ensure C++17 or higher for filesystem support
00032 #if __cplusplus < 201703L
00033 #error "C++17 or higher is required"
00034 #endif
00035
00036 inline constexpr double number_pi = std::numbers::pi_v<double>;
00037
00038 // -----
00039 // Optional debug logging (compile-time gated)
00040 // -----
00041 // Define DATA_PREP_DEBUG_TIME_STEPS to print time-step diagnostics.
00042 // #define DATA_PREP_DEBUG_TIME_STEPS
00043
00044 namespace std {
00045     template <> struct hash<::Date> {
00046         std::size_t operator()(const ::Date &d) const noexcept {
00047             return static_cast<std::size_t>(d.getYear()) * 32 * 13 +
00048                 static_cast<std::size_t>(d.getMonth()) * 32 +
00049                 static_cast<std::size_t>(d.getDay());
00050         }
00051     };
00052 } // namespace std
00053
00054 namespace particle_tracker {
00055     // Grid struct templated on DataType
00056     template <typename DT> struct Grid {
00057         std::vector<DT> lon_rad;
00058         std::vector<DT> lat_rad;
00059         std::vector<bool> in_the_sea;
00060         std::size_t num_points;
00061
00062         Grid() : num_points(0) {}
00063     };
00064
00065     // TimeStepData struct templated on DataType and TimeType
00066     template <typename DT, typename TT = double> struct TimeStepData {
00067         TT time_value;
00068         std::map<std::string, std::vector<DT>> variable_data;
00069     };
00070
00071     // -----
00072     // Function to initialize the grid
00073     //
00074     // variable_names is expected to contain:
00075     //   [0] lon variable name
00076     //   [1] lat variable name
00077     //   [2] a "mask variable" used to detect sea points (NaN = land)
00078     // -----
00079     template <typename DT>
00080     Grid<DT> initializeGrid(const std::string &file_path,
00081                           const std::vector<std::string> &variable_names) {
00082         if (variable_names.size() < 3) {
00083             throw std::runtime_error("initializeGrid(): variable_names must contain "
00084                                     "lon_var, lat_var, mask_var.");
00085         }
00086
00087         const std::string &lon_var = variable_names[0];
00088         const std::string &lat_var = variable_names[1];
00089         const std::string &mask_var = variable_names[2];
00090
00091         Grid<DT> grid;
00092
00093         // Create the processor for the grid initialization

```

```

00108     NetCDFProcessor<DT> processor(file_path, variable_names);
00109
00110     // Retrieve variable shapes
00111     const auto &lon_shape = processor.getVariableShape(lon_var);
00112     const auto &lat_shape = processor.getVariableShape(lat_var);
00113
00114     // Retrieve variable data (lon/lat are treated as static and stored under key
00115     // 0.0)
00116     const auto &lon_data = processor.getVariableData(lon_var).at(0.0);
00117     const auto &lat_data = processor.getVariableData(lat_var).at(0.0);
00118
00119     // Use first available time slice for mask_var (works for both time-dependent
00120     // and not)
00121     const auto &mask_map = processor.getVariableData(mask_var);
00122     if (mask_map.empty()) {
00123         throw std::runtime_error("initializeGrid(): mask variable '" + mask_var +
00124                                 "' has no data.");
00125     }
00126     const auto &mask_data = mask_map.begin()->second;
00127
00128     // Check if 'lon' and 'lat' are 1D arrays
00129     if (!(lon_shape.size() == 1 && lat_shape.size() == 1)) {
00130         throw std::runtime_error("initializeGrid(): unsupported dimensions for '" +
00131                                 lon_var + "' and '" + lat_var +
00132                                 "'. Expected 1D lon/lat arrays.");
00133     }
00134
00135     // Sizes of 'lon' and 'lat'
00136     std::size_t n_lon = lon_shape[0];
00137     std::size_t n_lat = lat_shape[0];
00138
00139     grid.num_points = n_lon * n_lat;
00140
00141     grid.lon_rad.resize(grid.num_points);
00142     grid.lat_rad.resize(grid.num_points);
00143     grid.in_the_sea.resize(grid.num_points);
00144
00145     // Convert 'lon' and 'lat' to radians
00146     const DT grad_to_rad_factor = DT(number_pi / 180.);
00147     std::vector<DT> lon_rad_ld(n_lon);
00148     std::vector<DT> lat_rad_ld(n_lat);
00149
00150     for (std::size_t i = 0; i < n_lon; i++) {
00151         lon_rad_ld[i] = lon_data[i] * grad_to_rad_factor;
00152     }
00153
00154     for (std::size_t j = 0; j < n_lat; j++) {
00155         lat_rad_ld[j] = lat_data[j] * grad_to_rad_factor;
00156     }
00157
00158     // Create grid points by combining 'lon' and 'lat'
00159     for (std::size_t j = 0; j < n_lat; j++) {
00160         for (std::size_t i = 0; i < n_lon; i++) {
00161             std::size_t idx = j * n_lon + i;
00162             grid.lon_rad[idx] = lon_rad_ld[i];
00163             grid.lat_rad[idx] = lat_rad_ld[j];
00164         }
00165     }
00166
00167     // Ensure that mask_data size matches grid.num_points
00168     if (mask_data.size() != grid.num_points) {
00169         throw std::runtime_error("initializeGrid(): mask variable '" + mask_var +
00170                                 "' size does not match grid size.");
00171     }
00172
00173     // Determine in_the_sea vector using mask variable (NaN = land)
00174     for (std::size_t idx = 0; idx < grid.num_points; idx++) {
00175         const DT m_value = mask_data[idx];
00176         grid.in_the_sea[idx] = !std::isnan(m_value);
00177     }
00178
00179     return grid;
00180 }
00181
00182 // -----
00183 // Function to populate data from multiple files
00184 // -----
00185 template <typename DT, typename TT = double>
00186 void populateData(const std::vector<std::string> &file_paths,
00187                  const Grid<DT> &grid,
00188                  std::vector<TimeStepData<DT, TT>> &all_time_steps_data,
00189                  const std::vector<std::string> &data_variable_names,
00190                  bool check_land_points = false) {
00191     (void)check_land_points;
00192
00193     for (const auto &file_path : file_paths) {
00194         try {

```

```

00209     NetCDFProcessor<DT, TT> processor(file_path, data_variable_names);
00210
00211     // Processing function for data population
00212     auto processFunc = [&](const std::string &var_name, TT time_value,
00213                          const std::vector<DT> &data_vector) {
00214         // Ensure data size matches grid size
00215         if (data_vector.size() != grid.num_points) {
00216             throw std::runtime_error("Data size (" +
00217                                     std::to_string(data_vector.size()) +
00218                                     ") does not match grid size (" +
00219                                     std::to_string(grid.num_points) + ").");
00220         }
00221
00222         // Find or create TimeStepData for this time_value
00223         auto it =
00224             std::find_if(all_time_steps_data.begin(), all_time_steps_data.end(),
00225                         [&](const TimeStepData<DT, TT> &tsd) {
00226                             return tsd.time_value == time_value;
00227                         });
00228
00229         if (it == all_time_steps_data.end()) {
00230             // Create a new TimeStepData
00231             TimeStepData<DT, TT> tsd;
00232             tsd.time_value = time_value;
00233
00234             // Initialize data vectors for each variable
00235             for (const auto &var : data_variable_names) {
00236                 tsd.variable_data[var].resize(grid.num_points,
00237                                                std::numeric_limits<DT>::quiet_NaN());
00238             }
00239
00240             all_time_steps_data.push_back(std::move(tsd));
00241             it = std::prev(all_time_steps_data.end());
00242         }
00243
00244         // Populate data for the current variable
00245         auto &var_data = it->variable_data[var_name];
00246
00247         for (std::size_t i = 0; i < grid.num_points; i++) {
00248             if (grid.in_the_sea[i]) {
00249                 var_data[i] = data_vector[i];
00250             } else {
00251                 var_data[i] = std::numeric_limits<DT>::quiet_NaN();
00252             }
00253         }
00254     };
00255
00256     // Process variables and populate data
00257     processor.processVariables(processFunc);
00258 } catch (const std::exception &e) {
00259     std::cerr << "Error processing data from file '" << file_path
00260               << "': " << e.what() << std::endl;
00261     continue;
00262 }
00263 }
00264 }
00265
00266 // Function to extract sea point indices
00267 template <typename DT>
00268 std::vector<std::size_t> getSeaPointIndices(const Grid<DT> &grid) {
00269     std::vector<std::size_t> sea_point_indices;
00270     sea_point_indices.reserve(grid.num_points / 2);
00271
00272     for (std::size_t idx = 0; idx < grid.num_points; idx++) {
00273         if (grid.in_the_sea[idx]) {
00274             sea_point_indices.push_back(idx);
00275         }
00276     }
00277     return sea_point_indices;
00278 }
00279
00280 template <typename TT>
00281 std::vector<TT> generateExpectedTimeSteps(TT start_time, TT end_time,
00282                                          TT time_step_interval) {
00283     std::vector<TT> expected_time_steps;
00284     for (TT t = start_time; t <= end_time; t += time_step_interval) {
00285         expected_time_steps.push_back(t);
00286     }
00287     return expected_time_steps;
00288 }
00289
00300 template <typename DT, typename TT>
00301 void collectTimeSeriesData(
00302     const Grid<DT> &grid, const std::vector<std::size_t> &sea_point_indices,
00303     const std::vector<std::string> &variable_names,
00304     const std::vector<std::string> &file_paths,
00305     std::map<std::string, std::vector<vector<DT>>> &variable_time_series,

```

```

00308     std::vector<TT> &expected_time_steps, TT time_step_interval) {
00309 (void)grid;
00310
00311 // Determine the start and end times
00312 TT earliest_time = std::numeric_limits<TT>::max();
00313 TT latest_time = std::numeric_limits<TT>::lowest();
00314 std::map<TT, std::string> time_to_file_map;
00315
00316 // Collect all available time steps from the files
00317 std::set<TT> actual_time_steps_set;
00318
00319 for (const auto &file_path : file_paths) {
00320     try {
00321         NetCDFProcessor<DT, TT> processor(file_path, variable_names);
00322
00323         // Get time values
00324         const auto &time_values = processor.getTimeValues();
00325
00326         for (const auto &time_step : time_values) {
00327             actual_time_steps_set.insert(time_step);
00328             time_to_file_map[time_step] = file_path;
00329             earliest_time = std::min(earliest_time, time_step);
00330             latest_time = std::max(latest_time, time_step);
00331         }
00332     } catch (const std::exception &e) {
00333         std::cerr << "Error processing data from file '" << file_path
00334             << "': " << e.what() << std::endl;
00335         continue;
00336     }
00337 }
00338
00339 if (actual_time_steps_set.empty()) {
00340     throw std::runtime_error("No time steps found in data files.");
00341 }
00342
00343 #ifndef DATA_PREP_DEBUG_TIME_STEPS
00344 std::cerr << "\n=== Actual time steps found in data files ===\n";
00345 std::cerr << "Earliest: " << earliest_time << "\n";
00346 std::cerr << "Latest: " << latest_time << "\n";
00347 std::cerr << "All actual time steps:\n";
00348 for (auto t : actual_time_steps_set)
00349     std::cerr << t << " ";
00350 std::cerr << "\n\n";
00351 #endif
00352
00353 // Generate expected time steps
00354 expected_time_steps =
00355     generateExpectedTimeSteps(earliest_time, latest_time, time_step_interval);
00356 std::size_t num_time_steps = expected_time_steps.size();
00357
00358 #ifndef DATA_PREP_DEBUG_TIME_STEPS
00359 std::cerr << "=== Expected time steps (" << num_time_steps << ") ===\n";
00360 #endif
00361
00362 // Initialize time series with NaNs
00363 for (const auto &var_name : variable_names) {
00364     variable_time_series[var_name].resize(sea_point_indices.size());
00365     for (auto &ts_data : variable_time_series[var_name]) {
00366         ts_data.resize(num_time_steps, std::numeric_limits<DT>::quiet_NaN());
00367     }
00368 }
00369
00370 // Map actual time steps to indices in expected time steps
00371 std::map<TT, std::size_t> time_step_indices;
00372 for (std::size_t idx = 0; idx < expected_time_steps.size(); idx++) {
00373     time_step_indices[expected_time_steps[idx]] = idx;
00374 }
00375
00376 // Load data and place it in the correct position in time series
00377 for (const auto &[time_step, file_path] : time_to_file_map) {
00378     try {
00379         NetCDFProcessor<DT, TT> processor(file_path, variable_names);
00380
00381         // Load data for this time step
00382         std::map<std::string, std::vector<DT>> data_at_time;
00383
00384         for (const auto &var_name : variable_names) {
00385             const auto data_vector =
00386                 processor.getVariableDataAtTime(var_name, time_step);
00387
00388             // Collect data for sea points
00389             std::vector<DT> sea_point_data;
00390             sea_point_data.reserve(sea_point_indices.size());
00391             for (auto idx : sea_point_indices) {
00392                 sea_point_data.push_back(data_vector[idx]);
00393             }
00394             data_at_time[var_name] = std::move(sea_point_data);

```

```

00395     }
00396
00397     // Find the index in expected_time_steps
00398     auto it = time_step_indices.find(time_step);
00399     if (it != time_step_indices.end()) {
00400         std::size_t t_idx = it->second;
00401
00402         // Place data into time series
00403         for (const auto &var_name : variable_names) {
00404             const auto &var_data = data_at_time.at(var_name);
00405             auto &time_series = variable_time_series[var_name];
00406
00407             for (std::size_t i = 0; i < sea_point_indices.size(); i++) {
00408                 time_series[i][t_idx] = var_data[i];
00409             }
00410         }
00411     } else {
00412         std::cerr << "Warning: Time step " << time_step
00413             << " not in expected time steps." << std::endl;
00414     }
00415 } catch (const std::exception &e) {
00416     std::cerr << "Error processing data from file '" << file_path
00417         << "': " << e.what() << std::endl;
00418     continue;
00419 }
00420 }
00421 }
00422
00423 template <typename DT, typename TT = double>
00424 void interpolateMissingData(std::vector<DT> &time_series,
00425     const std::vector<TT> &time_points) {
00426     std::size_t n = time_series.size();
00427     if (n != time_points.size()) {
00428         throw std::runtime_error("Time series and time points size mismatch.");
00429     }
00430
00431     std::size_t start_idx = 0;
00432     while (start_idx < n) {
00433         // Skip over existing data
00434         while (start_idx < n && !std::isnan(time_series[start_idx])) {
00435             start_idx++;
00436         }
00437
00438         if (start_idx == n)
00439             break;
00440
00441         // Find the end of the NaN segment
00442         std::size_t end_idx = start_idx;
00443         while (end_idx < n && std::isnan(time_series[end_idx])) {
00444             end_idx++;
00445         }
00446
00447         // Handle edge cases
00448         if (start_idx == 0 || end_idx == n) {
00449             // Cannot interpolate, decide how to handle
00450             DT fill_value;
00451             if (start_idx == 0 && end_idx < n) {
00452                 // Missing data at the beginning, fill with first non-NaN value
00453                 fill_value = time_series[end_idx];
00454             } else if (end_idx == n && start_idx > 0) {
00455                 // Missing data at the end, fill with last non-NaN value
00456                 fill_value = time_series[start_idx - 1];
00457             } else {
00458                 // Entire time series is NaN
00459                 fill_value = static_cast<DT>(0);
00460             }
00461
00462             for (std::size_t idx = start_idx; idx < end_idx; idx++) {
00463                 time_series[idx] = fill_value;
00464             }
00465         } else {
00466             // Interpolate between existing data
00467             DT y0 = time_series[start_idx - 1];
00468             DT y1 = time_series[end_idx];
00469             TT x0 = time_points[start_idx - 1];
00470             TT x1 = time_points[end_idx];
00471
00472             for (std::size_t idx = start_idx; idx < end_idx; idx++) {
00473                 TT xi = time_points[idx];
00474                 DT yi = y0 + ((y1 - y0) * (xi - x0) / (x1 - x0));
00475                 time_series[idx] = yi;
00476             }
00477         }
00478         start_idx = end_idx;
00479     }
00480 }
00481 }

```



```

00482
00483 template <typename DT, typename TT = double>
00484 void createSplinesForSeaPoints(
00485     std::map<std::string, std::vector<std::vector<DT>>> &variable_time_series,
00486     std::map<std::string, std::vector<TimeSpline<DT, TT>>> &variable_splines,
00487     TT start_time, TT time_step) {
00488     for (auto &[var_name, time_series_data] : variable_time_series) {
00489         std::vector<TimeSpline<DT, TT> &splines = variable_splines[var_name];
00490         splines.resize(time_series_data.size());
00491
00492         // each i touches a unique element -> safe to move in parallel
00493         oneapi::tbb::parallel_for(
00494             std::size_t(0), time_series_data.size(), [&](std::size_t i) {
00495                 auto &data = time_series_data[i];
00496                 TimeSpline<DT, TT> spline;
00497                 spline.values = std::move(data); // <-- move, no copy
00498                 spline.createSpline(start_time, time_step);
00499                 splines[i] = std::move(spline);
00500             });
00501
00502         // optional: release the now-empty storage for this variable
00503         time_series_data.clear();
00504         time_series_data.shrink_to_fit();
00505     }
00506
00507     // optional: free the whole (now moved-from) map early
00508     variable_time_series.clear();
00509     variable_time_series = {};
00510 }
00511
00512 // -----
00513 // helper: zero-pad an integer to 2 characters
00514 // -----
00515 inline std::string formatTwo(int v) {
00516     std::ostringstream oss;
00517     oss << std::setw(2) << std::setfill('0') << v;
00518     return oss.str();
00519 }
00520
00521 // Helper function: Convert a Date into a string formatted as "YYYY-MM-DD".
00522 inline std::string formatDate(Date const &d) {
00523     std::ostringstream oss;
00524     oss << std::setw(4) << std::setfill('0') << d.getYear() << "_" << std::setw(2)
00525         << std::setfill('0') << d.getMonth() << "_" << std::setw(2)
00526         << std::setfill('0') << d.getDay();
00527     return oss.str();
00528 }
00529
00530 // -----
00531 // Utility: parse "YYYY-MM-DD" or "YYYY_MM_DD" into Date
00532 // -----
00533 inline Date parseDate(const std::string &s) {
00534     std::regex re(R"((\d{4})[_-](\d{2})[_-](\d{2}))");
00535     std::smatch m;
00536     if (!std::regex_match(s, m, re)) {
00537         std::cerr << "Invalid date format: " << s << " (expected YYYY-MM-DD)\n";
00538         std::exit(EXIT_FAILURE);
00539     }
00540     int y = std::stoi(m[1]), mth = std::stoi(m[2]), d = std::stoi(m[3]);
00541     return Date(d, mth, y);
00542 }
00543
00544 // comma-separated list of args to vector<string>
00545 inline std::vector<std::string> splitArgs(const std::string &s) {
00546     std::vector<std::string> v;
00547     std::stringstream ss(s);
00548     std::string item;
00549     while (std::getline(ss, item, ',')) {
00550         if (!item.empty())
00551             v.push_back(item);
00552     }
00553     return v;
00554 }
00555
00556 // -----
00557 // helper: expand pattern for one date
00558 // -----
00559 inline std::string expandPattern(const std::string &pattern, const Date &d,
00560                                 int level) {
00561     std::ostringstream lev_ss;
00562     lev_ss << std::setw(4) << std::setfill('0') << level;
00563     std::string out = pattern;
00564
00565     const std::pair<std::regex, std::string> subs[] = {
00566         {std::regex("\\{YYYY\\}"), std::to_string(d.getYear())},
00567         {std::regex("\\{MM\\}"), formatTwo(d.getMonth())},
00568         {std::regex("\\{DD\\}"), formatTwo(d.getDay())},

```

```

00569         {std::regex("\\{LEV\\}"), lev_ss.str()});
00570     for (auto const &s : subs)
00571         out = std::regex_replace(out, s.first, s.second);
00572     return out;
00573 }
00574 }
00575
00576 // -----
00577 // build path vectors for both sub-domains
00578 // -----
00579 inline void buildPathVectors(const DateRange &dr, int level,
00580                             const std::string &path_1,
00581                             const std::string &path_2,
00582                             std::vector<std::string> &out_1,
00583                             std::vector<std::string> &out_2) {
00584     out_1.clear();
00585     out_2.clear();
00586     out_1.reserve(dr.length());
00587     out_2.reserve(dr.length());
00588
00589     for (const Date &d : dr) {
00590         out_1.push_back(expandPattern(path_1, d, level));
00591         out_2.push_back(expandPattern(path_2, d, level));
00592     }
00593 }
00594
00595 using DataType = float;
00596 using TimeType = double;
00597
00598 // -----
00599 // Spline containers |
00600 // -----
00601
00602 template <class DT> using spline_vec = std::vector<TimeSpline<DT, TimeType>>;
00603
00604 template <class DT> using spline_map = std::map<std::string, spline_vec<DT>>;
00605
00606 struct MonthData // lives one month, then discarded
00607 {
00608     Date month{1, 1, 2000}; // default so MonthData() works
00609     spline_map<DataType> spl_1; // sub-domain 1 ("left half")
00610     spline_map<DataType> spl_2; // sub-domain 2 ("right half")
00611     spline_map<DataType> spl;
00612     Grid<DataType> grid_1, grid_2;
00613     std::vector<std::size_t> sea_idx_1, sea_idx_2;
00614     std::vector<DataType> sea_x, sea_y;
00615     PointCloud<DataType> sea_cloud, land_cloud;
00616     std::unique_ptr<KDTree<DataType>> kd_sea{nullptr};
00617     std::unique_ptr<KDTree<DataType>> kd_land{nullptr};
00618 };
00619
00620 // -----
00621 // Helper: Join two sets of splines for each variable
00622 // -----
00623 template <typename DT>
00624 void joinSplines(
00625     std::map<std::string, std::vector<TimeSpline<DT, TimeType>>> &&splines_1,
00626     std::map<std::string, std::vector<TimeSpline<DT, TimeType>>> &&splines_2,
00627     std::map<std::string, std::vector<TimeSpline<DT, TimeType>>> &&joined_splines) {
00628     joined_splines = std::move(splines_1); // take ownership
00629
00630     for (auto &kv : splines_2) {
00631         auto &vec = joined_splines[kv.first];
00632         vec.insert(vec.end(), std::make_move_iterator(kv.second.begin()),
00633             std::make_move_iterator(kv.second.end())); // move, not copy
00634     }
00635     for (auto &kv : splines_2)
00636         kv.second.shrink_to_fit();
00637     splines_2.clear(); // optional: drop map nodes
00638 }
00639
00640
00641 inline void dumpCoastCSV(const MonthData &m, const std::string &path,
00642                         int stride = 8) {
00643     std::ofstream out(path);
00644     if (!out)
00645         return;
00646     out << std::setprecision(10);
00647     // header
00648     out << "lon_deg,lat_deg,label\n";
00649     // land points (used for beaching)
00650     for (size_t i = 0; i + 1 < m.land_cloud.pts.size(); i += 2 * stride) {
00651         double lon_deg = m.land_cloud.pts[i] * 180. / number_pi;
00652         double lat_deg = m.land_cloud.pts[i + 1] * 180. / number_pi;
00653         out << lon_deg << ", " << lat_deg << ",land\n";
00654     }
00655     // (optional) sea points

```

```

00656 // for (size_t i=0;i+1<m.sea_cloud.pts.size();i+=2*stride)
00657 //{
00658 //     double lon_deg=m.sea_cloud.pts[i] *180./pi;
00659 //     double lat_deg=m.sea_cloud.pts[i+1]*180./pi;
00660 //     out<<"lon_deg<<","lat_deg<<","sea<<n";
00661 //}
00662 }
00663
00664 inline void buildKdTrees(MonthData &m) {
00665     auto kd_params = nanoflann::KDTreeSingleIndexAdaptorParams(10);
00666     m.kd_sea = std::make_unique<KDTree<DataType>>(2, m.sea_cloud, kd_params);
00667     m.kd_land = std::make_unique<KDTree<DataType>>(2, m.land_cloud, kd_params);
00668     m.kd_sea->buildIndex();
00669     m.kd_land->buildIndex();
00670     assert(m.kd_sea && m.kd_land);
00671     assert(m.sea_cloud.kdtree_get_point_count() > 0);
00672     assert(m.land_cloud.kdtree_get_point_count() > 0);
00673 }
00674
00675 inline MonthData loadOneMonth(Date month, // 2000-01-01, 2000-02-01 ...
00676                               int level, const std::string &pattern_1,
00677                               const std::string &pattern_2,
00678                               const std::vector<std::string> &grid_vars,
00679                               const std::vector<std::string> &data_vars,
00680                               TimeType dt_seconds) {
00681     // ----- 1. daily file list for *this* month (+ padding) -----
00682     Date d_0 = month.addDays(-1); // day before 1st
00683     Date d_1 = month.addMonths(1).addDays(1); // day after last
00684     DateRange span(d_0, d_1);
00685
00686     std::vector<std::string> fp_1, fp_2;
00687     buildPathVectors(span, level, pattern_1, pattern_2, fp_1, fp_2);
00688
00689     // ----- 2. minimal grid init (first file of each sub-domain) -----
00690     MonthData m;
00691     m.month = month;
00692     m.grid_1 = initializeGrid<DataType>(fp_1.front(), grid_vars);
00693     m.grid_2 = initializeGrid<DataType>(fp_2.front(), grid_vars);
00694
00695     auto norm_pi_inplace = [](std::vector<DataType> &a) {
00696         for (auto &L : a) {
00697             while (L <= -number_pi)
00698                 L += 2 * number_pi;
00699             while (L > number_pi)
00700                 L -= 2 * number_pi;
00701         }
00702     };
00703
00704     norm_pi_inplace(m.grid_1.lon_rad);
00705     norm_pi_inplace(m.grid_2.lon_rad);
00706
00707     m.sea_idx_1 = getSeaPointIndices(m.grid_1);
00708     m.sea_idx_2 = getSeaPointIndices(m.grid_2);
00709
00710     // ----- 3. Build sea & land clouds + KD-trees for this month */
00711     auto buildClouds = [&](MonthData &m) {
00712         const std::size_t n_1 = m.grid_1.lon_rad.size();
00713         const std::size_t n_2 = m.grid_2.lon_rad.size();
00714         const std::size_t total = n_1 + n_2;
00715
00716         // 1) SEA CLOUD (order must match spline vectors: spl_1 then spl_2)
00717         m.sea_cloud.pts.reserve((m.sea_idx_1.size() + m.sea_idx_2.size()) * 2);
00718
00719         // domain 1
00720         for (auto idx : m.sea_idx_1) {
00721             m.sea_cloud.pts.push_back(m.grid_1.lon_rad[idx]);
00722             m.sea_cloud.pts.push_back(m.grid_1.lat_rad[idx]);
00723         }
00724         // domain 2
00725         for (auto idx : m.sea_idx_2) {
00726             m.sea_cloud.pts.push_back(m.grid_2.lon_rad[idx]);
00727             m.sea_cloud.pts.push_back(m.grid_2.lat_rad[idx]);
00728         }
00729
00730         m.sea_x.clear();
00731         m.sea_y.clear();
00732         m.sea_x.reserve(m.sea_cloud.kdtree_get_point_count());
00733         m.sea_y.reserve(m.sea_cloud.kdtree_get_point_count());
00734
00735         for (size_t i = 0; i < m.sea_cloud.kdtree_get_point_count(); i++) {
00736             m.sea_x.push_back(m.sea_cloud.kdtree_get_pt(i, 0));
00737             m.sea_y.push_back(m.sea_cloud.kdtree_get_pt(i, 1));
00738         }
00739
00740         // 2) LAND CLOUD (anything not in sea_idx_*; order doesn't matter)
00741         std::vector<char> is_sea(total, 0);
00742         for (auto i : m.sea_idx_1)

```

```

00743     is_sea[i] = 1;
00744     for (auto i : m.sea_idx_2)
00745         is_sea[i + n_1] = 1;
00746
00747     m.land_cloud.pts.reserve((total - m.sea_idx_1.size() - m.sea_idx_2.size()) *
00748                             2);
00749
00750     for (std::size_t idx = 0; idx < total; idx++) {
00751         if (is_sea[idx])
00752             continue;
00753
00754         DataType lon, lat;
00755         if (idx < n_1) {
00756             lon = m.grid_1.lon_rad[idx];
00757             lat = m.grid_1.lat_rad[idx];
00758         } else {
00759             std::size_t jdx = idx - n_1;
00760             lon = m.grid_2.lon_rad[jdx];
00761             lat = m.grid_2.lat_rad[jdx];
00762         }
00763         m.land_cloud.pts.push_back(lon);
00764         m.land_cloud.pts.push_back(lat);
00765     }
00766
00767     const auto sea_N = m.sea_cloud.kdtree_get_point_count();
00768     const auto land_N = m.land_cloud.kdtree_get_point_count();
00769
00770     if (sea_N == 0) {
00771         throw std::runtime_error("No SEA points for month " +
00772                                 formatDate(m.month) +
00773                                 " (check masks/variables/region).");
00774     }
00775     if (land_N == 0) {
00776         throw std::runtime_error("No LAND points for month " +
00777                                 formatDate(m.month) +
00778                                 " (check masks/variables/region).");
00779     }
00780
00781     // 3) KD trees
00782     // auto kd_params=nanoflann::KDTreeSingleIndexAdaptorParams(10);
00783     // m.kd_sea =std::make_unique<KDTree<DataType>(2,m.sea_cloud ,kd_params);
00784     // m.kd_land=std::make_unique<KDTree<DataType>(2,m.land_cloud,kd_params);
00785     // m.kd_sea ->buildIndex();
00786     // m.kd_land->buildIndex();
00787 };
00788
00789 buildClouds(m);
00790
00791 dumpCoastCSV(m, "coast_" + formatDate(m.month) + ".csv", 8);
00792
00793 // ----- 3. read time series + build splines -----
00794 std::vector<TimeType> time_steps_1, time_steps_2;
00795 // spline_map<DataType> tmp_spl_1,tmp_spl_2;
00796 std::map<std::string, std::vector<std::vector<DataType>>> raw_time_series_1,
00797         raw_time_series_2;
00798
00799 collectTimeSeriesData(m.grid_1, m.sea_idx_1, data_vars, fp_1,
00800                     raw_time_series_1, time_steps_1, dt_seconds);
00801 collectTimeSeriesData(m.grid_2, m.sea_idx_2, data_vars, fp_2,
00802                     raw_time_series_2, time_steps_2, dt_seconds);
00803
00804 if (time_steps_1.empty() || time_steps_2.empty()) {
00805     throw std::runtime_error(
00806         "No time steps found for month " + formatDate(m.month) +
00807         " (check file patterns / level / variable names).");
00808 }
00809
00810 /* fill gaps, build splines */
00811 auto patch = [&](auto &ts, auto &steps) {
00812     for (auto &v : data_vars)
00813         for (auto &vec : ts[v])
00814             interpolateMissingData<DataType>(vec, steps);
00815 };
00816 patch(raw_time_series_1, time_steps_1);
00817 patch(raw_time_series_2, time_steps_2);
00818
00819 createSplinesForSeaPoints(std::move(raw_time_series_1), m.spl_1,
00820                          static_cast<TimeType>(time_steps_1.front()),
00821                          dt_seconds);
00822 createSplinesForSeaPoints(std::move(raw_time_series_2), m.spl_2,
00823                          static_cast<TimeType>(time_steps_2.front()),
00824                          dt_seconds);
00825
00826 m.spl.clear();
00827 joinSplines<DataType>(std::move(m.spl_1), m.spl_2, m.spl);
00828 m.spl_2.clear(); // we moved spl_1; spl_2 was copied into m.spl
00829 m.spl_2 = {}; // drop its map/vector overhead

```

```

00830
00831 // Optional sanity check (debug builds)
00832 for (auto &kv : m.spl)
00833     assert(kv.second.size() == m.sea_cloud.kdtree_get_point_count());
00834
00835 return m;
00836 }
00837
00838 // -----
00839 // Build a NeighborsData object for one month (uses that month's KD-sea)
00840 // -----
00841 inline NeighborsData<DataType, TimeType>
00842 makeNeighborsData(const MonthData &m, DataType search_radius_rad,
00843                 DataType shape_param) {
00844     // const size_t N=m.sea_cloud.kdtree_get_point_count();
00845
00846     DataType radius_squared = search_radius_rad * search_radius_rad;
00847     return NeighborsData<DataType, TimeType>(*m.kd_sea, m.sea_x, m.sea_y,
00848                                             radius_squared, shape_param);
00849 }
00850
00851 // seconds since 2000-01-01 00:00
00852 inline TimeType secondsSince2000(const Date &d) {
00853     return static_cast<TimeType>(d.secondsSince(Date(1, 1, 2000)));
00854 }
00855
00856 inline Date dateFromSecondsSince2000(TimeType sec) {
00857     // brute force
00858     Date ref(1, 1, 2000);
00859     long long days = static_cast<long long>(
00860         std::floor(static_cast<long double>(sec) / 86400.L));
00861     return ref.addDays(static_cast<int>(days));
00862 }
00863
00864 enum class EndReason : uint8_t {
00865     none = 0,
00866     landed = 1,
00867     neverleft = 2,
00868     lifetime = 3
00869 };
00870
00871 // -----
00872 // Structure to store final trajectory results for a particle
00873 // -----
00874 template <typename DT, typename TT> struct Particle {
00875     DT init_lon; // initial longitude (radians)
00876     DT init_lat; // initial latitude (radians)
00877     TT init_time; // initial time (seconds since 01.01.2000 0:00)
00878     DT lon; // final longitude (radians)
00879     DT lat; // final latitude (radians)
00880     TT time; // final time (seconds since 01.01.2000 0:00)
00881     EndReason reason;
00882 };
00883
00884 } // namespace particle_tracker
00885
00886 #endif // DATA_PREP_UTILS_HPP

```

6.4 include/particle_tracker/detail/datetime_utils.hpp File Reference

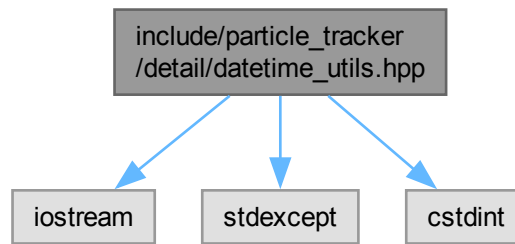
Small date and date-range utilities (civil calendar, Gregorian rules).

```

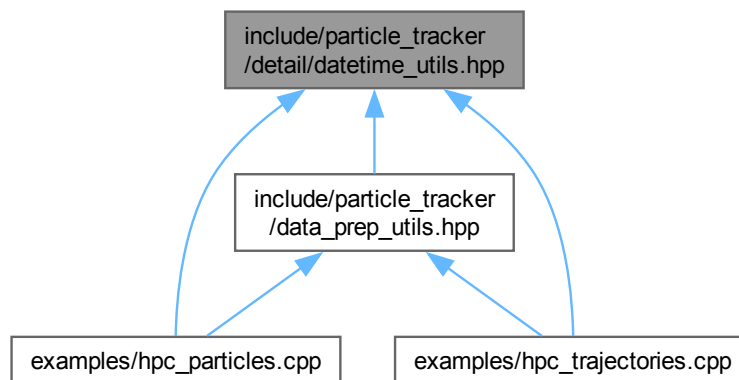
#include <iostream>
#include <stdexcept>
#include <cstdint>

```

Include dependency graph for `datetime_utils.hpp`:



This graph shows which files directly or indirectly include this file:



Classes

- class [Date](#)
Represents a calendar date (day/month/year) in the Gregorian calendar.
- class [DateRange](#)
Inclusive date range [startDate .
- struct [DateRange::iterator](#)

6.4.1 Detailed Description

Small date and date-range utilities (civil calendar, Gregorian rules).

This header provides:

- [Date](#) : a simple civil date (day, month, year) with basic arithmetic.
- [DateRange](#) : an inclusive range [start..end] with an iterator and `length()`.

6.4.1.1 Notes / assumptions

- Uses proleptic Gregorian calendar rules (leap years: divisible by 4, except centuries not divisible by 400).
- No timezone logic. This is pure calendar arithmetic.
- daysSince/hoursSince/secondsSince are computed via an O(1) day-number conversion (not by looping day-by-day).

Definition in file [datetime_utils.hpp](#).

6.5 datetime_utils.hpp

[Go to the documentation of this file.](#)

```

00001 #pragma once
00002
00003 #include <iostream>
00004 #include <stdexcept>
00005 #include <cstdint>
00006
00023
00044 class Date
00045 {
00046     private:
00047         int day,month,year;
00048
00049         // Helper: leap year test (Gregorian).
00050         bool isLeapYear(int y) const
00051         {
00052             return (y%4==0) && ((y%100!=0) || (y%400==0));
00053         }
00054
00055         // Helper: days in month (1..12) for given year.
00056         int daysInMonth(int month,int year) const
00057         {
00058             switch(month)
00059             {
00060                 case 1: case 3: case 5: case 7: case 8: case 10: case 12:
00061                     return 31;
00062                 case 4: case 6: case 9: case 11:
00063                     return 30;
00064                 case 2:
00065                     return isLeapYear(year) ? 29 : 28;
00066                 default:
00067                     throw std::runtime_error("Invalid month");
00068             }
00069         }
00070
00071         // Helper: convert civil date to a monotonically increasing day number.
00072         //
00073         // This is an O(1) algorithm (Howard Hinnant / "civil_from_days" family).
00074         // It works for a proleptic Gregorian calendar and avoids looping by days.
00075         //
00076         // Returns days since 1970-01-01 (can be negative for earlier dates).
00077         static std::int64_t daysFromCivil(int y,int m,int d)
00078         {
00079             // Shift March to the start of the year to make leap handling simple.
00080             y -= (m <= 2);
00081             const int era = (y >= 0 ? y : y-399) / 400;
00082             const unsigned yoe = static_cast<unsigned>(y - era * 400);           // [0, 399]
00083             const unsigned doy = (153u*(static_cast<unsigned>(m + (m > 2 ? -3 : 9))) + 2u)/5u
00084                                 + static_cast<unsigned>(d) - 1u;                 // [0, 365]
00085             const unsigned doe = yoe*365u + yoe/4u - yoe/100u + doy;             // [0, 146096]
00086             return static_cast<std::int64_t>(era)*146097 + static_cast<std::int64_t>(doe) - 719468;
00087         }
00088
00089     public:
00094     Date(int day,int month,int year) : day(day),month(month),year(year)
00095     {
00096         if (month<1 || month>12)
00097             throw std::runtime_error("Date: month out of range");
00098
00099         const int dim=daysInMonth(month,year);
00100         if (day<1 || day>dim)
00101             throw std::runtime_error("Date: day out of range for month/year");

```

```

00102     }
00103
00104     // Increment the date by one day (with rollover).
00105     void increment()
00106     {
00107         day++;
00108         if (day>daysInMonth(month,year))
00109         {
00110             day=1;
00111             month++;
00112             if (month>12)
00113             {
00114                 month=1;
00115                 year++;
00116             }
00117         }
00118     }
00119
00120     // Decrement the date by one day (with rollover).
00121     void decrement()
00122     {
00123         if (day>1)
00124         {
00125             --day;
00126             return;
00127         }
00128
00129         if (month==1)
00130         {
00131             month=12;
00132             --year;
00133         }
00134         else
00135         {
00136             --month;
00137         }
00138
00139         day=daysInMonth(month,year);
00140     }
00141
00142     // Comparisons (lexicographic by year, month, day).
00143     bool operator<=(const Date& other) const
00144     {
00145         if (year<other.year) return true;
00146         if (year==other.year && month<other.month) return true;
00147         if (year==other.year && month==other.month && day<=other.day) return true;
00148         return false;
00149     }
00150
00151     bool operator<(const Date& other) const
00152     {
00153         if (year<other.year) return true;
00154         if (year==other.year && month<other.month) return true;
00155         if (year==other.year && month==other.month && day<other.day) return true;
00156         return false;
00157     }
00158
00159     bool operator==(const Date& other) const
00160     {
00161         return (year==other.year && month==other.month && day==other.day);
00162     }
00163
00164     bool operator>(const Date& other) const {return other<*this;}
00165
00166     bool operator>=(const Date& other) const {return !(*this<other);}
00167
00168     bool operator!=(const Date& other) const {return !(*this==other);}
00169
00170     // Stream output (debug-friendly).
00171     friend std::ostream& operator<<(std::ostream& os,const Date& date)
00172     {
00173         os<<"Month: "<<date.month<<"", Day: "<<date.day<<"", Year: "<<date.year;
00174         return os;
00175     }
00176
00177     int getYear() const {return year;}
00178
00179     int getMonth() const {return month;}
00180
00181     int getDay() const {return day;}
00182
00193     int daysSince(const Date& other) const
00194     {
00195         const std::int64_t a=daysFromCivil(year,month,day);
00196         const std::int64_t b=daysFromCivil(other.year,other.month,other.day);
00197
00198         // Cast to int: safe for typical simulation ranges (years ~ 1900..2100).

```



```

00199         // If you plan to handle centuries-wide spans, switch return type to int64_t.
00200         return static_cast<int>(a-b);
00201     }
00202
00203     int hoursSince(const Date& other) const
00204     {
00205         int days=daysSince(other);
00206         return days*24;
00207     }
00208
00209     int secondsSince(const Date& other) const
00210     {
00211         int days=daysSince(other);
00212         return days*24*3600;
00213     }
00214
00220     Date addDays(int delta) const
00221     {
00222         if (delta==0) return *this;
00223
00224         Date tmp=*this;
00225         if (delta>0)
00226         {
00227             for (int i=0;i<delta;i++) tmp.increment();
00228         }
00229         else
00230         {
00231             for (int i=0;i<=-delta;i++) tmp.decrement();
00232         }
00233         return tmp;
00234     }
00235
00242     Date addMonths(int delta) const
00243     {
00244         if (delta==0) return *this;
00245
00246         int y=year;
00247         int m=month;
00248         int d=day;
00249
00250         int index=y*12+(m-1)+delta;
00251         int new_y=index/12;
00252         int new_m=index%12+1;
00253
00254         int max_day=daysInMonth(new_m,new_y);
00255         if (d>max_day) d=max_day;
00256
00257         return Date(d,new_m,new_y);
00258     }
00259
00260     // First day of current month.
00261     Date beginOfMonth() const
00262     {
00263         return Date(1,month,year);
00264     }
00265 };
00266
00273 class DateRange
00274 {
00275     Date startDate,endDate;
00276
00277 public:
00278     // inclusive range [startDate ... endDate]
00279     DateRange(const Date& s,const Date& e) : startDate(s),endDate(e)
00280     {
00281         if (e<s)
00282             throw std::runtime_error("DateRange: endDate is before startDate");
00283     }
00284
00285     struct iterator
00286     {
00287         Date current,last;                // 'last' is inclusive
00288
00289         iterator(const Date& cur,const Date& last) : current(cur),last(last) {}
00290
00291         const Date& operator*() const {return current;}
00292
00293         iterator& operator++()
00294         {
00295             current.increment();
00296             return *this;
00297         }
00298
00299         bool operator!=(const iterator& other) const
00300         {
00301             return current!=other.current;
00302         }

```

```

00303     };
00304
00305     iterator begin() const {return iterator(startDate,endDate); }
00306
00307     iterator end() const
00308     {
00309         Date onePastEnd=endDate;
00310         onePastEnd.increment();
00311         return iterator(onePastEnd,endDate);
00312     }
00313
00314     // number of days in the range (inclusive)
00315     int length() const
00316     {
00317         return endDate.daysSince(startDate)+1;
00318     }
00319 };

```

6.6 include/particle_tracker/detail/ncdf_utils.hpp File Reference

Header-only NetCDF-C++4 helpers for loading gridded variables (optionally time-dependent).

```

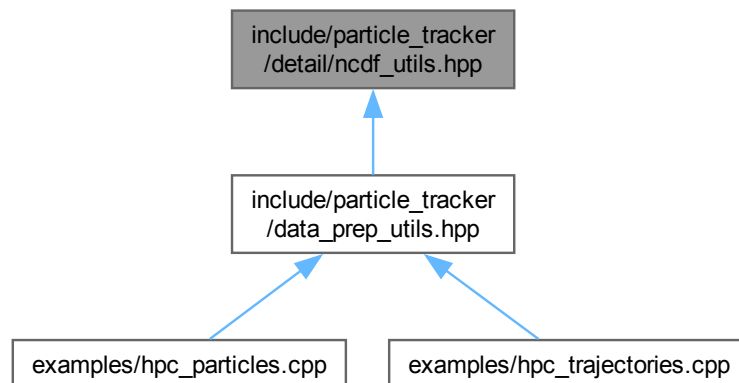
#include <algorithm>
#include <atomic>
#include <cctype>
#include <cstdint>
#include <cmath>
#include <exception>
#include <functional>
#include <iostream>
#include <limits>
#include <map>
#include <set>
#include <stdexcept>
#include <string>
#include <type_traits>
#include <utility>
#include <vector>
#include <netcdf>

```

Include dependency graph for ncdf_utils.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [particle_tracker::NetCDFDataLoader< DataType, TimeType >](#)
Load one or more NetCDF files and store selected variables keyed by time.
- class [particle_tracker::NetCDFProcessor< DataType, TimeType >](#)
Convenience wrapper to load a single file and iterate over selected variables and times.

Macros

- `#define PARTICLE\_TRACKER\_NCDF\_WARN 1`
- `#define PARTICLE\_TRACKER\_NCDF\_STRICT 0`

Functions

- void [particle_tracker::detail::warn_once](#) (const std::string &msg)
- void [particle_tracker::detail::warn_once_per_file](#) (const std::string &file, const std::string &msg)
- std::string [particle_tracker::detail::to_lower](#) (std::string s)
- std::vector< std::string > [particle_tracker::ncdf_utils::checkFileHeaders](#) (const std::string &fileName, const std::vector< std::string > &requiredVariables, int expectedTimeCount)
Validate netCDF file headers (metadata-only; does not read full arrays).

6.6.1 Detailed Description

Header-only NetCDF-C++4 helpers for loading gridded variables (optionally time-dependent).

This header contains:

- [particle_tracker::NetCDFDataLoader](#) : loads one or more NetCDF files and stores variables by time.
- [particle_tracker::NetCDFProcessor](#) : convenience wrapper to iterate over loaded data.
- [particle_tracker::ncdf_utils::checkFileHeaders](#) : fast header validation without loading full arrays.

6.6.1.1 Design goals

- Practical: load variables into `std::vector` for later interpolation / tracking.
- Robust failure modes: prefer explicit warnings/errors over silent wrong behavior.
- Header-only.

6.6.1.2 Important unit policy (do NOT “fix” units here)

- This loader does NOT normalize or convert lon/lat units.
- It only reads raw arrays and applies `scale_factor/add_offset` and missing-value handling.
- Unit conversion / longitude wrapping should happen in higher-level code where the expected convention is known.

6.6.1.3 Missing-value policy

- If `_FillValue` or `missing_value` attributes exist, those are treated as missing.
- Otherwise (fallback) a sentinel of -30000 is treated as missing for non-coordinate variables.
- Missing samples are stored as `quiet_NaN()` in the output `DataType`.

6.6.1.4 Time policy

- Reads variable "time" if present.
- Attempts to interpret "time:units" to convert to seconds (hours/days/seconds).
- Quantizes times using `llround()` so that common products have stable integer-second steps.

6.6.1.5 Optional warning / strict mode

- Define `PARTICLE_TRACKER_NCDF_WARN=0` to disable warnings to `std::cerr`.
- Define `PARTICLE_TRACKER_NCDF_STRICT=1` to throw on certain structural problems (e.g. missing "time" while loading time-dependent variables). Default is non-strict (warn + skip).

Definition in file [ncdf_utils.hpp](#).

6.6.2 Macro Definition Documentation

6.6.2.1 PARTICLE_TRACKER_NCDF_STRICT

```
#define PARTICLE_TRACKER_NCDF_STRICT 0
```

Definition at line 66 of file [ncdf_utils.hpp](#).

6.6.2.2 PARTICLE_TRACKER_NCDF_WARN

```
#define PARTICLE_TRACKER_NCDF_WARN 1
```

Definition at line 62 of file [ncdf_utils.hpp](#).

6.6.3 Function Documentation

6.6.3.1 checkFileHeaders()

```
std::vector< std::string > particle_tracker::ncdf_utils::checkFileHeaders (  
    const std::string & fileName,  
    const std::vector< std::string > & requiredVariables,  
    int expectedTimeCount) [inline]
```

Validate netCDF file headers (metadata-only; does not read full arrays).

Definition at line 681 of file [ncdf_utils.hpp](#).

6.6.3.2 to_lower()

```
std::string particle_tracker::detail::to_lower (  
    std::string s) [inline]
```

Definition at line 98 of file [ncdf_utils.hpp](#).

6.6.3.3 warn_once()

```
void particle_tracker::detail::warn_once (  
    const std::string & msg) [inline]
```

Definition at line 79 of file [ncdf_utils.hpp](#).

6.6.3.4 warn_once_per_file()

```
void particle_tracker::detail::warn_once_per_file (  
    const std::string & file,  
    const std::string & msg) [inline]
```

Definition at line 88 of file [ncdf_utils.hpp](#).

6.7 ncdf_utils.hpp

[Go to the documentation of this file.](#)

```

00001 #pragma once
00037
00038 #include <algorithm>    // std::transform, std::find, std::find_if
00039 #include <atomic>       // std::atomic_flag
00040 #include <cctype>       // std::tolower
00041 #include <cstdint>      // std::size_t
00042 #include <cmath>        // std::llround
00043 #include <exception>
00044 #include <functional>
00045 #include <iostream>     // std::cerr
00046 #include <limits>       // std::numeric_limits
00047 #include <map>
00048 #include <set>
00049 #include <stdexcept>    // std::runtime_error
00050 #include <string>
00051 #include <type_traits>
00052 #include <utility>
00053 #include <vector>
00054
00055 #include <ncdf>
00056
00057 //-----
00058 // Compile-time configuration
00059 //-----
00060
00061 #ifndef PARTICLE_TRACKER_NCDF_WARN
00062 #define PARTICLE_TRACKER_NCDF_WARN 1
00063 #endif
00064
00065 #ifndef PARTICLE_TRACKER_NCDF_STRICT
00066 #define PARTICLE_TRACKER_NCDF_STRICT 0
00067 #endif
00068
00069 namespace particle_tracker
00070 {
00071
00072 //-----
00073 // Internal warning helpers (cheap, throttled)
00074 //-----
00075
00076 namespace detail
00077 {
00078 #if PARTICLE_TRACKER_NCDF_WARN
00079     inline void warn_once(const std::string& msg)
00080     {
00081         static std::atomic_flag warned = ATOMIC_FLAG_INIT;
00082         if (!warned.test_and_set(std::memory_order_relaxed))
00083         {
00084             std::cerr << msg << '\n';
00085         }
00086     }
00087
00088     inline void warn_once_per_file(const std::string& file, const std::string& msg)
00089     {
00090         // Kept intentionally cheap: once per process. Include file name in message.
00091         warn_once("particle_tracker::ncdf_utils: " + file + ": " + msg);
00092     }
00093 #else
00094     inline void warn_once(const std::string&) {}
00095     inline void warn_once_per_file(const std::string&, const std::string&) {}
00096 #endif
00097
00098     inline std::string to_lower(std::string s)
00099     {
00100         std::transform(s.begin(), s.end(), s.begin(),
00101             [](unsigned char c){ return static_cast<char>(std::tolower(c)); });
00102         return s;
00103     }
00104 } // namespace detail
00105
00106 //-----
00107 // NetCDFDataLoader
00108 //-----
00109
00126 template<typename DataType, typename TimeType = double>
00127 class NetCDFDataLoader
00128 {
00129     public:
00130         using DataMap = std::map<TimeType, std::vector<DataType>;
00131
00132         NetCDFDataLoader(const std::vector<std::string>& file_paths,
00133             const std::vector<std::string>& variable_names)

```

```

00134     : file_paths_(file_paths), variable_names_(variable_names)
00135     {}
00136
00137     void loadData()
00138     {
00139         for (const auto& file_path : file_paths_)
00140         {
00141             loadFile(file_path);
00142         }
00143     }
00144
00145     const DataMap& getVariableData(const std::string& variable_name) const
00146     {
00147         auto it = data_.find(variable_name);
00148         if (it != data_.end())
00149             return it->second;
00150
00151         throw std::runtime_error("NetCDFDataLoader: variable '" + variable_name + "' not loaded.");
00152     }
00153
00154     const std::vector<std::size_t>& getVariableShape(const std::string& variable_name) const
00155     {
00156         auto it = shapes_.find(variable_name);
00157         if (it != shapes_.end())
00158             return it->second;
00159
00160         throw std::runtime_error("NetCDFDataLoader: shape for variable '" + variable_name + "' not
available.");
00161     }
00162
00163     const std::string& getVariableType(const std::string& variable_name) const
00164     {
00165         auto it = types_.find(variable_name);
00166         if (it != types_.end())
00167             return it->second;
00168
00169         throw std::runtime_error("NetCDFDataLoader: type for variable '" + variable_name + "' not
available.");
00170     }
00171
00172     const std::vector<TimeType>& getTimeValues(const std::string& file_path) const
00173     {
00174         auto it = time_values_.find(file_path);
00175         if (it != time_values_.end())
00176             return it->second;
00177
00178         throw std::runtime_error("NetCDFDataLoader: no time values found for file: " + file_path);
00179     }
00180
00181     std::vector<DataType> getVariableDataAtIndex(const std::string& var_name, std::size_t index) const
00182     {
00183         auto var_it = data_.find(var_name);
00184         if (var_it == data_.end())
00185             throw std::runtime_error("NetCDFDataLoader: variable not found: " + var_name);
00186
00187         if (index >= var_it->second.size())
00188             throw std::runtime_error("NetCDFDataLoader: index out of range for variable: " +
var_name);
00189
00190         auto map_iter = var_it->second.begin();
00191         std::advance(map_iter, static_cast<long>(index));
00192         return map_iter->second;
00193     }
00194
00202     netCDF::NcVar getNetCDFVariable(const std::string& variable_name) const
00203     {
00204         if (file_paths_.empty())
00205             throw std::runtime_error("NetCDFDataLoader: no files configured.");
00206
00207         netCDF::NcFile dataFile(file_paths_.front(), netCDF::NcFile::read);
00208         netCDF::NcVar var = dataFile.getVar(variable_name);
00209         if (var.isNull())
00210             throw std::runtime_error("NetCDFDataLoader: variable not found in first file: " +
variable_name);
00211
00212         return var;
00213     }
00214
00215     private:
00216     void loadFile(const std::string& file_path)
00217     {
00218         netCDF::NcFile dataFile(file_path, netCDF::NcFile::read);
00219
00220         // -----
00221         // Read "time" variable (if present) and convert to seconds
00222         // -----
00223         std::vector<TimeType> times;

```

```

00224
00225     try
00226     {
00227         netCDF::NcVar time_var = dataFile.getVar("time");
00228         if (!time_var.isNull())
00229         {
00230             std::size_t time_size = time_var.getDim(0).getSize();
00231             times.resize(time_size);
00232             time_var.getVar(times.data());
00233
00234             std::string units;
00235             try
00236             {
00237                 netCDF::NcVarAtt ua = time_var.getAtt("units");
00238                 if (!ua.isNull())
00239                 {
00240                     units.resize(256, '\0');
00241                     ua.getValues(units.data());
00242                     units.erase(std::find(units.begin(), units.end(), '\0'), units.end());
00243                 }
00244             }
00245             catch (...) {}
00246
00247             const std::string ul = detail::to_lower(units);
00248
00249             double mul = 1.0;
00250             if (ul.find("hour") != std::string::npos) mul = 3600.0;
00251             else if (ul.find("day") != std::string::npos) mul = 86400.0;
00252             else if (ul.find("sec") != std::string::npos) mul = 1.0;
00253             else
00254             {
00255                 // Unknown units: keep raw but warn (once).
00256                 if (!units.empty())
00257                     detail::warn_once_per_file(file_path, "unknown time units '" + units + "'.
Time values are not rescaled.");
00258             }
00259
00260             for (auto& t : times)
00261                 t = static_cast<TimeType>(std::llround(static_cast<double>(t) * mul));
00262
00263             time_values_[file_path] = times;
00264             all_time_values_.insert(times.begin(), times.end());
00265         }
00266         else
00267         {
00268             #if PARTICLE_TRACKER_NCDF_STRICT
00269                 throw std::runtime_error("missing variable 'time'");
00270             #else
00271                 detail::warn_once_per_file(file_path, "missing variable 'time'. Time-dependent
variables may be skipped.");
00272             #endif
00273         }
00274     }
00275     catch (const std::exception& e)
00276     {
00277         #if PARTICLE_TRACKER_NCDF_STRICT
00278             throw;
00279         #else
00280             detail::warn_once_per_file(file_path, std::string("error reading 'time': ") + e.what());
00281         #endif
00282     }
00283
00284     // -----
00285     // Load requested variables
00286     // -----
00287     for (const auto& var_name : variable_names_)
00288     {
00289         try
00290         {
00291             netCDF::NcVar var = dataFile.getVar(var_name);
00292             if (var.isNull())
00293                 throw std::runtime_error("variable not found");
00294
00295             std::vector<std::size_t> shape;
00296             for (int i = 0; i < var.getDimCount(); i++)
00297             {
00298                 std::size_t dim_size = var.getDim(i).getSize();
00299                 shape.push_back(dim_size);
00300             }
00301
00302             double scale_factor = 1.0;
00303             double add_offset = 0.0;
00304
00305             try
00306             {
00307                 netCDF::NcVarAtt a = var.getAtt("scale_factor");
00308                 if (!a.isNull()) a.getValues(&scale_factor);

```



```

00309         }
00310         catch (...) { scale_factor = 1.0; }
00311
00312         try
00313         {
00314             netCDF::NcVarAtt a = var.getAtt("add_offset");
00315             if (!a.isNull()) a.getValues(&add_offset);
00316         }
00317         catch (...) { add_offset = 0.0; }
00318
00319         netCDF::NcType varType = var.getType();
00320         const std::string typeName = varType.getName();
00321
00322         if (!canConvert(varType))
00323         {
00324             detail::warn_once_per_file(file_path,
00325                 "variable '" + var_name + "' has netCDF type '" + typeName + "' which cannot
be converted to the template DataType.");
00326             continue;
00327         }
00328
00329         readVariableData(var, var_name, shape, scale_factor, add_offset, file_path);
00330
00331         shapes_[var_name] = shape;
00332         types_[var_name] = typeName;
00333     }
00334     catch (const std::exception& e)
00335     {
00336         detail::warn_once_per_file(file_path,
00337             "error loading variable '" + var_name + "': " + e.what());
00338     }
00339 }
00340
00341 void readVariableData(netCDF::NcVar& var,
00342                     const std::string& var_name,
00343                     const std::vector<std::size_t>& shape,
00344                     double scale_factor,
00345                     double add_offset,
00346                     const std::string& file_path)
00347 {
00348     netCDF::NcType varType = var.getType();
00349     auto typeId = varType.getId();
00350
00351     switch (typeId)
00352     {
00353     case NC_BYTE: readAndTransformData<signed char>(var, var_name, shape, scale_factor,
add_offset, file_path); break;
00354     case NC_CHAR: readAndTransformData<char>(var, var_name, shape, scale_factor, add_offset,
file_path); break;
00355     case NC_SHORT: readAndTransformData<short>(var, var_name, shape, scale_factor,
add_offset, file_path); break;
00356     case NC_INT: readAndTransformData<int>(var, var_name, shape, scale_factor, add_offset,
file_path); break;
00357     case NC_INT64: readAndTransformData<long long>(var, var_name, shape, scale_factor,
add_offset, file_path); break;
00358     case NC_FLOAT: readAndTransformData<float>(var, var_name, shape, scale_factor,
add_offset, file_path); break;
00359     case NC_DOUBLE: readAndTransformData<double>(var, var_name, shape, scale_factor,
add_offset, file_path); break;
00360     default:
00361         throw std::runtime_error("unsupported netCDF variable type id");
00362     }
00363 }
00364
00365 template<typename VarDataType>
00366 void readAndTransformData(netCDF::NcVar& var,
00367                         const std::string& var_name,
00368                         const std::vector<std::size_t>& shape,
00369                         double scale_factor,
00370                         double add_offset,
00371                         const std::string& file_path)
00372 {
00373     // -----
00374     // Determine missing value (attribute or fallback)
00375     // -----
00376     VarDataType missing_value = VarDataType{};
00377     bool has_missing_value = false;
00378
00379     try
00380     {
00381         netCDF::NcVarAtt a = var.getAtt("_FillValue");
00382         if (!a.isNull())
00383         {
00384             a.getValues(&missing_value);
00385             has_missing_value = true;
00386         }
00387     }

```

```

00388         else
00389         {
00390             a = var.getAtt("missing_value");
00391             if (!a.isNull())
00392             {
00393                 a.getValues(&missing_value);
00394                 has_missing_value = true;
00395             }
00396         }
00397     }
00398     catch (...) {}
00399
00400     // Fallback sentinel for "data variables" (skip obvious coordinate vars)
00401     if (!has_missing_value && var_name != "lon" && var_name != "lat" && var_name != "depth")
00402     {
00403         missing_value = static_cast<VarDataType>(-30000);
00404         has_missing_value = true;
00405     }
00406
00407     // -----
00408     // Identify time dimension (if any)
00409     // -----
00410     int time_dim_index = -1;
00411     for (int i = 0; i < var.getDimCount(); i++)
00412     {
00413         if (var.getDim(i).getName() == "time")
00414         {
00415             time_dim_index = i;
00416             break;
00417         }
00418     }
00419
00420     // -----
00421     // Read time-dependent variable slice-by-slice
00422     // -----
00423     if (time_dim_index != -1)
00424     {
00425         auto it = time_values_.find(file_path);
00426         if (it == time_values_.end())
00427         {
00428             #if PARTICLE_TRACKER_NCDF_STRICT
00429                 throw std::runtime_error("variable '" + var_name + "' has a time dimension but no
'time' values were loaded");
00430             #else
00431                 detail::warn_once_per_file(file_path,
00432                     "variable '" + var_name + "' has a time dimension but no 'time' values were
00433                     loaded. Skipping.");
00434                 return;
00435             #endif
00436
00437             const auto& times = it->second;
00438             const std::size_t time_size = shape[static_cast<std::size_t>(time_dim_index)];
00439
00440             if (times.size() < time_size)
00441             {
00442                 #if PARTICLE_TRACKER_NCDF_STRICT
00443                     throw std::runtime_error("time vector shorter than time dimension for variable '" +
var_name + "'");
00444                 #else
00445                     detail::warn_once_per_file(file_path,
00446                         "time vector shorter than time dimension for variable '" + var_name + "'
00447                         Skipping.");
00448                     return;
00449                 #endif
00450
00451                 std::vector<std::size_t> start(static_cast<std::size_t>(var.getDimCount()), 0);
00452                 std::vector<std::size_t> count = shape;
00453                 count[static_cast<std::size_t>(time_dim_index)] = 1;
00454
00455                 for (std::size_t t = 0; t < time_size; t++)
00456                 {
00457                     start[static_cast<std::size_t>(time_dim_index)] = t;
00458                     TimeType time_value = times[t];
00459
00460                     std::size_t data_size = 1;
00461                     for (std::size_t dim_size : count)
00462                         data_size *= dim_size;
00463
00464                     std::vector<VarDataType> raw_data(data_size);
00465                     var.getVar(start, count, raw_data.data());
00466
00467                     std::vector<DataType> transformed_data(data_size);
00468
00469                     for (std::size_t i = 0; i < data_size; i++)
00470                     {

```

```

00471         if (has_missing_value && raw_data[i] == missing_value)
00472         {
00473             transformed_data[i] = std::numeric_limits<DataType>::quiet_NaN();
00474         }
00475         else
00476         {
00477             transformed_data[i] =
00478                 static_cast<DataType>(raw_data[i]) * static_cast<DataType>(scale_factor) +
00479                 static_cast<DataType>(add_offset);
00480         }
00481     }
00482
00483     data_[var_name][time_value] = std::move(transformed_data);
00484 }
00485 }
00486 else
00487 {
00488     // -----
00489     // Read variable without time dimension
00490     // -----
00491     std::size_t total_size = 1;
00492     for (std::size_t dim_size : shape)
00493         total_size *= dim_size;
00494
00495     std::vector<VarDataType> raw_data(total_size);
00496     var.getVar(raw_data.data());
00497
00498     std::vector<DataType> transformed_data(total_size);
00499
00500     for (std::size_t i = 0; i < total_size; i++)
00501     {
00502         if (has_missing_value && raw_data[i] == missing_value)
00503         {
00504             transformed_data[i] = std::numeric_limits<DataType>::quiet_NaN();
00505         }
00506         else
00507         {
00508             transformed_data[i] =
00509                 static_cast<DataType>(raw_data[i]) * static_cast<DataType>(scale_factor) +
00510                 static_cast<DataType>(add_offset);
00511         }
00512     }
00513
00514     TimeType time_value = TimeType{};
00515     data_[var_name][time_value] = std::move(transformed_data);
00516 }
00517 }
00518
00519 bool canConvert(const netCDF::NcType& varType) const
00520 {
00521     return std::is_arithmetic<DataType>::value && (
00522         varType == netCDF::ncByte ||
00523         varType == netCDF::ncChar ||
00524         varType == netCDF::ncShort ||
00525         varType == netCDF::ncInt ||
00526         varType == netCDF::ncInt64 ||
00527         varType == netCDF::ncFloat ||
00528         varType == netCDF::ncDouble
00529     );
00530 }
00531
00532 std::vector<std::string> file_paths_;
00533 std::vector<std::string> variable_names_;
00534
00535 std::map<std::string, std::map<TimeType, std::vector<DataType>>> data_;
00536 std::map<std::string, std::vector<std::size_t>> shapes_;
00537 std::map<std::string, std::string> types_;
00538 std::map<std::string, std::vector<TimeType>> time_values_;
00539 std::set<TimeType> all_time_values_;
00540 };
00541
00542 //-----
00543 // NetCDFProcessor
00544 //-----
00545
00546 template<typename DataType, typename TimeType = double>
00547 class NetCDFProcessor
00548 {
00549 public:
00550     using DataMap = std::map<TimeType, std::vector<DataType>>;
00551     using VariableProcessingFunc =
00552         std::function<void(const std::string&, TimeType, const std::vector<DataType>&)>;
00553
00554     NetCDFProcessor(const std::string& file_path,
00555                     const std::vector<std::string>& variable_names)
00556         : file_path_(file_path),
00557           variable_names_(variable_names),

```

```

00561     data_loader_({file_path}, variable_names)
00562     {
00563     try
00564     {
00565         data_loader_.loadData();
00566     }
00567     catch (const std::exception& e)
00568     {
00569         std::cerr << "NetCDFProcessor: error loading file '" << file_path_ << "': " << e.what() <<
'\n';
00570         throw;
00571     }
00572     }
00573
00574 void processVariables(const VariableProcessingFunc& processFunc)
00575 {
00576     try
00577     {
00578         for (const auto& var_name : variable_names_)
00579         {
00580             const auto& var_data = data_loader_.getVariableData(var_name);
00581             for (const auto& kv : var_data)
00582             {
00583                 processFunc(var_name, kv.first, kv.second);
00584             }
00585         }
00586     }
00587     catch (const std::exception& e)
00588     {
00589         std::cerr << "NetCDFProcessor: error processing file '" << file_path_ << "': " << e.what() <<
'\n';
00590         throw;
00591     }
00592     }
00593
00594 netCDF::NcVar getNetCDFVariable(const std::string& variable_name) const
00595 {
00596     return data_loader_.getNetCDFVariable(variable_name);
00597 }
00598
00599 const std::vector<std::size_t>& getVariableShape(const std::string& variable_name) const
00600 {
00601     return data_loader_.getVariableShape(variable_name);
00602 }
00603
00604 const std::string& getVariableType(const std::string& variable_name) const
00605 {
00606     return data_loader_.getVariableType(variable_name);
00607 }
00608
00609 const std::vector<TimeType>& getTimeValues() const
00610 {
00611     return data_loader_.getTimeValues(file_path_);
00612 }
00613
00614 const typename NetCDFDataLoader<DataType, TimeType>::DataMap&
00615 getVariableData(const std::string& variable_name) const
00616 {
00617     return data_loader_.getVariableData(variable_name);
00618 }
00619
00620 /*std::vector<DataType> getVariableDataAtTime(const std::string& var_name, TimeType time_value)
00621 {
00622     std::size_t time_index = getTimeIndex(time_value);
00623     return data_loader_.getVariableDataAtIndex(var_name, time_index);
00624 }*/
00625
00626 std::vector<DataType> getVariableDataAtTime(const std::string& var_name, TimeType time_value)
00627 {
00628     const auto& var_data = data_loader_.getVariableData(var_name);
00629
00630     const TimeType key = static_cast<TimeType>(std::llround(static_cast<double>(time_value)));
00631
00632     auto it = var_data.find(key);
00633     if (it == var_data.end())
00634         throw std::runtime_error("NetCDFProcessor: requested time slice not loaded for variable '"
+
00635                                     var_name + "' at time=" + std::to_string(static_cast<long
long>(key)));
00636
00637     return it->second; // returns a copy (your current API returns by value anyway)
00638 }
00639
00640 const std::vector<DataType>& getVariableDataAtTimeRef(const std::string& var_name, TimeType
time_value) const
00641 {
00642     const auto& var_data = data_loader_.getVariableData(var_name);

```

```

00643         const TimeType key = static_cast<TimeType>(std::llround(static_cast<double>(time_value)));
00644
00645         auto it = var_data.find(key);
00646         if (it == var_data.end())
00647             throw std::runtime_error("NetCDFProcessor: requested time slice not loaded...");
00648
00649         return it->second;
00650     }
00651
00652     private:
00653     std::string file_path_;
00654     std::vector<std::string> variable_names_;
00655     NetCDFDataLoader<DataType, TimeType> data_loader_;
00656
00657     std::size_t getTimeIndex(TimeType time_value)
00658     {
00659         const auto& time_values = data_loader_.getTimeValues(file_path_);
00660         const long long q = std::llround(static_cast<double>(time_value));
00661
00662         auto it = std::find_if(time_values.begin(), time_values.end(),
00663             [q](TimeType t){ return std::llround(static_cast<double>(t)) == q; });
00664
00665         if (it != time_values.end())
00666             return static_cast<std::size_t>(std::distance(time_values.begin(), it));
00667
00668         throw std::runtime_error("NetCDFProcessor: requested time value not found in loaded data.");
00669     }
00670 };
00671
00672 //-----
00673 // ncdf_utils helper functions
00674 //-----
00675
00676 namespace ncdf_utils
00677 {
00681 inline std::vector<std::string> checkFileHeaders(const std::string& fileName,
00682     const std::vector<std::string>& requiredVariables,
00683     int expectedTimeCount)
00684 {
00685     std::vector<std::string> errors;
00686
00687     try
00688     {
00689         netCDF::NcFile dataFile(fileName, netCDF::NcFile::read);
00690
00691         netCDF::NcVar timeVar = dataFile.getVar("time");
00692         if (timeVar.isNull())
00693         {
00694             errors.push_back("Missing variable 'time'");
00695         }
00696         else
00697         {
00698             if (timeVar.getDimCount() < 1)
00699             {
00700                 errors.push_back("Variable 'time' has no dimensions.");
00701             }
00702             else
00703             {
00704                 std::size_t timeSize = timeVar.getDim(0).getSize();
00705                 if (timeSize != static_cast<std::size_t>(expectedTimeCount))
00706                 {
00707                     errors.push_back("Variable 'time' dimension size is " +
00708                         std::to_string(timeSize) + " but expected " +
00709                         std::to_string(expectedTimeCount));
00710                 }
00711             }
00712         }
00713
00714         for (const auto& varName : requiredVariables)
00715         {
00716             if (varName == "time") continue;
00717
00718             netCDF::NcVar var = dataFile.getVar(varName);
00719             if (var.isNull())
00720             {
00721                 errors.push_back("Missing variable '" + varName + "'");
00722                 continue;
00723             }
00724
00725             int dimCount = var.getDimCount();
00726             if (dimCount == 0)
00727             {
00728                 errors.push_back("Variable '" + varName + "' has no dimensions.");
00729                 continue;
00730             }
00731
00732             for (int i = 0; i < dimCount; i++)

```

```

00733         {
00734             std::size_t dSize = var.getDim(i).getSize();
00735             if (dSize == 0)
00736             {
00737                 errors.push_back("Variable '" + varName + "' has dimension " +
00738                                 std::to_string(i) + " equal to 0.");
00739             }
00740         }
00741
00742         if (dimCount > 1)
00743         {
00744             std::size_t timeDim = var.getDim(0).getSize();
00745             if (timeDim != static_cast<std::size_t>(expectedTimeCount))
00746             {
00747                 errors.push_back("Variable '" + varName + "' has time dimension " +
00748                                 std::to_string(timeDim) + " but expected " +
00749                                 std::to_string(expectedTimeCount));
00750             }
00751         }
00752     }
00753 }
00754 catch (const std::exception& e)
00755 {
00756     errors.push_back("Error processing file '" + fileName + "': " + std::string(e.what()));
00757 }
00758
00759 return errors;
00760 }
00761 } // namespace ncdf_utils
00762
00763 } // namespace particle_tracker

```

6.8 fixed_point_core.hpp

```

00001 // fixed_point_core.hpp
00002 // Core math + quantization + packed structs + encode/decode.
00003 #pragma once
00004 #include <cstdint>
00005 #include <cmath>
00006 #include <array>
00007 #include <limits>
00008 #include <numbers> // std::numbers::pi_v
00009
00010 // ----- constants -----
00011 // Keep a single source of truth for scaling across PKD1/PKD2 and seed loaders.
00012 inline constexpr double fp_pi = std::numbers::pi_v<double>;
00013 inline constexpr uint32_t fp_ticks_per_rad = 1'000'000u; // ticks per radian
00014 inline constexpr double dbl_ticks_per_rad = 1'000'000.; // ticks per radian (double)
00015 inline constexpr uint64_t fp_ticks_per_sec = 1u; // seconds per tick (PKD1)
00016
00017 // ----- angle utils -----
00018 inline double wrap_lon(double r)
00019 {
00020     r = std::fmod(r, 2.*fp_pi);
00021     if (r <= -fp_pi) r += 2.*fp_pi;
00022     else if (r > fp_pi) r -= 2.*fp_pi;
00023     return r;
00024 }
00025
00026 // ----- quantization -----
00027 // Position: ticks = radians * fp_ticks_per_rad
00028 template <typename Real>
00029 inline int32_t quant_coord(Real rad)
00030 {
00031     long double v = std::llround(static_cast<long double>(rad) * static_cast<long
double>(fp_ticks_per_rad));
00032     if (v > std::numeric_limits<int32_t>::max()) v = std::numeric_limits<int32_t>::max();
00033     if (v < std::numeric_limits<int32_t>::min()) v = std::numeric_limits<int32_t>::min();
00034     return static_cast<int32_t>(v);
00035 }
00036
00037 template <typename Real>
00038 inline Real dequant_coord(int32_t q)
00039 {
00040     return static_cast<Real>(q) / static_cast<Real>(fp_ticks_per_rad);
00041 }
00042
00043 // Time: 1 tick = 1 second by default (PKD1 helpers)
00044 template <typename Real>
00045 inline int64_t quant_time(Real seconds)
00046 {
00047     return static_cast<int64_t>(std::llround(static_cast<long double>(seconds)));
00048 }

```

```

00049
00050 template <typename Real>
00051 inline Real dequant_time(int64_t ticks)
00052 {
00053     return static_cast<Real>(ticks);
00054 }
00055
00056 // ----- packed record (PKD1) -----
00057 #pragma pack(push,1)
00058 struct PackedParticle
00059 {
00060     int32_t lon0_q; // ticks (micro-rad)
00061     int32_t lat0_q; // ticks
00062     int32_t lon1_q; // ticks
00063     int32_t lat1_q; // ticks
00064     int64_t t0_s; // seconds since reference
00065     int64_t t1_s; // seconds since reference
00066     uint8_t flags;
00067 };
00068 #pragma pack(pop)
00069
00070 // ----- PKD1 file header (struct only) -----
00071 struct FileHeaderPKD1
00072 {
00073     std::array<char,8> magic = {'P','K','D','1',0,0,0,0};
00074     uint32_t version = 1;
00075     uint32_t pos_scale = fp_ticks_per_rad; // ticks per rad
00076     uint64_t time_tick = fp_ticks_per_sec; // seconds per tick
00077     uint64_t reserved0 = 0;
00078     uint64_t reserved1 = 0;
00079 };
00080
00081 // ----- encode/decode one record -----
00082 template<typename PosT,typename TimeT>
00083 inline PackedParticle encode_record(PosT lon0, PosT lat0, TimeT t0,
00084                                     PosT lon1, PosT lat1, TimeT t1,
00085                                     uint8_t flags = 0)
00086 {
00087     PackedParticle r{};
00088     r.lon0_q = quant_coord(wrap_lon(lon0));
00089     r.lat0_q = quant_coord(lat0);
00090     r.lon1_q = quant_coord(wrap_lon(lon1));
00091     r.lat1_q = quant_coord(lat1);
00092     r.t0_s = quant_time(t0);
00093     r.t1_s = quant_time(t1);
00094     r.flags = flags;
00095     return r;
00096 }
00097
00098 template<typename PosT,typename TimeT>
00099 inline void decode_record(const PackedParticle& r,
00100                          PosT& lon0, PosT& lat0, TimeT& t0,
00101                          PosT& lon1, PosT& lat1, TimeT& t1,
00102                          uint8_t& flags)
00103 {
00104     lon0 = dequant_coord<PosT>(r.lon0_q);
00105     lat0 = dequant_coord<PosT>(r.lat0_q);
00106     lon1 = dequant_coord<PosT>(r.lon1_q);
00107     lat1 = dequant_coord<PosT>(r.lat1_q);
00108     t0 = dequant_time<TimeT>(r.t0_s);
00109     t1 = dequant_time<TimeT>(r.t1_s);
00110     flags = r.flags;
00111 }

```

6.9 fixed_point_io_codec.hpp

```

00001 // fixed_point_io_codec.hpp
00002 // PKD1 "record-stream" compressed helpers (zstd or gzip/zlib).
00003 // Requires pk_config.hpp to be included first (or compile flags defined).
00004
00005 // NOTE ON SIZE LIMITS AND FORMAT CONSTRAINTS
00006 //
00007 // * gzip/zlib path:
00008 //   - zlib APIs (deflate, crc32) use `uInt` for input sizes (typically 32-bit).
00009 //   - A single gzip member therefore cannot safely process >4 GiB of
00010 //     uncompressed data in one call.
00011 //   - This implementation hard-fails on overflow instead of silently
00012 //     truncating and corrupting the stream.
00013 //   - Supporting >4 GiB would require explicit chunked deflate + rolling CRC.
00014 //
00015 // * zstd path:
00016 //   - Zstd compression/decompression APIs use `size_t` and are not subject
00017 //     to the 4 GiB zlib limitation.

```

```

00018 // - Zstd skippable frame headers store payload size as uint32_t by format;
00019 // this is safe here because FileHeaderPKD1 is small.
00020 // - Decompression accumulates the full output in memory; no hard size
00021 // limit is enforced for corrupted or untrusted inputs.
00022
00023 #pragma once
00024 #include <vector>
00025 #include <string>
00026 #include <fstream>
00027 #include <stdexcept>
00028 #include <cstdint>
00029 #include <algorithm>
00030 #include <limits>
00031 #include <cstring>
00032
00033 #include "pk_config.hpp"
00034 #include "fixed_point_core.hpp"
00035 #include "fixed_point_pkd1_io.hpp" // for write_all/read_all + FileHeaderPKD1
00036
00037 #ifdef PK_IO_USE_ZSTD
00038     #include <zstd.h> // link with -lzstd
00039 #endif
00040 #ifdef PK_IO_USE_ZLIB
00041     #include <zlib.h> // link with -lz
00042 #endif
00043
00044 // ----- ZSTD implementation -----
00045 #ifdef PK_IO_USE_ZSTD
00046
00047 inline void write_records_zstd(const std::string& path,
00048                               const std::vector<PackedParticle>& rec,
00049                               std::uint64_t released_count,
00050                               int level = 3,
00051                               bool append = false)
00052 {
00053     if (!append)
00054     {
00055         std::ofstream os(path, std::ios::binary | std::ios::out | std::ios::trunc);
00056         if (!os) throw std::runtime_error("write_records_zstd: cannot open " + path);
00057
00058         FileHeaderPKD1 h{};
00059         h.reserved0 = released_count;
00060         h.reserved1 = static_cast<uint64_t>(rec.size());
00061
00062         // skippable frame magic (0x184D2A50..57)
00063         const uint32_t magic = 0x184D2A50u;
00064         const uint32_t size = static_cast<uint32_t>(sizeof(FileHeaderPKD1));
00065         os.write(reinterpret_cast<const char*>(&magic), 4);
00066         os.write(reinterpret_cast<const char*>(&size), 4);
00067         write_header(os, h);
00068         os.flush();
00069     }
00070     else
00071     {
00072         // bump reserved1 inside skippable frame at file start
00073         std::fstream hs(path, std::ios::binary | std::ios::in | std::ios::out);
00074         if (!hs) throw std::runtime_error("write_records_zstd: cannot reopen for header " + path);
00075
00076         uint32_t magic = 0, size = 0;
00077         hs.read(reinterpret_cast<char*>(&magic), 4);
00078         hs.read(reinterpret_cast<char*>(&size), 4);
00079         if ((magic & 0xFFFFFFFFu) != 0x184D2A50u || size < sizeof(FileHeaderPKD1))
00080             throw std::runtime_error("write_records_zstd: no skippable header found");
00081
00082         FileHeaderPKD1 h = read_header(hs);
00083         h.reserved1 += static_cast<uint64_t>(rec.size());
00084
00085         hs.seekp(8, std::ios::beg); // after magic+size
00086         write_header(hs, h);
00087         hs.flush();
00088     }
00089
00090     if (rec.empty()) return;
00091
00092     std::ofstream os(path, std::ios::binary | std::ios::out | std::ios::app);
00093     if (!os) throw std::runtime_error("write_records_zstd: cannot open for append " + path);
00094
00095     const size_t src_bytes = rec.size() * sizeof(PackedParticle);
00096     const size_t bound = ZSTD_compressBound(src_bytes);
00097     std::vector<char> cbuf(bound);
00098
00099     const size_t rc = ZSTD_compress(cbuf.data(), bound, rec.data(), src_bytes, level);
00100     if (ZSTD_isError(rc))
00101         throw std::runtime_error("write_records_zstd: " + std::string(ZSTD_getErrorName(rc)));
00102
00103     write_all(os, cbuf.data(), rc);
00104 }

```



```

00105
00106 inline std::vector<PackedParticle> read_records_zstd(const std::string& path)
00107 {
00108     std::ifstream is(path, std::ios::binary);
00109     if (!is) throw std::runtime_error("read_records_zstd: cannot open " + path);
00110
00111     std::streampos start = is.tellg();
00112     uint32_t magic = 0, size = 0;
00113     is.read(reinterpret_cast<char*>(&magic), 4);
00114     is.read(reinterpret_cast<char*>(&size), 4);
00115
00116     if (is && ((magic & 0xFFFFFFFFu) == 0x184D2A50u))
00117     {
00118         if (size < sizeof(FileHeaderPKD1))
00119             throw std::runtime_error("read_records_zstd: skippable too small");
00120         FileHeaderPKD1 h{};
00121         read_all(is, &h);
00122         if (h.magic[0] != 'P' || h.magic[1] != 'K' || h.magic[2] != 'D')
00123             throw std::runtime_error("read_records_zstd: bad PKD header");
00124         if (size > sizeof(FileHeaderPKD1))
00125             is.seekg(static_cast<std::streamoff>(size - sizeof(FileHeaderPKD1)), std::ios::cur);
00126     }
00127     else
00128     {
00129         // backward compat: old files had raw PKD1 header
00130         is.clear();
00131         is.seekg(start);
00132         (void)read_header(is);
00133     }
00134
00135     ZSTD_DCtx* dctx = ZSTD_createDCtx();
00136     if (!dctx) throw std::runtime_error("read_records_zstd: createDCtx failed");
00137
00138     const size_t in_chunk = ZSTD_DStreamInSize();
00139     const size_t out_chunk = ZSTD_DStreamOutSize();
00140     std::vector<char> inbuf(in_chunk), outbuf(out_chunk), raw;
00141
00142     ZSTD_inBuffer in { inbuf.data(), 0, 0 };
00143     ZSTD_outBuffer out { outbuf.data(), outbuf.size(), 0 };
00144
00145     while (true)
00146     {
00147         if (in.pos == in.size)
00148         {
00149             is.read(inbuf.data(), static_cast<std::streamsize>(inbuf.size()));
00150             in.size = static_cast<size_t>(is.gcount());
00151             in.pos = 0;
00152             if (in.size == 0) break;
00153         }
00154
00155         out.pos = 0;
00156         size_t ret = ZSTD_decompressStream(dctx, &out, &in);
00157         if (ZSTD_isError(ret))
00158         {
00159             ZSTD_freeDCtx(dctx);
00160             throw std::runtime_error("read_records_zstd: " + std::string(ZSTD_getErrorName(ret)));
00161         }
00162         if (out.pos) raw.insert(raw.end(), outbuf.data(), outbuf.data() + out.pos);
00163     }
00164
00165     ZSTD_freeDCtx(dctx);
00166
00167     if (raw.size() % sizeof(PackedParticle) != 0)
00168         throw std::runtime_error("read_records_zstd: size mismatch");
00169
00170     std::vector<PackedParticle> rec(raw.size() / sizeof(PackedParticle));
00171     std::memcpy(rec.data(), raw.data(), raw.size());
00172     return rec;
00173 }
00174
00175 #endif // PK_IO_USE_ZSTD
00176
00177 // ----- ZLIB/GZIP implementation -----
00178 #ifdef PK_IO_USE_ZLIB
00179 namespace gzdetail
00180 {
00181     inline uint16_t get_le16(std::istream& is)
00182     {
00183         unsigned char b[2]{};
00184         if (!is.read(reinterpret_cast<char*>(b), 2))
00185             throw std::runtime_error("get_le16: read failed");
00186         return static_cast<uint16_t>(b[0]) | (static_cast<uint16_t>(b[1]) << 8);
00187     }
00188
00189     inline uint32_t get_le32(std::istream& is)
00190     {

```

```

00192     unsigned char b[4]{};
00193     if (!is.read(reinterpret_cast<char*>(b), 4))
00194         throw std::runtime_error("get_le32: read failed");
00195     return static_cast<uint32_t>(b[0])
00196         | (static_cast<uint32_t>(b[1]) << 8)
00197         | (static_cast<uint32_t>(b[2]) << 16)
00198         | (static_cast<uint32_t>(b[3]) << 24);
00199 }
00200
00201 inline void put_le16(std::ostream& os, uint16_t v)
00202 {
00203     unsigned char b[2] = {
00204         static_cast<unsigned char>(v & 0xFFu),
00205         static_cast<unsigned char>((v >> 8) & 0xFFu)
00206     };
00207     os.write(reinterpret_cast<const char*>(b), 2);
00208 }
00209
00210 inline void put_le32(std::ostream& os, uint32_t v)
00211 {
00212     unsigned char b[4] = {
00213         static_cast<unsigned char>(v & 0xFFu),
00214         static_cast<unsigned char>((v >> 8) & 0xFFu),
00215         static_cast<unsigned char>((v >> 16) & 0xFFu),
00216         static_cast<unsigned char>((v >> 24) & 0xFFu)
00217     };
00218     os.write(reinterpret_cast<const char*>(b), 4);
00219 }
00220
00221 // gzip header with FEXTRA containing FileHeaderPKD1
00222 inline void write_gzip_header(std::ostream& os, const FileHeaderPKD1& h, bool with_extra)
00223 {
00224     const uint8_t ID1=0x1f, ID2=0x8b, CM=8;
00225     uint8_t FLG = with_extra ? 0x04 : 0x00; // FEXTRA
00226
00227     os.put(ID1); os.put(ID2); os.put(CM); os.put(FLG);
00228     put_le32(os, 0); // MTIME
00229     os.put(0); // XFL
00230     unsigned char os_byte=0xFF;
00231     os.write(reinterpret_cast<const char*>(&os_byte), 1);
00232
00233     if (with_extra)
00234     {
00235         const uint16_t sublen = static_cast<uint16_t>(sizeof(FileHeaderPKD1));
00236         const uint16_t xlen = static_cast<uint16_t>(2 + 2 + sublen);
00237         put_le16(os, xlen);
00238
00239         os.put('P'); os.put('K'); // subfield ID
00240         put_le16(os, sublen);
00241         write_header(os, h);
00242     }
00243 }
00244
00245 inline void bump_gzip_extra_reserved1(const std::string& path, uint64_t add)
00246 {
00247     std::fstream hs(path, std::ios::binary | std::ios::in | std::ios::out);
00248     if (!hs) throw std::runtime_error("write_records_gzip: cannot reopen " + path);
00249
00250     unsigned char hdr[10];
00251     hs.read(reinterpret_cast<char*>(hdr), 10);
00252     if (hs.gcount() != 10 || hdr[0] != 0x1f || hdr[1] != 0x8b || hdr[2] != 8)
00253         throw std::runtime_error("write_records_gzip: not a gzip file");
00254
00255     const uint8_t flg = hdr[3];
00256     if ((flg & 0x04) == 0) return; // no FEXTRA
00257
00258     const uint16_t xlen = gzdetail::get_le16(hs);
00259     if (xlen < 4)
00260         throw std::runtime_error("write_records_gzip: corrupt FEXTRA (xlen < 4)");
00261
00262     char si1 = 0, si2 = 0;
00263     hs.get(si1); hs.get(si2);
00264     const uint16_t len = gzdetail::get_le16(hs);
00265
00266     if (si1 != 'P' || si2 != 'K' || len < sizeof(FileHeaderPKD1))
00267         throw std::runtime_error("write_records_gzip: PKD extra not found");
00268
00269     FileHeaderPKD1 h = read_header(hs);
00270     h.reserved1 += add;
00271
00272     // fixed offset because layout is exactly:
00273     // 10 (gzip) + 2 (XLEN) + 2 (SI) + 2 (LEN)
00274     hs.seekp(10 + 2 + 2 + 2, std::ios::beg);
00275     write_header(hs, h);
00276     hs.flush();
00277 }
00278 }

```

```

00279
00280 static inline void deflate_raw_to_stream(std::ostream& os,
00281                                         const void* src,
00282                                         std::size_t src_len,
00283                                         int level)
00284 {
00285     // zlib's z_stream::avail_in is uInt (usually 32-bit). Do NOT truncate silently.
00286     if (src_len > static_cast<std::size_t>(std::numeric_limits<uInt>::max()))
00287         throw std::runtime_error("deflate_raw_to_stream: input too large for zlib ( > 4GiB )");
00288
00289     z_stream zs{};
00290     if (deflateInit2(&zs, level, Z_DEFLATED, -15, 9, Z_DEFAULT_STRATEGY) != Z_OK)
00291         throw std::runtime_error("deflateInit2 raw failed");
00292
00293     zs.next_in = reinterpret_cast<Bytef*>(const_cast<void*>(src));
00294     zs.avail_in = static_cast<uInt>(src_len);
00295
00296     unsigned char outbuf[64 * 1024];
00297     int rc;
00298     do
00299     {
00300         zs.next_out = outbuf;
00301         zs.avail_out = static_cast<uInt>(sizeof(outbuf));
00302
00303         rc = deflate(&zs, zs.avail_in ? Z_NO_FLUSH : Z_FINISH);
00304         if (rc != Z_OK && rc != Z_STREAM_END)
00305         {
00306             deflateEnd(&zs);
00307             throw std::runtime_error("deflate error");
00308         }
00309
00310         const std::size_t produced = sizeof(outbuf) - zs.avail_out;
00311         if (produced)
00312             os.write(reinterpret_cast<const char*>(outbuf),
00313                     static_cast<std::streamsize>(produced));
00314     }
00315     while (rc != Z_STREAM_END);
00316
00317     deflateEnd(&zs);
00318 }
00319
00320 inline void write_records_gzip(const std::string& path,
00321                               const std::vector<PackedParticle*>& rec,
00322                               std::uint64_t released_count,
00323                               int level = 6,
00324                               bool append = false)
00325 {
00326     const std::size_t bytes = rec.size() * sizeof(PackedParticle);
00327
00328     // zlib crc32() length parameter is uInt -> avoid silent truncation
00329     if (bytes > static_cast<std::size_t>(std::numeric_limits<uInt>::max()))
00330         throw std::runtime_error("write_records_gzip: buffer too large for zlib ( > 4GiB )");
00331
00332     if (!append)
00333     {
00334         std::ofstream os(path, std::ios::binary | std::ios::out | std::ios::trunc);
00335         if (!os)
00336             throw std::runtime_error("write_records_gzip: cannot open " + path);
00337
00338         FileHeaderPKD1 h{};
00339         h.reserved0 = released_count;
00340         h.reserved1 = static_cast<uint64_t>(rec.size());
00341
00342         gzdetail::write_gzip_header(os, h, /*with_extra=*/true);
00343
00344         // CRC32 over uncompressed data
00345         uLong crc = crc32(0L, Z_NULL, 0);
00346         crc = crc32(crc,
00347                   reinterpret_cast<const Bytef*>(rec.data()),
00348                   static_cast<uInt>(bytes));
00349
00350         // gzip trailer ISIZE is modulo 2^32
00351         const uint32_t isize32 = static_cast<uint32_t>(static_cast<uint64_t>(bytes));
00352
00353         // raw deflate payload
00354         deflate_raw_to_stream(os, rec.data(), bytes, level);
00355         if (!os.good())
00356             throw std::runtime_error("write_records_gzip: ostream error during deflate");
00357
00358         // gzip trailer: CRC32 + ISIZE
00359         gzdetail::put_le32(os, static_cast<uint32_t>(crc));
00360         gzdetail::put_le32(os, isize32);
00361         return;
00362     }
00363
00364     // append path
00365     if (rec.empty())

```

```

00366         return;
00367
00368         // bump record count in first gzip member
00369         gzdetail::bump_gzip_extra_reserved1(path, static_cast<uint64_t>(rec.size()));
00370
00371         std::ofstream os(path, std::ios::binary | std::ios::out | std::ios::app);
00372         if (!os)
00373             throw std::runtime_error("write_records_gzip: cannot open for append " + path);
00374
00375         // appended member has no extra
00376         gzdetail::write_gzip_header(os, FileHeaderPKD1{}, /*with_extra=*/false);
00377
00378         uLong crc = crc32(0L, Z_NULL, 0);
00379         crc = crc32(crc,
00380                     reinterpret_cast<const Bytef*>(rec.data()),
00381                     static_cast<uInt>(bytes));
00382
00383         const uint32_t isize32 = static_cast<uint32_t>(static_cast<uint64_t>(bytes));
00384
00385         deflate_raw_to_stream(os, rec.data(), bytes, level);
00386         if (!os.good())
00387             throw std::runtime_error("write_records_gzip: ostream error during deflate");
00388
00389         gzdetail::put_le32(os, static_cast<uint32_t>(crc));
00390         gzdetail::put_le32(os, isize32);
00391     }
00392
00393     inline std::vector<PackedParticle> read_records_gzip(const std::string& path)
00394     {
00395         std::ifstream is(path, std::ios::binary);
00396         if (!is) throw std::runtime_error("read_records_gzip: cannot open " + path);
00397
00398         // Rewind-safe peek for PKD extra (optional)
00399         const std::streampos start = is.tellg();
00400         unsigned char hdr[10];
00401         if (!is.read(reinterpret_cast<char*>(hdr), 10))
00402             throw std::runtime_error("read_records_gzip: truncated header");
00403
00404         if (hdr[0]==0x1f && hdr[1]==0x8b && hdr[2]==8)
00405         {
00406             const uint8_t flg = hdr[3];
00407             if (flg & 0x04) // FEXTRA present
00408             {
00409                 const uint16_t xlen = gzdetail::get_le16(is);
00410                 if (xlen < 4)
00411                     throw std::runtime_error("read_records_gzip: corrupt FEXTRA (xlen < 4)");
00412
00413                 // We only inspect the *first* subfield. If it's not ours, we skip all extras.
00414                 char si1 = 0, si2 = 0;
00415                 is.get(si1); is.get(si2);
00416                 const uint16_t len = gzdetail::get_le16(is);
00417
00418                 if (si1=='P' && si2=='K' && len >= sizeof(FileHeaderPKD1))
00419                 {
00420                     // Read our header payload
00421                     FileHeaderPKD1 h{};
00422                     read_all(is, &h);
00423
00424                     if (h.magic[0] != 'P' || h.magic[1] != 'K' || h.magic[2] != 'D')
00425                         throw std::runtime_error("read_records_gzip: bad PKD header");
00426
00427                     // Skip any extra bytes inside this subfield
00428                     const std::streamoff extra_in_subfield =
00429                         static_cast<std::streamoff>(len - sizeof(FileHeaderPKD1));
00430                     if (extra_in_subfield > 0)
00431                         is.seekg(extra_in_subfield, std::ios::cur);
00432
00433                     // Skip remaining extra subfields if present
00434                     const uint16_t consumed = static_cast<uint16_t>(2 + 2 + len); // SI1+SI2+LEN+payload
00435                     if (xlen > consumed)
00436                         is.seekg(static_cast<std::streamoff>(xlen - consumed), std::ios::cur);
00437                 }
00438                 else
00439                 {
00440                     // Not our subfield -> skip the rest of FEXTRA.
00441                     // We already consumed 4 bytes (SI1,SI2,LEN). Need to skip remaining (xlen - 4).
00442                     is.seekg(static_cast<std::streamoff>(xlen - 4), std::ios::cur);
00443                 }
00444             }
00445         }
00446
00447         // rewind so zlib sees the full gzip stream from the beginning
00448         is.clear();
00449         is.seekg(start);
00450
00451         z_stream zs{};
00452         if (inflateInit2(&zs, 15 + 32) != Z_OK)

```

```

00453         throw std::runtime_error("inflateInit2 failed");
00454
00455     std::vector<char> inbuf(64*1024), outbuf(64*1024), raw;
00456     zs.next_in = Z_NULL;
00457     zs.avail_in = 0;
00458
00459     for (;;)
00460     {
00461         if (zs.avail_in == 0)
00462         {
00463             is.read(inbuf.data(), static_cast<std::streamsize>(inbuf.size()));
00464             const std::streamsize got = is.gcount();
00465             if (got == 0) break;
00466
00467             zs.next_in = reinterpret_cast<Bytef*>(inbuf.data());
00468             zs.avail_in = static_cast<uInt>(got);
00469         }
00470
00471         zs.next_out = reinterpret_cast<Bytef*>(outbuf.data());
00472         zs.avail_out = static_cast<uInt>(outbuf.size());
00473
00474         const int ret = inflate(&zs, Z_NO_FLUSH);
00475         if (ret != Z_OK && ret != Z_STREAM_END)
00476         {
00477             inflateEnd(&zs);
00478             throw std::runtime_error("inflate error: " + std::to_string(ret));
00479         }
00480
00481         const std::size_t produced = outbuf.size() - zs.avail_out;
00482         if (produced)
00483             raw.insert(raw.end(), outbuf.data(), outbuf.data() + produced);
00484
00485         if (ret == Z_STREAM_END)
00486         {
00487             // More concatenated members?
00488             if (zs.avail_in > 0 || is.peek() != EOF)
00489             {
00490                 if (inflateReset2(&zs, 15 + 32) != Z_OK)
00491                 {
00492                     inflateEnd(&zs);
00493                     throw std::runtime_error("inflateReset2 failed");
00494                 }
00495                 continue;
00496             }
00497             break;
00498         }
00499     }
00500     inflateEnd(&zs);
00501
00502     if (raw.size() % sizeof(PackedParticle) != 0)
00503         throw std::runtime_error("read_records_gzip: size mismatch");
00504
00505     std::vector<PackedParticle> rec(raw.size() / sizeof(PackedParticle));
00506     std::memcpy(rec.data(), raw.data(), raw.size());
00507     return rec;
00508 }
00509
00510 #endif // PK_IO_USE_ZLIB

```

6.10 fixed_point_pkd1_io.hpp

```

00001 // fixed_point_pkd1_io.hpp
00002 // PKD1 plain (uncompressed) file I/O kept out of fixed_point_core.hpp.
00003 #pragma once
00004 #include <fstream>
00005 #include <string>
00006 #include <vector>
00007 #include <stdexcept>
00008 #include <cstring>
00009 #include "fixed_point_core.hpp" // FileHeaderPKD1, PackedParticle
00010
00011 // ----- safe read/write bytes -----
00012 template <class T>
00013 inline void write_all(std::ostream& os, const T* ptr, std::size_t count = 1)
00014 {
00015     os.write(reinterpret_cast<const char*>(ptr), sizeof(T)*count);
00016     if (!os) throw std::runtime_error("write_all: write failed");
00017 }
00018
00019 template <class T>
00020 inline void read_all(std::istream& is, T* ptr, std::size_t count = 1)
00021 {

```

```

00022     is.read(reinterpret_cast<char*>(ptr), sizeof(T) * count);
00023     if (!is) throw std::runtime_error("read_all: read failed");
00024 }
00025
00026 inline void write_header(std::ostream& os, const FileHeaderPKD1& h)
00027 {
00028     write_all(os, &h);
00029 }
00030
00031 inline FileHeaderPKD1 read_header(std::istream& is)
00032 {
00033     FileHeaderPKD1 h{};
00034     read_all(is, &h);
00035
00036     if (h.magic[0] != 'P' || h.magic[1] != 'K' || h.magic[2] != 'D' || h.magic[3] != '1')
00037         throw std::runtime_error("read_header: bad magic (expected PKD1)");
00038     if (h.version != 1)
00039         throw std::runtime_error("read_header: unsupported PKD1 version");
00040
00041     return h;
00042 }
00043
00044 /* ----- plain (uncompressed) I/O ----- */
00045 inline void write_records_binary(const std::string& path,
00046                                const std::vector<PackedParticle>& rec,
00047                                bool append = false)
00048 {
00049     std::ios::openmode mode = std::ios::binary | std::ios::out;
00050     if (append) mode |= std::ios::app;
00051
00052     std::ofstream os(path, mode);
00053     if (!os) throw std::runtime_error("cannot open " + path);
00054
00055     if (!append)
00056     {
00057         FileHeaderPKD1 h{};
00058         write_header(os, h);
00059     }
00060
00061     if (!rec.empty()) write_all(os, rec.data(), rec.size());
00062 }
00063
00064 inline std::vector<PackedParticle> read_records_binary(const std::string& path)
00065 {
00066     std::ifstream is(path, std::ios::binary);
00067     if (!is) throw std::runtime_error("read_records_binary: cannot open file " + path);
00068
00069     (void)read_header(is);
00070
00071     is.seekg(0, std::ios::end);
00072     std::streamoff end = is.tellg();
00073     is.seekg(static_cast<std::streamoff>(sizeof(FileHeaderPKD1)), std::ios::beg);
00074     std::streamoff bytes = end - static_cast<std::streamoff>(sizeof(FileHeaderPKD1));
00075
00076     if (bytes < 0 || bytes % static_cast<std::streamoff>(sizeof(PackedParticle)) != 0)
00077         throw std::runtime_error("read_records_binary: file size mismatch");
00078
00079     std::size_t n = static_cast<std::size_t>(bytes / sizeof(PackedParticle));
00080     std::vector<PackedParticle> rec(n);
00081     if (n) read_all(is, rec.data(), n);
00082     return rec;
00083 }
00084

```

6.11 fixed_point_pkd2.hpp

```

00001 /*****
00002  * fixed_point_pkd2.hpp -- per-trajectory compressed storage (PKD-2)
00003  * Requires: fixed_point_core.hpp
00004  * Define ONE of:
00005  *     PK_IO_USE_ZSTD   (link with -lzstd)
00006  *     PK_IO_USE_ZLIB   (link with -lz)
00007  *****/
00008
00009 // NOTE ON PKD2 BLOCK SIZE LIMITS
00010 //
00011 // PKD2 stores each trajectory as one independently-compressed block.
00012 // The TOC stores `raw_size` and `comp_size` as uint32_t, therefore:
00013 //
00014 // * Each individual trajectory block must be < 4 GiB uncompressed.
00015 // * Each individual trajectory block must be < 4 GiB compressed.
00016 //
00017 // This limit applies regardless of codec (zstd or zlib).

```

```

00018 //
00019 // Additionally, the zlib single-shot APIs used here (compress2/uncompress)
00020 // take sizes as uLong/uLongf (platform-dependent). On some platforms
00021 // (notably Windows), this is effectively a 4 GiB limit as well.
00022 // If you ever need trajectories larger than this, you must split them
00023 // into multiple PKD2 blocks (i.e., change the file format), or implement
00024 // streaming compression and store 64-bit sizes in the TOC.
00025
00026 #pragma once
00027 #include "fixed_point_core.hpp" // fp_pi, quant_coord, wrap_lon, ...
00028 #include <stdint>
00029 #include <cstdio>
00030 #include <cstring>
00031 #include <limits>
00032 #include <stdexcept>
00033 #include <string>
00034 #include <type_traits>
00035 #include <unordered_map>
00036 #include <vector>
00037
00038 #if !defined(PK_IO_USE_ZSTD) && !defined(PK_IO_USE_ZLIB)
00039 #error
00040     "Define either PK_IO_USE_ZSTD or PK_IO_USE_ZLIB before including fixed_point_pkd2.hpp"
00041 #endif
00042 #if defined(PK_IO_USE_ZSTD) && defined(PK_IO_USE_ZLIB)
00043 #error "Define only ONE of PK_IO_USE_ZSTD or PK_IO_USE_ZLIB, not both"
00044 #endif
00045
00046 #ifdef PK_IO_USE_ZSTD
00047 #include <zstd.h>
00048 #endif
00049 #ifdef PK_IO_USE_ZLIB
00050 #include <zlib.h>
00051 #endif
00052
00053 #if defined(PK_IO_USE_ZSTD)
00054 constexpr int kDefaultLevel = 3;
00055 #elif defined(PK_IO_USE_ZLIB)
00056 constexpr int kDefaultLevel = 6;
00057 #endif
00058
00059 namespace pkd2 {
00060
00061 /* ----- file layout ----- */
00062 #pragma pack(push, 1)
00063 struct Header {
00064     char magic[8] = {'P', 'K', 'D', '2', 0, 0, 0, 0};
00065     uint32_t version = 3; // bump
00066     uint8_t little = 1;
00067     uint8_t _r1{0};
00068     uint16_t _r2{0};
00069     uint32_t pos_scale_ticks_per_rad = 1'000'000;
00070     int64_t t0_epoch_s = 0;
00071     int32_t dt_s = 0; // global base step (e.g., 3h)
00072     uint32_t _r3{0};
00073     uint64_t n_traj = 0;
00074     uint64_t toc_offset = 0;
00075 };
00076
00077 // header of the uncompressed per-trajectory block
00078 struct BlockHeader {
00079     uint64_t seed_id;
00080     int32_t start_index; // index of first stored sample on the global 3h grid
00081     uint16_t stride; // >=1 (number of base steps between stored samples)
00082     uint16_t _pad{0}; // keep 4-byte alignment
00083     uint32_t n_points; // >=1 number of stored samples
00084     int32_t lon0_q; // ticks = radians * pos_scale_ticks_per_rad
00085     int32_t lat0_q; // ticks = radians * pos_scale_ticks_per_rad
00086     // then (n_points-1) dlon_q[], then (n_points-1) dlat_q[] (int32 each)
00087 };
00088
00089 // TOC entry at EOF
00090 struct TocEntry {
00091     uint64_t seed_id;
00092     uint64_t file_offset;
00093     uint32_t comp_size;
00094     uint32_t raw_size;
00095     uint32_t n_points;
00096     int32_t start_index;
00097     uint16_t stride; // >=1
00098     uint16_t _pad{0};
00099 };
00100 #pragma pack(pop)
00101
00102 /* ----- writer ----- */
00103 class Writer {
00104 public:

```

```

00105 // open for write; header is written immediately
00106 Writer(const std::string &path, int64_t t0_epoch_s, int32_t dt_s,
00107         uint32_t pos_scale = 1'000'000, int compression_level = kDefaultLevel)
00108     : lvl_(compression_level), pos_scale_(pos_scale) {
00109     f_ = std::fopen(path.c_str(), "wb+");
00110     if (!f_)
00111         throw std::runtime_error("pkd2::Writer: cannot open " + path);
00112     std::memset(&hdr_, 0, sizeof(hdr_));
00113     hdr_.magic[0] = 'P';
00114     hdr_.magic[1] = 'K';
00115     hdr_.magic[2] = 'D';
00116     hdr_.magic[3] = '2';
00117     hdr_.version = 3;
00118     hdr_.little = 1;
00119     hdr_.pos_scale_ticks_per_rad = pos_scale_;
00120     hdr_.t0_epoch_s = t0_epoch_s;
00121     hdr_.dt_s = dt_s;
00122     write_at(0, &hdr_, sizeof(hdr_));
00123     std::fseek(f_, 0, SEEK_END);
00124 }
00125
00126 ~Writer() {
00127     try {
00128         close();
00129     } catch (...) {
00130     }
00131 }
00132
00133 // Append a trajectory given lon/lat in **radians** (the native state)
00134 // Optional stride lets to decimate (e.g. stride=8 turns 3h to 24h).
00135 template <class Real>
00136 void add_traj_rad(uint64_t seed_id, const Real *lon_rad, const Real *lat_rad,
00137                 uint32_t n_points, int32_t start_index,
00138                 uint32_t stride = 1) {
00139     static_assert(std::is_floating_point_v<Real>, "Real must be float/double");
00140     if (n_points == 0)
00141         return;
00142
00143     const bool need_copy = stride > 1 || !std::is_same_v<Real, double>;
00144
00145     if (need_copy) {
00146         tmp_lon_.clear();
00147         tmp_lat_.clear();
00148         tmp_lon_.reserve((n_points + stride - 1) / stride);
00149         tmp_lat_.reserve(tmp_lon_.capacity());
00150         for (uint32_t i = 0; i < n_points; i += stride) {
00151             tmp_lon_.push_back(static_cast<double>(lon_rad[i]));
00152             tmp_lat_.push_back(static_cast<double>(lat_rad[i]));
00153         }
00154         emit_block(seed_id, tmp_lon_.data(), tmp_lat_.data(),
00155                 static_cast<uint32_t>(tmp_lon_.size()), start_index,
00156                 static_cast<uint16_t>(stride));
00157     } else {
00158         // Real == double and stride == 1
00159         emit_block(seed_id, reinterpret_cast<const double *>(lon_rad),
00160                 reinterpret_cast<const double *>(lat_rad), n_points,
00161                 start_index, /*stride*/ 1);
00162     }
00163 }
00164
00165 void close() {
00166     if (!f_)
00167         return;
00168     // write TOC
00169     long pos = std::ftell(f_);
00170     if (pos < 0)
00171         throw std::runtime_error("pkd2::Writer: ftell failed");
00172     if (!toc_.empty()) {
00173         size_t nw = std::fwrite(toc_.data(), sizeof(TocEntry), toc_.size(), f_);
00174         if (nw != toc_.size())
00175             throw std::runtime_error("pkd2::Writer: fwrite TOC failed");
00176     }
00177     std::fflush(f_);
00178     hdr_.n_traj = static_cast<uint64_t>(toc_.size());
00179     hdr_.toc_offset = static_cast<uint64_t>(pos);
00180     write_at(0, &hdr_, sizeof(hdr_));
00181     std::fflush(f_);
00182     std::fclose(f_);
00183     f_ = nullptr;
00184 }
00185
00186 private:
00187     FILE *f_{nullptr};
00188     Header hdr_{};
00189     int lvl_;
00190     uint32_t pos_scale_;
00191     std::vector<TocEntry> toc_;

```



```

00192     std::vector<double> tmp_lon_, tmp_lat_;
00193     std::vector<int32_t> dlon_, dlat_;
00194     std::vector<unsigned char> raw_, comp_;
00195
00196     static inline double clamp_lat(double r) {
00197         const double hp = fp_pi * .5;
00198         return (r < -hp) ? -hp : (r > hp ? hp : r);
00199     }
00200     static inline double unwrap_pi(double r) {
00201         // (-pi,pi]
00202         while (r <= -fp_pi)
00203             r += 2 * fp_pi;
00204         while (r > fp_pi)
00205             r -= 2 * fp_pi;
00206         return r;
00207     }
00208     static inline double unwrap_follow(double prev, double r) {
00209         r = unwrap_pi(r);
00210         double d = r - prev;
00211         if (d > fp_pi)
00212             r -= 2 * fp_pi;
00213         if (d < -fp_pi)
00214             r += 2 * fp_pi;
00215         return r;
00216     }
00217     inline int32_t to_q(double radians) const {
00218         long long v = llround(radians * static_cast<double>(pos_scale_));
00219         if (v > std::numeric_limits<int32_t>::max())
00220             v = std::numeric_limits<int32_t>::max();
00221         if (v < std::numeric_limits<int32_t>::min())
00222             v = std::numeric_limits<int32_t>::min();
00223         return static_cast<int32_t>(v);
00224     }
00225
00226     void emit_block(uint64_t seed_id, const double *lon_r, const double *lat_r,
00227                    uint32_t n, int32_t start_index, uint16_t stride) {
00228         // build uncompressed block [BlockHeader | dlon[n-1] | dlat[n-1]]
00229         BlockHeader bh{};
00230         bh.seed_id = seed_id;
00231         bh.start_index = start_index;
00232         bh.stride = stride ? stride : 1;
00233         bh.n_points = n;
00234
00235         double lon0 = unwrap_pi(lon_r[0]);
00236         double lat0 = clamp_lat(lat_r[0]);
00237         bh.lon0_q = to_q(lon0);
00238         bh.lat0_q = to_q(lat0);
00239
00240         dlon_.assign(n > 1 ? n - 1 : 0, 0);
00241         dlat_.assign(n > 1 ? n - 1 : 0, 0);
00242
00243         double prevL = lon0, prevB = lat0;
00244         for (uint32_t i = 1; i < n; i++) {
00245             double lr = unwrap_follow(prevL, lon_r[i]);
00246             double br = clamp_lat(lat_r[i]);
00247             dlon_[i - 1] = to_q(lr - prevL);
00248             dlat_[i - 1] = to_q(br - prevB);
00249             prevL = lr;
00250             prevB = br;
00251         }
00252
00253         const size_t raw_bytes = sizeof(BlockHeader) +
00254                                 dlon_.size() * sizeof(int32_t) +
00255                                 dlat_.size() * sizeof(int32_t);
00256
00257         // PKD2 format constraint: TOC stores sizes as uint32_t.
00258         if (raw_bytes > static_cast<size_t>(std::numeric_limits<uint32_t>::max()))
00259             throw std::runtime_error("pkd2: raw block exceeds 4GiB (format limit)");
00260
00261         raw_.resize(raw_bytes);
00262         std::memcpy(raw_.data(), &bh, sizeof(bh));
00263         if (!dlon_.empty()) {
00264             std::memcpy(raw_.data() + sizeof(bh), dlon_.data(),
00265                         dlon_.size() * sizeof(int32_t));
00266
00267             std::memcpy(raw_.data() + sizeof(bh) + dlon_.size() * sizeof(int32_t),
00268                         dlat_.data(), dlat_.size() * sizeof(int32_t));
00269         }
00270
00271         // compress to memory
00272         uint32_t comp_sz = 0;
00273
00274 #ifdef PK_IO_USE_ZSTD
00275     {
00276         const size_t bound = ZSTD_compressBound(raw_bytes);
00277
00278         // Still must fit PKD2's uint32_t TOC fields.

```

```

00279     if (bound > static_cast<size_t>(std::numeric_limits<uint32_t>::max()))
00280         throw std::runtime_error(
00281             "pkd2: zstd compressBound exceeds 4GiB (format limit)");
00282
00283     comp_.resize(bound);
00284
00285     const size_t z =
00286         ZSTD_compress(comp_.data(), bound, raw_.data(), raw_bytes, lvl_);
00287     if (ZSTD_isError(z))
00288         throw std::runtime_error(std::string("ZSTD_compress: ") +
00289             ZSTD_getErrorName(z));
00290
00291     if (z > static_cast<size_t>(std::numeric_limits<uint32_t>::max()))
00292         throw std::runtime_error(
00293             "pkd2: compressed block exceeds 4GiB (format limit)");
00294
00295     comp_sz = static_cast<uint32_t>(z);
00296 }
00297 #elif defined(PK_IO_USE_ZLIB)
00298 {
00299     // zlib APIs here take uLong/uLongf (platform-dependent width).
00300     if (raw_bytes > static_cast<size_t>(std::numeric_limits<uLong>::max()))
00301         throw std::runtime_error("pkd2: raw block too large for zlib uLong");
00302
00303     const uLongf bound = compressBound(static_cast<uLong>(raw_bytes));
00304
00305     if (static_cast<size_t>(bound) >
00306         static_cast<size_t>(std::numeric_limits<uint32_t>::max()))
00307         throw std::runtime_error(
00308             "pkd2: zlib compressBound exceeds 4GiB (format limit)");
00309
00310     comp_.resize(static_cast<size_t>(bound));
00311
00312     uLongf z = bound;
00313     const int rc = compress2(comp_.data(), &z,
00314         reinterpret_cast<const Bytef *>(raw_.data()),
00315         static_cast<uLong>(raw_bytes), lvl_);
00316     if (rc != Z_OK)
00317         throw std::runtime_error("pkd2: compress2 failed");
00318
00319     if (static_cast<size_t>(z) >
00320         static_cast<size_t>(std::numeric_limits<uint32_t>::max()))
00321         throw std::runtime_error(
00322             "pkd2: compressed block exceeds 4GiB (format limit)");
00323
00324     comp_sz = static_cast<uint32_t>(z);
00325 }
00326 #endif
00327
00328 // write to file, remember offset
00329 long off = std::ftell(f_);
00330 if (off < 0)
00331     throw std::runtime_error("pkd2: ftell failed");
00332
00333 const size_t nw = std::fwrite(comp_.data(), 1, comp_sz, f_);
00334 if (nw != comp_sz)
00335     throw std::runtime_error("pkd2: fwrite failed");
00336
00337 TocEntry te{};
00338 te.seed_id = seed_id;
00339 te.file_offset = static_cast<uint64_t>(off);
00340 te.comp_size = comp_sz;
00341 te.raw_size = static_cast<uint32_t>(raw_bytes);
00342 te.n_points = n;
00343 te.start_index = start_index;
00344 te.stride = bh.stride;
00345
00346 toc_.push_back(te);
00347 }
00348
00349 void write_at(long off, const void *p, size_t n) {
00350     if (std::fseek(f_, off, SEEK_SET) != 0)
00351         throw std::runtime_error("fseek failed");
00352     size_t nw = std::fwrite(p, 1, n, f_);
00353     if (nw != n)
00354         throw std::runtime_error("header write failed");
00355     std::fseek(f_, 0, SEEK_END);
00356 }
00357 };
00358
00359 /* ----- reader ----- */
00360 class Reader {
00361 public:
00362     explicit Reader(const std::string &path) {
00363         f_ = std::fopen(path.c_str(), "rb");
00364         if (!f_)
00365             throw std::runtime_error("pkd2::Reader: cannot open " + path);

```

```

00366     read_at(0, &hdr_, sizeof(hdr_));
00367     if (hdr_.magic[0] != 'P' || hdr_.magic[1] != 'K' || hdr_.magic[2] != 'D' ||
00368         hdr_.magic[3] != '2')
00369         throw std::runtime_error("pkd2: bad magic");
00370     if (hdr_.version != 3)
00371         throw std::runtime_error("pkd2: version != 3");
00372     // read TOC
00373     toc_.resize(hdr_.n_traj);
00374     read_at(static_cast<long>(hdr_.toc_offset), toc_.data(),
00375         toc_.size() * sizeof(TocEntry));
00376     for (size_t i = 0; i < toc_.size(); ++i)
00377         index_[toc_[i].seed_id] = static_cast<uint32_t>(i);
00378 }
00379 ~Reader() {
00380     if (f_)
00381         std::fclose(f_);
00382 }
00383
00384 bool has(uint64_t seed_id) const {
00385     return index_.find(seed_id) != index_.end();
00386 }
00387
00388 // Read one trajectory; outputs in **radians** (to match sim)
00389 void read_traj(uint64_t seed_id, std::vector<double> &lon_r,
00390     std::vector<double> &lat_r, std::vector<int64_t> &t_s) const {
00391     auto it = index_.find(seed_id);
00392     if (it == index_.end())
00393         throw std::runtime_error("pkd2: seed_id not found");
00394     const TocEntry &te = toc_[it->second];
00395
00396     // read compressed block
00397     comp_.resize(te.comp_size);
00398     read_at(static_cast<long>(te.file_offset), comp_.data(), comp_.size());
00399
00400     // decompress
00401     raw_.resize(te.raw_size);
00402 #ifdef PK_IO_USE_ZSTD
00403     size_t out =
00404         ZSTD_decompress(raw_.data(), raw_.size(), comp_.data(), comp_.size());
00405     if (ZSTD_isError(out) || out != raw_.size())
00406         throw std::runtime_error("ZSTD_decompress failed");
00407 #elif defined(PK_IO_USE_ZLIB)
00408     uLongf out = static_cast<uLongf>(raw_.size());
00409     int rc = uncompress(raw_.data(), &out, comp_.data(),
00410         static_cast<uLong>(comp_.size()));
00411     if (rc != Z_OK || out != raw_.size())
00412         throw std::runtime_error("uncompress failed");
00413 #endif
00414     // parse block
00415     if (raw_.size() < sizeof(BlockHeader))
00416         throw std::runtime_error("pkd2: raw too small");
00417     const BlockHeader *bh = reinterpret_cast<const BlockHeader *>(raw_.data());
00418     const uint16_t s = bh->stride ? bh->stride : 1;
00419     if (bh->n_points != te.n_points)
00420         throw std::runtime_error("pkd2: n_points mismatch");
00421
00422     const int32_t *dlon =
00423         reinterpret_cast<const int32_t *>(raw_.data() + sizeof(BlockHeader));
00424     const int32_t *dlat = dlon + (bh->n_points > 0 ? bh->n_points - 1 : 0);
00425
00426     lon_r.resize(bh->n_points);
00427     lat_r.resize(bh->n_points);
00428     t_s.resize(bh->n_points);
00429
00430     // reconstruct
00431     const double inv_scale =
00432         1. / static_cast<double>(hdr_.pos_scale_ticks_per_rad);
00433     double L = static_cast<double>(bh->lon0_q) * inv_scale;
00434     double B = static_cast<double>(bh->lat0_q) * inv_scale;
00435     lon_r[0] = L;
00436     lat_r[0] = B;
00437     t_s[0] =
00438         hdr_.t0_epoch_s + static_cast<int64_t>(bh->start_index) * hdr_.dt_s;
00439
00440     for (uint32_t i = 1; i < bh->n_points; ++i) {
00441         L += static_cast<double>(dlon[i - 1]) * inv_scale;
00442         B += static_cast<double>(dlat[i - 1]) * inv_scale;
00443         lon_r[i] = L;
00444         lat_r[i] = B;
00445         t_s[i] = t_s[0] + static_cast<int64_t>(i) * hdr_.dt_s * s;
00446     }
00447 }
00448
00449 const Header &header() const { return hdr_; }
00450 const std::vector<TocEntry> &toc() const { return toc_; }
00451
00452 private:

```

```

00453 FILE *f_{nullptr};
00454 Header hdr_{};
00455 std::vector<TocEntry> toc_;
00456 std::unordered_map<uint64_t, uint32_t> index_;
00457 mutable std::vector<unsigned char> comp_, raw_;
00458
00459 void read_at(long off, void *p, size_t n) const {
00460     if (std::fseek(f_, off, SEEK_SET) != 0)
00461         throw std::runtime_error("fseek failed");
00462     size_t nr = std::fread(p, 1, n, f_);
00463     if (nr != n)
00464         throw std::runtime_error("fread failed");
00465 }
00466 };
00467
00468 } // namespace pkd2

```

6.12 pk_config.hpp

```

00001 // pk_config.hpp
00002 // Central project-wide configuration for PKD compression backend.
00003 // Include this BEFORE any PKD headers.
00004 #pragma once
00005
00006 // Choose exactly ONE codec in your build system instead of here:
00007 // -DPK_IO_USE_ZLIB (link with -lz)
00008 // -DPK_IO_USE_ZSTD (link with -lzstd)
00009 //
00010 // #define PK_IO_USE_ZLIB
00011 // #define PK_IO_USE_ZSTD
00012
00013 #if !defined(PK_IO_USE_ZSTD) && !defined(PK_IO_USE_ZLIB)
00014 # error "Define either PK_IO_USE_ZSTD or PK_IO_USE_ZLIB before including PKD headers."
00015 #endif
00016 #if defined(PK_IO_USE_ZSTD) && defined(PK_IO_USE_ZLIB)
00017 # error "Define only ONE of PK_IO_USE_ZSTD or PK_IO_USE_ZLIB, not both."
00018 #endif
00019

```

6.13 include/particle_tracker/geography_utils.hpp File Reference

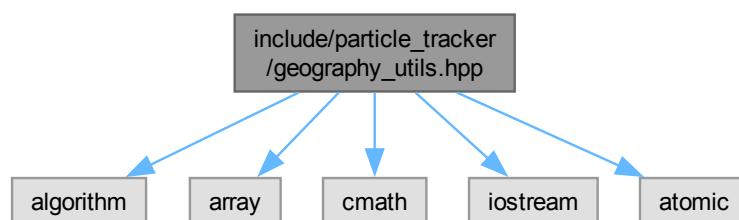
Small geographic utilities for spherical (lon/lat) computations in radians.

```

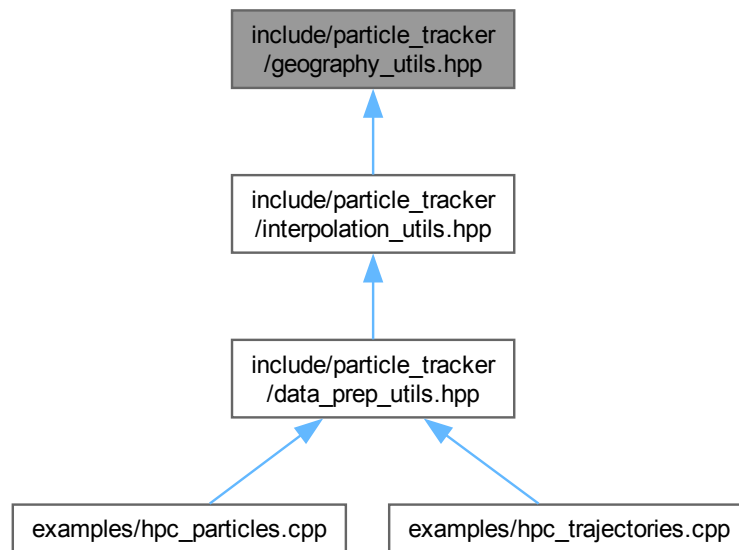
#include <algorithm>
#include <array>
#include <cmath>
#include <iostream>
#include <atomic>

```

Include dependency graph for geography_utils.hpp:



This graph shows which files directly or indirectly include this file:



Macros

- `#define PARTICLE_TRACKER_GCD_DERIV_WARN 1`
- `#define PARTICLE_TRACKER_GCD_DERIV_WARN_ALWAYS 0`

Functions

- `template<typename DataType>`
`DataType particle_tracker::clamp (DataType val, DataType lo, DataType hi)`
Clamp a value to the closed interval [lo, hi].
- `template<typename DataType>`
`DataType particle_tracker::GCD (const DataType &lon1, const DataType &lat1, const DataType &lon2, const DataType &lat2)`
Great-circle distance (central angle) between two points on a sphere.
- `template<typename DataType>`
`std::array< DataType, 3 > particle_tracker::GCD_deriv (const DataType &lon1, const DataType &lat1, const DataType &lon2, const DataType &lat2)`
Great-circle distance (central angle) and its partial derivatives w.r.t.
- `template<typename DataType>`
`void particle_tracker::inverseTransform (const DataType &x, const DataType &y, DataType &lon, DataType &lat)`
Apply a local tangent-plane offset (x,y in meters) to a reference lon/lat (radians).

6.13.1 Detailed Description

Small geographic utilities for spherical (lon/lat) computations in radians.

This header provides:

- A generic clamp helper.
- Great-circle distance (central angle) on a sphere.
- Great-circle distance and partial derivatives w.r.t. the *first* point.
- A simple "inverse transform" that applies a local tangent-plane offset (x,y in meters) to a reference lon/lat and returns the new lon/lat.

6.13.1.1 Conventions

- All angles (lon/lat) are in **radians**.
- Great-circle distance returned by GCD / GCD_deriv is the **central angle** (radians), i.e. arc length is $R * \text{angle}$.

6.13.1.2 Error policy / "don't ruin the simulation"

- clamp() is used to keep floating-point roundoff from violating the domain constraints of trig inverse operations.
- GCD_deriv() does **not** throw. If the derivative is mathematically undefined or numerically unstable because the two points are coincident (or nearly coincident), it returns: { d, 0, 0 } and emits a very cheap warning to std::cerr (at most once).

6.13.1.3 Optional warning control

- By default, one warning per process is printed the first time a degenerate derivative is detected.
- Define PARTICLE_TRACKER_GCD_DERIV_WARN=0 to disable all warnings.
- Define PARTICLE_TRACKER_GCD_DERIV_WARN_ALWAYS=1 to warn every time (not recommended on HPC).

Definition in file [geography_utils.hpp](#).

6.13.2 Macro Definition Documentation

6.13.2.1 PARTICLE_TRACKER_GCD_DERIV_WARN

```
#define PARTICLE_TRACKER_GCD_DERIV_WARN 1
```

Definition at line 46 of file [geography_utils.hpp](#).

6.13.2.2 PARTICLE_TRACKER_GCD_DERIV_WARN_ALWAYS

```
#define PARTICLE_TRACKER_GCD_DERIV_WARN_ALWAYS 0
```

Definition at line 50 of file [geography_utils.hpp](#).

6.13.3 Function Documentation

6.13.3.1 clamp()

```
template<typename DataType>
DataType particle_tracker::clamp (
    DataType val,
    DataType lo,
    DataType hi) [inline]
```

Clamp a value to the closed interval [lo, hi].

Template Parameters

<i>DataType</i>	numeric type
-----------------	--------------

Parameters

<i>val</i>	input value
<i>lo</i>	lower bound
<i>hi</i>	upper bound

Returns

clamped value

Definition at line 67 of file [geography_utils.hpp](#).

6.13.3.2 GCD()

```
template<typename DataType>
DataType particle_tracker::GCD (
    const DataType & lon1,
    const DataType & lat1,
    const DataType & lon2,
    const DataType & lat2) [inline]
```

Great-circle distance (central angle) between two points on a sphere.

Template Parameters

<i>DataType</i>	floating-point type (float/double/long double)
-----------------	--

Parameters

<i>lon1</i>	longitude of point 1 (radians)
<i>lat1</i>	latitude of point 1 (radians)
<i>lon2</i>	longitude of point 2 (radians)
<i>lat2</i>	latitude of point 2 (radians)

Returns

central angle distance in radians (in $[0, \pi]$)

Implementation:

- Computes $\cos(d)$ using spherical law of cosines.
- Uses $\text{atan2}(\sin(d), \cos(d))$ for numerical robustness.

Definition at line 91 of file [geography_utils.hpp](#).

6.13.3.3 GCD_deriv()

```
template<typename DataType>
std::array< DataType, 3 > particle_tracker::GCD_deriv (
    const DataType & lon1,
    const DataType & lat1,
    const DataType & lon2,
    const DataType & lat2) [inline]
```

Great-circle distance (central angle) and its partial derivatives w.r.t.

the first point.

Template Parameters

<i>DataType</i>	floating-point type
-----------------	---------------------

Parameters

<i>lon1</i>	longitude of point 1 (radians)
<i>lat1</i>	latitude of point 1 (radians)
<i>lon2</i>	longitude of point 2 (radians)
<i>lat2</i>	latitude of point 2 (radians)

Returns

array { d, dd_dlon1, dd_dlat1 }

Where:

- d : central angle distance (radians)
- dd_dlon1 : partial derivative of d w.r.t. lon1
- dd_dlat1 : partial derivative of d w.r.t. lat1

6.13.3.3.1 Degenerate case handling ($d \sim 0$)

The analytic expressions contain division by $\sin(d)$. When the two points coincide (or are extremely close), $\sin(d) \rightarrow 0$ and the derivatives are undefined / numerically unstable.

Policy:

- Return {d, 0, 0} (safe "no directional information" fallback).
- Optionally print a warning to `std::cerr`:
 - by default: at most once per process (very cheap)
 - disable: `PARTICLE_TRACKER_GCD_DERIV_WARN=0`
 - always: `PARTICLE_TRACKER_GCD_DERIV_WARN_ALWAYS=1` (not recommended for HPC)

Notes:

- These derivatives are for the central angle d, not arc length $R*d$.
- If you need derivatives of arc length, multiply `dd_dlon1` and `dd_dlat1` by Earth radius.

Definition at line 136 of file [geography_utils.hpp](#).

6.13.3.4 inverseTransform()

```
template<typename DataType>
void particle_tracker::inverseTransform (
    const DataType & x,
    const DataType & y,
    DataType & lon,
    DataType & lat) [inline]
```

Apply a local tangent-plane offset (x,y in meters) to a reference lon/lat (radians).

This implements the spherical **azimuthal equidistant (AEQD)** inverse transform centered at the reference point (lon,lat). The offset (x,y) is interpreted in the local tangent plane:

- x: east (meters)
- y: north (meters)

Template Parameters

<i>DataType</i>	floating-point type
-----------------	---------------------

Parameters

<i>x</i>	eastward offset (meters)
<i>y</i>	northward offset (meters)
<i>lon</i>	[in,out] reference longitude (radians), updated in-place
<i>lat</i>	[in,out] reference latitude (radians), updated in-place

The returned point lies at great-circle distance $|(x,y)|$ from the center, with bearing given by (x,y) .

Note

Spherical Earth model with radius `EARTH_RADIUS` (not WGS84 ellipsoid).

Accuracy degrades for very large offsets (hundreds–thousands of km).

Definition at line 216 of file [geography_utils.hpp](#).

6.14 geography_utils.hpp

[Go to the documentation of this file.](#)

```

00001 #pragma once
00031
00032 #include <algorithm>    // std::min, std::max
00033 #include <array>
00034 #include <cmath>        // std::sin, std::cos, std::sqrt, std::atan2, std::asin
00035 #include <iostream>     // std::cerr
00036 #include <atomic>       // std::atomic_flag
00037
00038 namespace particle_tracker
00039 {
00040
00041 //-----
00042 // Compile-time configuration
00043 //-----
00044
00045 #ifndef PARTICLE_TRACKER_GCD_DERIV_WARN
00046 #define PARTICLE_TRACKER_GCD_DERIV_WARN 1
00047 #endif
00048
00049 #ifndef PARTICLE_TRACKER_GCD_DERIV_WARN_ALWAYS
00050 #define PARTICLE_TRACKER_GCD_DERIV_WARN_ALWAYS 0
00051 #endif
00052
00053 //-----
00054 // Utility helpers
00055 //-----
00056
00066 template<typename DataType>
00067 inline DataType clamp(DataType val, DataType lo, DataType hi)
00068 {
00069     return std::max(lo, std::min(val, hi));
00070 }
00071
00072 //-----
00073 // Great-circle distance
00074 //-----
00075
00090 template<typename DataType>
00091 inline DataType GCD(const DataType& lon1, const DataType& lat1,
00092                    const DataType& lon2, const DataType& lat2)
00093 {

```

```

00094     DataType cn =
00095         std::sin(lat1) * std::sin(lat2) +
00096         std::cos(lat1) * std::cos(lat2) * std::cos(lon1 - lon2);
00097
00098     cn = clamp(cn, static_cast<DataType>(-1), static_cast<DataType>(1));
00099
00100     DataType sn = std::sqrt(static_cast<DataType>(1) - cn * cn);
00101
00102     return std::atan2(sn, cn);
00103 }
00104
00105 template<typename DataType>
00106 inline std::array<DataType, 3> GCD_deriv(const DataType& lon1, const DataType& lat1,
00107                                         const DataType& lon2, const DataType& lat2)
00108 {
00109     // cos(d)
00110     DataType cn =
00111         std::sin(lat1) * std::sin(lat2) +
00112         std::cos(lat1) * std::cos(lat2) * std::cos(lon1 - lon2);
00113
00114     cn = clamp(cn, static_cast<DataType>(-1), static_cast<DataType>(1));
00115
00116     // sin(d)
00117     DataType sn = std::sqrt(static_cast<DataType>(1) - cn * cn);
00118
00119     // d
00120     DataType d = std::atan2(sn, cn);
00121
00122     // Guard: derivatives contain division by sin(d)
00123     // Choose eps relative to DataType.
00124     // - For double, 1e-14 is conservative; for float, it still works.
00125     const DataType eps = static_cast<DataType>(1e-14);
00126
00127     if (sn <= eps)
00128     {
00129         #if PARTICLE_TRACKER_GCD_DERIV_WARN
00130         #if PARTICLE_TRACKER_GCD_DERIV_WARN_ALWAYS
00131             std::cerr
00132                 << "particle_tracker::GCD_deriv: sin(d) ~ 0 (coincident or nearly coincident points). "
00133                 << "Returning {d,0,0} to avoid NaN/Inf.\n";
00134         #else
00135             // Print at most once per process.
00136             static std::atomic_flag warned = ATOMIC_FLAG_INIT;
00137             if (!warned.test_and_set(std::memory_order_relaxed))
00138             {
00139                 std::cerr
00140                     << "particle_tracker::GCD_deriv: sin(d) ~ 0 (coincident or nearly coincident points). "
00141                     << "Returning {d,0,0} to avoid NaN/Inf. (This warning is printed once.)\n";
00142             }
00143         #endif
00144     }
00145     #endif
00146     #endif
00147     return {d, static_cast<DataType>(0), static_cast<DataType>(0)};
00148 }
00149
00150 // Partial derivative w.r.t. lon1.
00151 // Note: lon enters through cos(lon1-lon2); derivative uses sin(lon2-lon1).
00152 DataType dd_dlon1 = d * std::cos(lat2) * std::sin(lon2 - lon1) / sn;
00153
00154 // Partial derivative w.r.t. lat1.
00155 DataType dd_dlat1 =
00156     d * (std::cos(lat1) * std::sin(lat2) -
00157         std::sin(lat1) * std::cos(lat2) * std::cos(lon2 - lon1)) / sn;
00158
00159 return {d, dd_dlon1, dd_dlat1};
00160 }
00161
00162 //-----
00163 // Local tangent-plane offset -> lon/lat update
00164 //-----
00165
00166 template<typename DataType>
00167 inline void inverseTransform(const DataType& x, const DataType& y, DataType& lon, DataType& lat)
00168 {
00169     constexpr DataType EARTH_RADIUS = static_cast<DataType>(6378000.0);
00170
00171     DataType nrm = std::sqrt(x * x + y * y);
00172
00173     const DataType eps = static_cast<DataType>(1e-10);
00174     if (nrm < eps)
00175         return;
00176
00177     DataType gcd = nrm / EARTH_RADIUS;
00178
00179     DataType cn = std::cos(gcd);
00180     DataType sn = std::sin(gcd);
00181
00182     DataType cl = std::cos(lat);

```

```

00232     DataType sl = std::sin(lat);
00233
00234     DataType arg = sl * cn + (y / nrm) * cl * sn;
00235     arg = clamp(arg, static_cast<DataType>(-1), static_cast<DataType>(1));
00236
00237     DataType lat_prime = std::asin(arg);
00238
00239     DataType lon_prime = lon + std::atan2(x * sn, nrm * cl * cn - y * sl * sn);
00240
00241     lat = lat_prime;
00242     lon = lon_prime;
00243 }
00244
00245 } // namespace particle_tracker
00246

```

6.15 include/particle_tracker/interpolation_utils.hpp File Reference

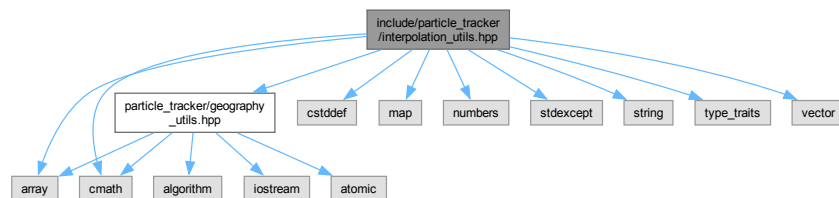
Time and spherical (lon/lat) interpolation utilities, including an experimental "neighbors-of-neighbors" spatial derivative estimator.

```

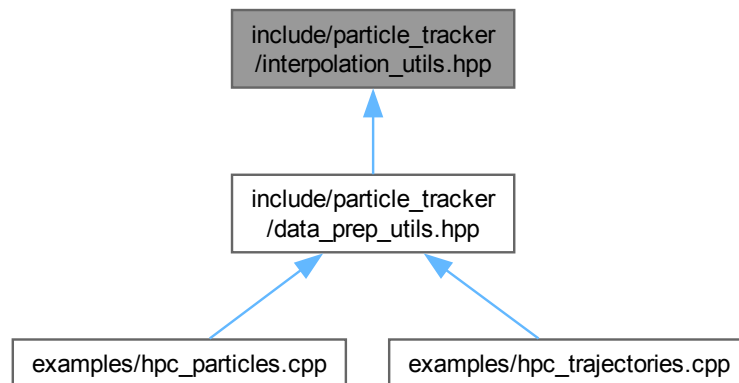
#include <array>
#include <cmath>
#include <cstdint>
#include <map>
#include <numbers>
#include <stdexcept>
#include <string>
#include <type_traits>
#include <vector>
#include <particle_tracker/geography_utils.hpp>

```

Include dependency graph for interpolation_utils.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class `particle_tracker::TimeSpline< DataType, TimeType >`
Wrapper around a Boost cubic spline interpolator for uniformly spaced samples.
- struct `particle_tracker::PointCloud< DataType >`
Minimal dataset adaptor for nanoflann storing (lon, lat) in a flat array.
- struct `particle_tracker::LonLatDistanceAdaptor< DataType, DatasetAdaptor >`
Approximate metric for KD-tree pruning in (lon, lat).
- struct `particle_tracker::Neighbor< DataType >`
One spatial neighbor (grid node) used in interpolation.
- class `particle_tracker::NeighborsData< DataType, TimeType >`
Spatial neighbor query + Gaussian weighting + multi-variable interpolation.
- struct `particle_tracker::NeighborPrime< DataType >`
A "secondary neighbor" (neighbor of a primary neighbor).
- struct `particle_tracker::NeighborWithNeighbors< DataType >`
A primary [Neighbor](#) plus a list of its secondary neighbors.
- struct `particle_tracker::VariableDerivatives< DataType >`
Output container: value + time derivative + (heuristic) spatial derivatives.
- class `particle_tracker::NeighborWithNeighborsData< DataType, TimeType >`
Two-level neighborhood builder + heuristic spatial derivative estimator.

Macros

- `#define PARTICLE_TRACKER_HAS_NANOFANN 1`
Set to 1 to enable nanoflann-backed KD-tree functionality.

Typedefs

- `template<typename DataType>`
using `particle_tracker::point` = `std::array<DataType, 2>`
2D point type used for (lon, lat).
- `template<typename DataType>`
using `particle_tracker::KDTree`
KD-tree alias used throughout this header when nanoflann is enabled.

Functions

- `template<typename T>`
`T particle_tracker::pt_wrap_lon (T lon)`
Wrap longitude into $(-pi, pi]$.

Variables

- `constexpr double particle_tracker::pt_pi = std::numbers::pi_v<double>`
 π in double precision (radians).
- `constexpr double particle_tracker::pt_two_pi = 2. * pt_pi`
 2π in double precision (radians).
- `constexpr double particle_tracker::dt = 10800.0`
Default time spacing used in some HYCOM-style datasets (3 hours).

6.15.1 Detailed Description

Time and spherical (lon/lat) interpolation utilities, including an experimental "neighbors-of-neighbors" spatial derivative estimator.

6.15.1.1 What this header provides

- **Time interpolation** of scalar fields using a cubic B-spline (Boost).
- **Spatial neighbor queries** on a sphere (lon/lat) via a KD-tree (nanoflann) using a fast approximate metric suitable for KD-tree pruning.
- **Spatial interpolation** using Gaussian weights derived from *true* great-circle distance.
- **Experimental** spatial derivative estimation using a two-level neighborhood ("neighbors of neighbors") and distance-derivative information.

6.15.1.2 Coordinate conventions

- Coordinates are in radians:
 - longitude `lon` in $(-pi, pi]$
 - latitude `lat` in $[-pi/2, pi/2]$

6.15.1.3 Dependencies

- Boost (either:
 - `<boost/math/interpolators/cardinal_cubic_b_spline.hpp>` (newer)
 - or `<boost/math/interpolators/cubic_b_spline.hpp>` (older / deprecated in some newer Boost)
- nanoflann (header-only) for KD-tree support.

6.15.1.4 Error policy

This header prefers **loud failures** over silent bad behavior:

- If enable nanoflann support and the header cannot be found: **compile-time error**.
- If disable nanoflann support but try to instantiate KD-tree dependent types: **compile-time error**.

6.15.1.5 Threading

- No internal synchronization is provided.
- Reads are safe if underlying data structures are immutable.
- The caller must protect shared mutable structures.

Definition in file [interpolation_utils.hpp](#).

6.15.2 Macro Definition Documentation

6.15.2.1 PARTICLE_TRACKER_HAS_NANOFLANN

```
#define PARTICLE_TRACKER_HAS_NANOFLANN 1
```

Set to 1 to enable nanoflann-backed KD-tree functionality.

If set to 0, KD-tree dependent classes/types in this header become unavailable and will produce a compile-time error if instantiated.

Definition at line 66 of file [interpolation_utils.hpp](#).

6.15.3 Typedef Documentation

6.15.3.1 KDTree

```
template<typename DataType>  
using particle_tracker::KDTree
```

Initial value:

```
nf::KDTreeSingleIndexAdaptor<  
    LonLatDistanceAdaptor<DataType, PointCloud<DataType>>, PointCloud<DataType>,  
    2>
```

KD-tree alias used throughout this header when nanoflann is enabled.

Definition at line 368 of file [interpolation_utils.hpp](#).

6.15.3.2 point

```
template<typename DataType>
using particle_tracker::point = std::array<DataType, 2>
```

2D point type used for (lon, lat).

Template Parameters

<i>DataType</i>	numeric type
-----------------	--------------

Definition at line 153 of file [interpolation_utils.hpp](#).

6.15.4 Function Documentation

6.15.4.1 pt_wrap_lon()

```
template<typename T>
T particle_tracker::pt_wrap_lon (
    T lon) [inline]
```

Wrap longitude into (-pi, pi].

Template Parameters

<i>T</i>	Floating-point type.
----------	----------------------

Parameters

<i>lon</i>	Longitude in radians.
------------	-----------------------

Returns

Wrapped longitude in (-pi, pi].

This is used to avoid discontinuities across the dateline.

Definition at line 137 of file [interpolation_utils.hpp](#).

6.15.5 Variable Documentation

6.15.5.1 dtt

```
double particle_tracker::dtt = 10800.0 [inline], [constexpr]
```

Default time spacing used in some HYCOM-style datasets (3 hours).

This constant is not required by the classes but is useful as a conventional default.

Definition at line 161 of file [interpolation_utils.hpp](#).

6.15.5.2 pt_pi

```
double particle_tracker::pt_pi = std::numbers::pi_v<double> [inline], [constexpr]
```

pi in double precision (radians).

Definition at line 125 of file [interpolation_utils.hpp](#).

6.15.5.3 pt_two_pi

```
double particle_tracker::pt_two_pi = 2. * pt_pi [inline], [constexpr]
```

2*pi in double precision (radians).

Definition at line 127 of file [interpolation_utils.hpp](#).

6.16 interpolation_utils.hpp

[Go to the documentation of this file.](#)

```
00001 #pragma once
00041
00042 #include <array>
00043 #include <cmath> // std::cos, std::exp
00044 #include <cstdint> // std::size_t
00045 #include <map>
00046 #include <numbers> // std::numbers::pi_v
00047 #include <stdexcept> // std::runtime_error, std::out_of_range
00048 #include <string>
00049 #include <type_traits> // std::false_type
00050 #include <vector>
00051
00052 #include <particle_tracker/geography_utils.hpp>
00053
00054 //-----
00055 // User-configurable feature switches
00056 //-----
00057
00065 #ifndef PARTICLE_TRACKER_HAS_NANOFANN
00066 #define PARTICLE_TRACKER_HAS_NANOFANN 1
00067 #endif
00068
00069 //-----
00070 // Boost spline selection (new vs old header layouts)
00071 //-----
00072 //
00073 // Newer Boost provides interpolators::cardinal_cubic_b_spline.
00074 // Older Boost provides cubic_b_spline.
00075 // We detect availability by header presence and define consistent macros.
00076 //-----
00077
00078 #if __has_include(<boost/math/interpolators/cardinal_cubic_b_spline.hpp>)
00079 #include <boost/math/interpolators/cardinal_cubic_b_spline.hpp>
00080 #define PARTICLE_TRACKER_HAS_BOOST_CARDINAL_CUBIC_B_SPLINE 1
00081 #define PARTICLE_TRACKER_HAS_BOOST_CUBIC_B_SPLINE 0
00082 #elif __has_include(<boost/math/interpolators/cubic_b_spline.hpp>)
00083 #include <boost/math/interpolators/cubic_b_spline.hpp>
00084 #define PARTICLE_TRACKER_HAS_BOOST_CARDINAL_CUBIC_B_SPLINE 0
00085 #define PARTICLE_TRACKER_HAS_BOOST_CUBIC_B_SPLINE 1
00086 #else
00087 #error \
00088     "particle_tracker: Boost spline header not found. Need either
00089     <boost/math/interpolators/cardinal_cubic_b_spline.hpp> or
00090     <boost/math/interpolators/cubic_b_spline.hpp>."
00089 #endif
00090
00091 //-----
00092 // nanoflann detection and include (only if enabled)
00093 //-----
00094 //
00095 // Supported include layouts:
```

```

00096 // - <nanoflann.hpp> (system installs)
00097 // - <nanoflann/nanoflann.hpp> (some packagers)
00098 // - <third_party/nanoflann/nanoflann.hpp> (vendored under include/third_party)
00099 //-----
00100
00101 #if PARTICLE_TRACKER_HAS_NANOFLANN
00102 #if __has_include(<nanoflann.hpp>)
00103 #include <nanoflann.hpp>
00104 #elif __has_include(<nanoflann/nanoflann.hpp>)
00105 #include <nanoflann/nanoflann.hpp>
00106 #elif __has_include(<third_party/nanoflann/nanoflann.hpp>)
00107 #include <third_party/nanoflann/nanoflann.hpp>
00108 #else
00109 #error
00110 "particle_tracker: nanoflann enabled (PARTICLE_TRACKER_HAS_NANOFLANN=1) but nanoflann header not
found. Install nanoflann or vendor it under include/third_party/nanoflann and include it as
<third_party/nanoflann/nanoflann.hpp>."
00111 #endif
00112 #endif
00113
00114 namespace particle_tracker {
00115
00116 #if PARTICLE_TRACKER_HAS_NANOFLANN
00117 namespace nf = nanoflann;
00118 #endif
00119
00120 //-----
00121 // Constants and helpers
00122 //-----
00123
00125 inline constexpr double pt_pi = std::numbers::pi_v<double>;
00127 inline constexpr double pt_two_pi = 2. * pt_pi;
00128
00137 template <typename T> inline T pt_wrap_lon(T lon) {
00138 // Use pt_pi constants but cast to T so comparisons behave correctly.
00139 const T pi = static_cast<T>(pt_pi);
00140 const T two_pi = static_cast<T>(pt_two_pi);
00141
00142 while (lon <= -pi)
00143     lon += two_pi;
00144 while (lon > pi)
00145     lon -= two_pi;
00146 return lon;
00147 }
00148
00153 template <typename DataType> using point = std::array<DataType, 2>;
00154
00161 inline constexpr double dtt = 10800.0;
00162
00163 //-----
00164 // TimeSpline
00165 //-----
00166
00188 template <typename DataType, typename TimeType = double> class TimeSpline {
00189 public:
00191     std::vector<DataType> values;
00192
00193 #if PARTICLE_TRACKER_HAS_BOOST_CUBIC_B_SPLINE
00195     boost::math::cubic_b_spline<DataType> spline;
00196 #elif PARTICLE_TRACKER_HAS_BOOST_CARDINAL_CUBIC_B_SPLINE
00198     boost::math::interpolators::cardinal_cubic_b_spline<DataType> spline;
00199 #endif
00200
00201     TimeSpline() = default;
00202
00210     void createSpline(TimeType start_time, TimeType time_step) {
00211         if (values.empty())
00212             throw std::runtime_error("particle_tracker::TimeSpline: no data to "
00213                                     "create spline (values is empty).");
00214
00215 #if PARTICLE_TRACKER_HAS_BOOST_CUBIC_B_SPLINE
00216 // Old Boost API accepts pointer + size + (t0, dt)
00217 spline = boost::math::cubic_b_spline<DataType>{
00218     values.data(), values.size(), static_cast<DataType>(start_time),
00219     static_cast<DataType>(time_step)};
00220 #elif PARTICLE_TRACKER_HAS_BOOST_CARDINAL_CUBIC_B_SPLINE
00221 // This Boost version accepts pointer + size + (t0, dt)
00222 spline = boost::math::interpolators::cardinal_cubic_b_spline<DataType>{
00223     values.data(), values.size(), static_cast<DataType>(start_time),
00224     static_cast<DataType>(time_step)};
00225 #endif
00226     }
00227 };
00228
00229 //-----
00230 // nanoflann dataset adaptor + distance adaptor + KDTree alias
00231 //-----

```

```

00232
00233 #if PARTICLE_TRACKER_HAS_NANOFLLANN
00234
00249 template <typename DataType> struct PointCloud {
00251     std::vector<DataType> pts;
00252
00254     inline std::size_t kdtree_get_point_count() const { return pts.size() / 2; }
00255
00262     inline DataType kdtree_get_pt(const std::size_t idx,
00263                                   const std::size_t dim) const {
00264         return pts[2 * idx + dim];
00265     }
00266
00268     inline const DataType *data() const { return pts.data(); }
00269
00274     template <class BBOX> bool kdtree_get_bbox(BBOX &) const { return false; }
00275 };
00276
00297 template <typename DataType, class DatasetAdaptor>
00298 struct LonLatDistanceAdaptor {
00299     typedef DataType ElementType;
00300     typedef DataType DistanceType;
00301
00302     const DatasetAdaptor &data;
00303
00304     explicit LonLatDistanceAdaptor(const DatasetAdaptor &data_) : data(data_) {}
00305
00307     inline const DataType *getPoint(std::size_t idx) const {
00308         return &data.pts[2 * idx];
00309     }
00310
00316     inline DistanceType evalMetric(const DataType *a, const DataType *b,
00317                                   std::size_t /*size*/) const {
00318         DataType dlon = a[0] - b[0];
00319         if (dlon < -static_cast<DataType>(pt_pi))
00320             dlon += static_cast<DataType>(pt_two_pi);
00321         else if (dlon > static_cast<DataType>(pt_pi))
00322             dlon -= static_cast<DataType>(pt_two_pi);
00323
00324         DataType dlat = a[1] - b[1];
00325         DataType avgLat = (a[1] + b[1]) * static_cast<DataType>(0.5);
00326
00327         DataType scaled_dlon = dlon * std::cos(avgLat);
00328         return scaled_dlon * scaled_dlon + dlat * dlat;
00329     }
00330
00335     inline DistanceType evalMetric(const DataType *a, std::size_t idx_b,
00336                                   std::size_t size) const {
00337         const DataType *b = getPoint(idx_b);
00338         return evalMetric(a, b, size);
00339     }
00340
00349     template <typename T, typename IndexType>
00350     inline DistanceType accum_dist(const T a, const T b, IndexType dim) const {
00351         DistanceType diff = static_cast<DistanceType>(a - b);
00352
00353         if (dim == 0) {
00354             if (diff < -static_cast<DistanceType>(pt_pi))
00355                 diff += static_cast<DistanceType>(pt_two_pi);
00356             else if (diff > static_cast<DistanceType>(pt_pi))
00357                 diff -= static_cast<DistanceType>(pt_two_pi);
00358         }
00359
00360         return diff * diff;
00361     }
00362 };
00363
00367 template <typename DataType>
00368 using KDTree = nf::KDTreeSingleIndexAdaptor<
00369     LonLatDistanceAdaptor<DataType, PointCloud<DataType>>, PointCloud<DataType>,
00370     2>;
00371
00372 #else // PARTICLE_TRACKER_HAS_NANOFLLANN == 0
00373
00374 // Provide "poison pill" templates that fail loudly if instantiated.
00375
00376 template <typename...> struct pt_dependent_false : std::false_type {};
00377
00384 template <typename DataType> struct KDTree {
00385     static_assert(pt_dependent_false<DataType>::value,
00386                 "particle_tracker: KDTree is unavailable because "
00387                 "PARTICLE_TRACKER_HAS_NANOFLLANN=0. "
00388                 "Enable nanoflann support or avoid KD-tree dependent APIs.");
00389 };
00390
00391 #endif // PARTICLE_TRACKER_HAS_NANOFLLANN
00392

```

```

00393 //-----
00394 // Neighbor and NeighborsData
00395 //-----
00396
00417 template <typename DataType> struct Neighbor {
00418     std::size_t ind{};
00419     DataType dist{};
00420     DataType weight{};
00421
00422     Neighbor(std::size_t index = 0, DataType distance = DataType{},
00423             DataType node_weight = DataType{})
00424         : ind(index), dist(distance), weight(node_weight) {}
00425 };
00426
00427 #if PARTICLE_TRACKER_HAS_NANOFANN
00428
00450 template <typename DataType, typename TimeType = double> class NeighborsData {
00451 private:
00452     const KDTree<DataType> &index;
00453     const std::vector<DataType> &xx_sparse{};
00454     const std::vector<DataType> &yy_sparse{};
00455     const DataType rad2{};
00456     const DataType shape{};
00457
00458 public:
00459     NeighborsData(const KDTree<DataType> &idx, const std::vector<DataType> &xxs,
00460                 const std::vector<DataType> &yys, DataType r2,
00461                 DataType shape_param)
00462         : index(idx), xx_sparse(xxs), yy_sparse(yys), rad2(r2),
00463           shape(shape_param) {}
00464
00476     void computeNeighbors(std::vector<Neighbor<DataType>> &neighbors, DataType x,
00477                         DataType y) {
00478         x = pt_wrap_lon(x);
00479
00480         nf::SearchParameters searchparams;
00481         searchparams.sorted = false;
00482
00483         std::vector<nf::ResultItem<unsigned int, DataType>> indices_dists;
00484
00485         neighbors.clear();
00486
00487         point<DataType> query_point = {x, y};
00488         index.radiusSearch(query_point.data(), rad2, indices_dists, searchparams);
00489
00490         neighbors.reserve(indices_dists.size());
00491
00492         const DataType lon_q = pt_wrap_lon(query_point[0]);
00493
00494         for (const auto &result : indices_dists) {
00495             Neighbor<DataType> nbr;
00496             nbr.ind = result.first;
00497
00498             const DataType lon_i = pt_wrap_lon(xx_sparse.at(nbr.ind));
00499
00500             nbr.dist = GCD(lon_q, query_point[1], lon_i, yy_sparse.at(nbr.ind));
00501             nbr.weight = std::exp(-nbr.dist * nbr.dist / shape);
00502
00503             neighbors.push_back(nbr);
00504         }
00505     }
00506
00515     void updateNeighborWeights(std::vector<Neighbor<DataType>> &neighbors,
00516                             DataType x, DataType y) {
00517         x = pt_wrap_lon(x);
00518
00519         point<DataType> query_point = {x, y};
00520         const DataType lon_q = pt_wrap_lon(query_point[0]);
00521
00522         for (auto &nbr : neighbors) {
00523             const DataType lon_i = pt_wrap_lon(xx_sparse.at(nbr.ind));
00524             nbr.dist = GCD(lon_q, query_point[1], lon_i, yy_sparse.at(nbr.ind));
00525             nbr.weight = std::exp(-nbr.dist * nbr.dist / shape);
00526         }
00527     }
00528
00539     std::map<std::string, DataType> interpolateVariables(
00540         TimeType t, const std::vector<Neighbor<DataType>> &neighbors,
00541         const std::map<std::string, std::vector<TimeSpline<DataType, TimeType>>>
00542             &variable_splines,
00543         const std::vector<std::string> &variable_names) {
00544         std::map<std::string, DataType> out;
00545         DataType den = static_cast<DataType>(0);
00546
00547         for (const auto &var_name : variable_names)
00548             out[var_name] = static_cast<DataType>(0);
00549     }

```

```

00550     for (const auto &nbr : neighbors) {
00551         for (const auto &var_name : variable_names) {
00552             const auto &splines = variable_splines.at(var_name);
00553
00554             if (nbr.ind >= splines.size()) {
00555                 throw std::out_of_range("particle_tracker::NeighborsData: index " +
00556                                         std::to_string(nbr.ind) +
00557                                         " out of bounds for variable '" + var_name +
00558                                         "'");
00559             }
00560
00561             const DataType v = splines[nbr.ind].spline(t);
00562             out[var_name] += nbr.weight * v;
00563         }
00564         den += nbr.weight;
00565     }
00566
00567     if (den > static_cast<DataType>(1e-8)) {
00568         for (const auto &var_name : variable_names)
00569             out[var_name] /= den;
00570     } else {
00571         for (const auto &var_name : variable_names)
00572             out[var_name] = static_cast<DataType>(0);
00573     }
00574
00575     return out;
00576 }
00577
00581 std::map<std::string, DataType> calculateVariables(
00582     TimeType t, std::vector<Neighbor<DataType>> &neighbors,
00583     const std::map<std::string, std::vector<TimeSpline<DataType, TimeType>>> &variable_splines,
00584     const std::vector<std::string> &variable_names, DataType x, DataType y) {
00585     computeNeighbors(neighbors, x, y);
00586     return interpolateVariables(t, neighbors, variable_splines, variable_names);
00587 }
00588
00593 std::map<std::string, DataType> calculateVariablesUpdate(
00594     TimeType t, std::vector<Neighbor<DataType>> &neighbors,
00595     const std::map<std::string, std::vector<TimeSpline<DataType, TimeType>>> &variable_splines,
00596     const std::vector<std::string> &variable_names, DataType x, DataType y) {
00597     updateNeighborWeights(neighbors, x, y);
00598     return interpolateVariables(t, neighbors, variable_splines, variable_names);
00599 }
00600
00601 };
00602
00603 #else // PARTICLE_TRACKER_HAS_NANOFANN == 0
00604
00605 template <typename DataType, typename TimeType = double> class NeighborsData {
00606     static_assert(pt_dependent_false<DataType, TimeType>::value,
00607                 "particle_tracker::NeighborsData is unavailable because "
00608                 "PARTICLE_TRACKER_HAS_NANOFANN=0. "
00609                 "Enable nanoflann support or avoid KD-tree dependent APIs.");
00610 };
00611
00612 #endif // PARTICLE_TRACKER_HAS_NANOFANN
00613
00614 //-----
00615 // Experimental derivative estimator: NeighborPrime, NeighborWithNeighbors,
00616 // VariableDerivatives
00617 //-----
00618
00638 template <typename DataType> struct NeighborPrime {
00639     std::size_t ind_prime{};
00640     std::array<DataType, 3> dist_prime{};
00641     DataType weight_prime{};
00642
00643     NeighborPrime(std::size_t index_prime = 0,
00644                 std::array<DataType, 3> distance_prime = {DataType{},
00645                                                         DataType{},
00646                                                         DataType{}},
00647                 DataType node_weight_prime = DataType{})
00648         : ind_prime(index_prime), dist_prime(distance_prime),
00649           weight_prime(node_weight_prime) {}
00650 };
00651
00658 template <typename DataType> struct NeighborWithNeighbors : Neighbor<DataType> {
00659     std::vector<NeighborPrime<DataType>> neighbors_prime;
00660
00661     NeighborWithNeighbors() = default;
00662
00663     NeighborWithNeighbors(std::size_t index, DataType distance,
00664                         DataType node_weight,
00665                         const std::vector<NeighborPrime<DataType>> &nbr_prime)
00666         : Neighbor<DataType>(index, distance, node_weight),
00667           neighbors_prime(nbr_prime) {}

```

```

00668 };
00669
00677 template <typename DataType> struct VariableDerivatives {
00678     DataType value{};
00679     DataType time_derivative{};
00680     DataType spatial_derivative_x{};
00681     DataType spatial_derivative_y{};
00682 };
00683
00684 #if PARTICLE_TRACKER_HAS_NANOFANN
00685
00717 template <typename DataType, typename TimeType = double>
00718 class NeighborWithNeighborsData {
00719 private:
00720     const KDTree<DataType> &index;
00721     const std::vector<DataType> &xx_sparse{};
00722     const std::vector<DataType> &yy_sparse{};
00723     const DataType rad2{};
00724     const DataType shape{};
00725 public:
00726     NeighborWithNeighborsData(const KDTree<DataType> &idx,
00727                             const std::vector<DataType> &xxs,
00728                             const std::vector<DataType> &yys, DataType r2,
00729                             DataType shape_param)
00730         : index(idx), xx_sparse(xxs), yy_sparse(yys), rad2(r2),
00731           shape(shape_param) {}
00732
00741 void computeNeighbors(std::vector<NeighborWithNeighbors<DataType>> &neighbors,
00742                     DataType x, DataType y) {
00743     x = pt_wrap_lon(x);
00744
00745     point<DataType> query_point = {x, y};
00746
00747     nf::SearchParameters searchparams;
00748     searchparams.sorted = false;
00749
00750     std::vector<nf::ResultItem<unsigned int, DataType>> indices_dists;
00751     index.radiusSearch(query_point.data(), rad2, indices_dists, searchparams);
00752
00753     neighbors.clear();
00754     neighbors.reserve(indices_dists.size());
00755
00756     const DataType lon_q = pt_wrap_lon(query_point[0]);
00757
00758     for (const auto &result : indices_dists) {
00759         NeighborWithNeighbors<DataType> nbr;
00760         nbr.ind = result.first;
00761
00762         const DataType lon_i = pt_wrap_lon(xx_sparse.at(nbr.ind));
00763         const DataType lat_i = yy_sparse.at(nbr.ind);
00764
00765         // True great-circle distance query -> primary neighbor (for primary
00766         // weights).
00767         nbr.dist = GCD(lon_q, query_point[1], lon_i, lat_i);
00768         nbr.weight = std::exp(-nbr.dist * nbr.dist / shape);
00769
00770         // Secondary search is centered at the primary grid node.
00771         point<DataType> query_point_prime = {lon_i, lat_i};
00772
00773         std::vector<nf::ResultItem<unsigned int, DataType>> indices_dists_prime;
00774         index.radiusSearch(query_point_prime.data(), rad2, indices_dists_prime,
00775                           searchparams);
00776
00777         if (indices_dists_prime.empty()) {
00778             // Loud behavior: do not keep a primary neighbor with no secondary
00779             // neighborhood because derivative estimation would be undefined/noisy.
00780             continue;
00781         }
00782
00783         nbr.neighbors_prime.clear();
00784         nbr.neighbors_prime.reserve(indices_dists_prime.size());
00785
00786         for (const auto &result_prime : indices_dists_prime) {
00787             NeighborPrime<DataType> nbr_prime;
00788             nbr_prime.ind_prime = result_prime.first;
00789
00790             if (nbr_prime.ind_prime >= xx_sparse.size() ||
00791                 nbr_prime.ind_prime >= yy_sparse.size()) {
00792                 throw std::out_of_range(
00793                     "particle_tracker::NeighborWithNeighborsData: index " +
00794                     std::to_string(nbr_prime.ind_prime) +
00795                     " out of bounds in computeNeighbors (secondary).");
00796             }
00797
00798             const DataType lon_j = pt_wrap_lon(xx_sparse.at(nbr_prime.ind_prime));
00799             const DataType lat_j = yy_sparse.at(nbr_prime.ind_prime);

```

```

00800
00801     // dist_prime = [d, ddx, ddy] from GCD_deriv implementation.
00802     nbr_prime.dist_prime =
00803         GCD_deriv(query_point_prime[0], query_point_prime[1], lon_j, lat_j);
00804     nbr_prime.weight_prime = std::exp(-nbr_prime.dist_prime[0] *
00805                                       nbr_prime.dist_prime[0] / shape);
00806
00807     nbr.neighbors_prime.push_back(nbr_prime);
00808 }
00809
00810     neighbors.push_back(nbr);
00811 }
00812 }
00813
00820 void
00821 updateNeighborWeights(std::vector<NeighborWithNeighbors<DataType>> &neighbors,
00822                     DataType x, DataType y) {
00823     x = pt_wrap_lon(x);
00824
00825     point<DataType> query_point = {x, y};
00826     const DataType lon_q = pt_wrap_lon(query_point[0]);
00827
00828     for (auto &nbr : neighbors) {
00829         const DataType lon_i = pt_wrap_lon(xx_sparse.at(nbr.ind));
00830         nbr.dist = GCD(lon_q, query_point[1], lon_i, yy_sparse.at(nbr.ind));
00831         nbr.weight = std::exp(-nbr.dist * nbr.dist / shape);
00832     }
00833 }
00834
00848 std::map<std::string, VariableDerivatives<DataType> interpolateDerivatives(
00849     TimeType t, const std::vector<NeighborWithNeighbors<DataType>> &neighbors,
00850     const std::map<std::string, std::vector<TimeSpline<DataType, TimeType>>
00851         &variable_splines,
00852     const std::vector<std::string> &variable_names, DataType shape) const {
00853     // Primary accumulators
00854     std::map<std::string, DataType> value_acc;
00855     std::map<std::string, DataType> dt_acc;
00856     std::map<std::string, DataType> dx_acc;
00857     std::map<std::string, DataType> dy_acc;
00858
00859     DataType den = static_cast<DataType>(0);
00860
00861     for (const auto &name : variable_names) {
00862         value_acc[name] = static_cast<DataType>(0);
00863         dt_acc[name] = static_cast<DataType>(0);
00864         dx_acc[name] = static_cast<DataType>(0);
00865         dy_acc[name] = static_cast<DataType>(0);
00866     }
00867
00868     for (const auto &nbr : neighbors) {
00869         const DataType w = nbr.weight;
00870         den += w;
00871
00872         // Value + time derivative from splines at primary neighbor grid nodes
00873         for (const auto &name : variable_names) {
00874             const auto &splines = variable_splines.at(name);
00875
00876             if (nbr.ind >= splines.size()) {
00877                 throw std::out_of_range(
00878                     "particle_tracker::NeighborWithNeighborsData: index " +
00879                     std::to_string(nbr.ind) + " out of bounds for variable '" + name +
00880                     "'.");
00881             }
00882
00883             const DataType v = splines[nbr.ind].spline(t);
00884             const DataType dv = splines[nbr.ind].spline.prime(t);
00885
00886             value_acc[name] += w * v;
00887             dt_acc[name] += w * dv;
00888         }
00889
00890         // Per-primary-neighbor spatial derivative accumulators (heuristic)
00891         DataType den_prime = static_cast<DataType>(0);
00892
00893         std::map<std::string, DataType> dx_prime;
00894         std::map<std::string, DataType> dy_prime;
00895
00896         for (const auto &name : variable_names) {
00897             dx_prime[name] = static_cast<DataType>(0);
00898             dy_prime[name] = static_cast<DataType>(0);
00899         }
00900
00901         for (const auto &nbr_prime : nbr.neighbors_prime) {
00902             // Skip self-pairs (no geometric meaning for dv)
00903             if (nbr_prime.ind_prime == nbr.ind)
00904                 continue;
00905

```

```

00906     const DataType d = nbr_prime.dist_prime[0];
00907     const DataType ddx = nbr_prime.dist_prime[1];
00908     const DataType ddy = nbr_prime.dist_prime[2];
00909
00910     // IMPORTANT: den_prime must be independent of number of variables
00911     // requested.
00912     den_prime += nbr_prime.weight_prime;
00913
00914     for (const auto &name : variable_names) {
00915         const auto &splines = variable_splines.at(name);
00916
00917         if (nbr_prime.ind_prime >= splines.size()) {
00918             throw std::out_of_range(
00919                 "particle_tracker::NeighborWithNeighborsData: index " +
00920                 std::to_string(nbr_prime.ind_prime) +
00921                 " out of bounds for variable '" + name + "' (secondary).");
00922         }
00923
00924         const DataType v_i = splines[nbr_prime.ind].spline(t);
00925         const DataType v_j = splines[nbr_prime.ind_prime].spline(t);
00926         const DataType dv = v_j - v_i;
00927
00928         // Heuristic estimator building block:
00929         // weight' * (d * d(d)/dx * dv), and analogously for y.
00930         dx_prime[name] += nbr_prime.weight_prime * (d * ddx * dv);
00931         dy_prime[name] += nbr_prime.weight_prime * (d * ddy * dv);
00932     }
00933 }
00934
00935 if (den_prime > static_cast<DataType>(1e-16)) {
00936     for (const auto &name : variable_names) {
00937         dx_acc[name] += w * (dx_prime[name] / den_prime);
00938         dy_acc[name] += w * (dy_prime[name] / den_prime);
00939     }
00940 }
00941 }
00942
00943 std::map<std::string, VariableDerivatives<DataType>> out;
00944
00945 if (den > static_cast<DataType>(1e-16)) {
00946     for (const auto &name : variable_names) {
00947         VariableDerivatives<DataType> vd;
00948
00949         vd.value = value_acc[name] / den;
00950         vd.time_derivative = dt_acc[name] / den;
00951
00952         vd.spatial_derivative_x =
00953             (static_cast<DataType>(2) / shape) * (dx_acc[name] / den);
00954         vd.spatial_derivative_y =
00955             (static_cast<DataType>(2) / shape) * (dy_acc[name] / den);
00956
00957         out[name] = vd;
00958     }
00959 } else {
00960     // No usable primary weights -> return zeros.
00961     for (const auto &name : variable_names)
00962         out[name] = VariableDerivatives<DataType>{};
00963 }
00964
00965 return out;
00966 }
00967
00971 std::map<std::string, VariableDerivatives<DataType>> calculateDerivatives(
00972     TimeType t, std::vector<NeighborWithNeighbors<DataType>> &neighbors,
00973     const std::map<std::string, std::vector<TimeSpline<DataType, TimeType>>>
00974         &variable_splines,
00975     const std::vector<std::string> &variable_names, DataType x, DataType y,
00976     DataType shape) {
00977     computeNeighbors(neighbors, x, y);
00978     return interpolateDerivatives(t, neighbors, variable_splines,
00979         variable_names, shape);
00980 }
00981
00985 std::map<std::string, VariableDerivatives<DataType>>
00986 calculateDerivativesUpdate(
00987     TimeType t, std::vector<NeighborWithNeighbors<DataType>> &neighbors,
00988     const std::map<std::string, std::vector<TimeSpline<DataType, TimeType>>>
00989         &variable_splines,
00990     const std::vector<std::string> &variable_names, DataType x, DataType y,
00991     DataType shape) {
00992     updateNeighborWeights(neighbors, x, y);
00993     return interpolateDerivatives(t, neighbors, variable_splines,
00994         variable_names, shape);
00995 }
00996 };
00997
00998 #else // PARTICLE_TRACKER_HAS_NANOFLANN == 0

```



```

00999
01000 template <typename DataType, typename TimeType = double>
01001 class NeighborWithNeighborsData {
01002     static_assert(pt_dependent_false<DataType, TimeType>::value,
01003         "particle_tracker::NeighborWithNeighborsData is unavailable "
01004         "because PARTICLE_TRACKER_HAS_NANOFANN=0. "
01005         "Enable nanoflann support or avoid KD-tree dependent APIs.");
01006 };
01007
01008 #endif // PARTICLE_TRACKER_HAS_NANOFANN
01009
01010 } // namespace particle_tracker

```

6.17 seed_loader_v2.hpp

```

00001 #pragma once
00002 #include <stdint>
00003 #include <string>
00004 #include <vector>
00005 #include <fstream>
00006 #include <sstream>
00007 #include <stdexcept>
00008 #include <algorithm>
00009 #include <cctype>
00010 #include <filesystem>
00011 #include <cstdio>
00012 #include <limits>
00013
00014 #include <zlib.h>
00015
00016 #if !defined(PK_IO_USE_ZSTD) && !defined(PK_IO_USE_ZLIB)
00017 #   define PK_IO_USE_ZLIB
00018 #endif
00019 #include "particle_tracker/fixed_point/fixed_point_core.hpp" // wrap_lon(), pi
00020 #include "particle_tracker/fixed_point/fixed_point_pkd2.hpp" // pkd2::{Header,TocEntry,BlockHeader}
00021
00022 // =====
00023 // Public API
00024 // =====
00025
00026 // MUST match Writer(pos_scale=1'000'000u) used for PKD2 trajectories.
00027 inline constexpr double POS_TICKS_PER_RAD = 1'000'000.0;
00028
00029 enum class SeedPoint { First, Last };
00030
00031 // Seed record:
00032 // - ACTIVE seed point: lon_rad/lat_rad/t_start_s (this is what simulation starts from)
00033 // - STOP target (optional): lon_stop_rad/lat_stop_rad/t_stop_s
00034 // - For convenience/back-compat: lon_start_rad/lat_start_rad mirror ACTIVE.
00035 struct SeedInit
00036 {
00037     uint64_t seed_id{};
00038
00039     // ACTIVE point
00040     double lon_rad{}; // radians in (-pi,pi]
00041     double lat_rad{}; // radians
00042     int64_t t_start_s{}; // seconds (t2000 for PKD2; CSV depends on caller)
00043
00044     // Convenience/back-compat: mirrors ACTIVE
00045     double lon_start_rad{};
00046     double lat_start_rad{};
00047
00048     // Optional STOP target
00049     double lon_stop_rad{};
00050     double lat_stop_rad{};
00051     int64_t t_stop_s{};
00052     bool has_stop{false};
00053 };
00054
00055 using SeedRec = SeedInit;
00056
00057 // =====
00058 // Internal helpers
00059 // =====
00060 namespace seed_v2_detail
00061 {
00062     inline bool ends_with_ci(const std::string& s, const std::string& suf)
00063     {
00064         if (s.size() < suf.size()) return false;
00065         auto lower = [](unsigned char c){ return static_cast<char>(std::tolower(c)); };
00066
00067         std::string a = s.substr(s.size() - suf.size());
00068         std::transform(a.begin(), a.end(), a.begin(), lower);
00069     }
00070 }

```

```

00069
00070     std::string b = suf;
00071     std::transform(b.begin(), b.end(), b.begin(), lower);
00072
00073     return a == b;
00074 }
00075
00076 inline void fread_or_throw(FILE* f, void* p, size_t n, const char* what)
00077 {
00078     if (std::fread(p, 1, n, f) != n)
00079         throw std::runtime_error(std::string("PKD2: fread failed: ") + what);
00080 }
00081
00082 inline void fseek_or_throw(FILE* f, long off, int whence, const char* what)
00083 {
00084     if (std::fseek(f, off, whence) != 0)
00085         throw std::runtime_error(std::string("PKD2: fseek failed: ") + what);
00086 }
00087
00088 inline std::vector<unsigned char> zlib_decompress_exact(const unsigned char* comp,
00089                                                         size_t comp_sz,
00090                                                         size_t raw_sz)
00091 {
00092     std::vector<unsigned char> raw(raw_sz);
00093     uLongf out = static_cast<uLongf>(raw_sz);
00094     int rc = ::uncompress(raw.data(), &out, comp, static_cast<uLong>(comp_sz));
00095     if (rc != Z_OK || out != raw_sz)
00096         throw std::runtime_error("PKD2: zlib uncompress failed (wrong codec or corrupt block)");
00097     return raw;
00098 }
00099
00100 // Decode first+last sample of one PKD2 block.
00101 // Returns two "ACTIVE-only" records: first and last (no stop metadata assigned here).
00102 inline void decode_block_first_last(const pkd2::Header& hdr,
00103                                     const pkd2::TocEntry& te,
00104                                     FILE* f,
00105                                     SeedInit& first_out,
00106                                     SeedInit& last_out)
00107 {
00108     std::vector<unsigned char> comp(te.comp_size);
00109     fseek_or_throw(f, static_cast<long>(te.file_offset), SEEK_SET, "block");
00110     fread_or_throw(f, comp.data(), comp.size(), "block bytes");
00111
00112     auto raw = zlib_decompress_exact(comp.data(), comp.size(), te.raw_size);
00113     if (raw.size() < sizeof(pkd2::BlockHeader))
00114         throw std::runtime_error("PKD2: raw block too small");
00115
00116     const auto* bh = reinterpret_cast<const pkd2::BlockHeader*>(raw.data());
00117     const uint32_t n = bh->n_points;
00118     const uint16_t stride = bh->stride ? bh->stride : 1;
00119     if (n == 0) throw std::runtime_error("PKD2: n_points==0 (invalid)");
00120
00121     // Strong sanity: pos_scale should match your writer
00122     if (hdr.pos_scale_ticks_per_rad != static_cast<uint32_t>(POS_TICKS_PER_RAD))
00123         throw std::runtime_error("PKD2: pos_scale_ticks_per_rad mismatch (file vs expected 1e6)");
00124
00125     const double inv = 1.0 / static_cast<double>(hdr.pos_scale_ticks_per_rad);
00126
00127     const double L0 = static_cast<double>(bh->lon0_q) * inv;
00128     const double B0 = static_cast<double>(bh->lat0_q) * inv;
00129
00130     const int64_t t0 = hdr.t0_epoch_s
00131         + static_cast<int64_t>(bh->start_index) * static_cast<int64_t>(hdr.dt_s);
00132
00133     const int32_t* dlon = reinterpret_cast<const int32_t*>(raw.data() +
00134     sizeof(pkd2::BlockHeader));
00135     const int32_t* dlat = dlon + (n > 1 ? (n - 1) : 0);
00136
00137     double L = L0;
00138     double B = B0;
00139     for (uint32_t i = 1; i < n; ++i)
00140     {
00141         L += static_cast<double>(dlon[i - 1]) * inv;
00142         B += static_cast<double>(dlat[i - 1]) * inv;
00143     }
00144
00145     const int64_t t_last = t0
00146         + static_cast<int64_t>(n - 1)
00147         * static_cast<int64_t>(hdr.dt_s)
00148         * static_cast<int64_t>(stride);
00149
00150     // FIRST active
00151     first_out = {};
00152     first_out.seed_id = bh->seed_id;
00153     first_out.lon_rad = wrap_lon(L0);
00154     first_out.lat_rad = B0;
00155     first_out.t_start_s = t0;

```

```

00155     first_out.lon_start_rad = first_out.lon_rad;
00156     first_out.lat_start_rad = first_out.lat_rad;
00157     first_out.has_stop = false;
00158
00159     // LAST active
00160     last_out = {};
00161     last_out.seed_id = bh->seed_id;
00162     last_out.lon_rad = wrap_lon(L);
00163     last_out.lat_rad = B;
00164     last_out.t_start_s = t_last;
00165     last_out.lon_start_rad = last_out.lon_rad;
00166     last_out.lat_start_rad = last_out.lat_rad;
00167     last_out.has_stop = false;
00168 }
00169
00170 // PKD2 loader: attach stop target depending on which_point.
00171 inline std::vector<SeedInit> load_seeds_from_pkd2_zlib(const std::string& path, SeedPoint
which_point)
00172 {
00173     FILE* f = std::fopen(path.c_str(), "rb");
00174     if (!f) throw std::runtime_error("PKD2: cannot open " + path);
00175
00176     pkd2::Header hdr{};
00177     try
00178     {
00179         fread_or_throw(f, &hdr, sizeof(hdr), "header");
00180
00181         if (!(hdr.magic[0]=='P' && hdr.magic[1]=='K' && hdr.magic[2]=='D' && hdr.magic[3]=='2'))
00182             throw std::runtime_error("PKD2: bad magic");
00183         if (hdr.version != 3)
00184             throw std::runtime_error("PKD2: unsupported version (expected 3)");
00185         if (hdr.dt_s <= 0)
00186             throw std::runtime_error("PKD2: dt_s <= 0");
00187         if (hdr.n_traj == 0)
00188         {
00189             std::fclose(f);
00190             return {};
00191         }
00192
00193         fseek_or_throw(f, static_cast<long>(hdr.toc_offset), SEEK_SET, "toc");
00194         std::vector<pkd2::TocEntry> toc(static_cast<size_t>(hdr.n_traj));
00195         fread_or_throw(f, toc.data(), toc.size() * sizeof(pkd2::TocEntry), "toc bytes");
00196
00197         std::vector<SeedInit> out;
00198         out.reserve(toc.size());
00199
00200         for (const auto& te : toc)
00201         {
00202             SeedInit first{}, last{};
00203             decode_block_first_last(hdr, te, f, first, last);
00204
00205             // Choose ACTIVE and STOP
00206             SeedInit active = (which_point == SeedPoint::First) ? first : last;
00207             const SeedInit stop = (which_point == SeedPoint::First) ? last : first;
00208
00209             active.has_stop = true;
00210             active.lon_stop_rad = stop.lon_rad;
00211             active.lat_stop_rad = stop.lat_rad;
00212             active.t_stop_s = stop.t_start_s;
00213
00214             // Mirror ACTIVE into lon_start/lat_start for back-compat
00215             active.lon_start_rad = active.lon_rad;
00216             active.lat_start_rad = active.lat_rad;
00217
00218             out.push_back(std::move(active));
00219         }
00220
00221         std::fclose(f);
00222         return out;
00223     }
00224     catch (...)
00225     {
00226         std::fclose(f);
00227         throw;
00228     }
00229 }
00230
00231 inline std::vector<SeedInit> load_seeds_from_csv_basic(const std::string& csv_path)
00232 {
00233     std::ifstream is(csv_path);
00234     if (!is) throw std::runtime_error("Seeds CSV: cannot open " + csv_path);
00235
00236     std::string header;
00237     if (!std::getline(is, header))
00238         throw std::runtime_error("Seeds CSV: empty file " + csv_path);
00239
00240     auto trim = [](std::string s){

```

```

00241         while (!s.empty() && std::isspace((unsigned char)s.front())) s.erase(s.begin());
00242         while (!s.empty() && std::isspace((unsigned char)s.back())) s.pop_back();
00243         return s;
00244     };
00245
00246     auto idx_of = [&](const std::string& name) -> int {
00247         std::stringstream ss(header);
00248         std::string tok;
00249         int k = 0;
00250         while (std::getline(ss, tok, ','))
00251         {
00252             tok = trim(tok);
00253             if (tok == name) return k;
00254             ++k;
00255         }
00256         return -1;
00257     };
00258
00259     const int i_seed = idx_of("seed_id");
00260     const int i_lon  = idx_of("lon_rad");
00261     const int i_lat  = idx_of("lat_rad");
00262     const int i_t    = idx_of("t_start_s");
00263     if (i_seed < 0 || i_lon < 0 || i_lat < 0 || i_t < 0)
00264         throw std::runtime_error("Seeds CSV: expected columns:
seed_id,lon_rad,lat_rad,t_start_s");
00265
00266     std::vector<SeedInit> out;
00267     std::string line;
00268     while (std::getline(is, line))
00269     {
00270         if (line.empty()) continue;
00271         std::stringstream ss(line);
00272         std::string f;
00273         int col = 0;
00274
00275         SeedInit s{};
00276         while (std::getline(ss, f, ','))
00277         {
00278             f = trim(f);
00279             if (col == i_seed) s.seed_id = std::stoull(f);
00280             if (col == i_lon) s.lon_rad = wrap_lon(std::stod(f));
00281             if (col == i_lat) s.lat_rad = std::stod(f);
00282             if (col == i_t) s.t_start_s = std::stoll(f);
00283             ++col;
00284         }
00285
00286         // CSV: only ACTIVE, no STOP.
00287         s.lon_start_rad = s.lon_rad;
00288         s.lat_start_rad = s.lat_rad;
00289         s.has_stop = false;
00290         out.push_back(s);
00291     }
00292     return out;
00293 }
00294 } // namespace seed_v2_detail
00295
00296 // =====
00297 // Public loaders
00298 // =====
00299
00300 inline std::vector<SeedInit> load_seeds_auto(const std::string& path,
00301                                             SeedPoint which_point = SeedPoint::Last)
00302 {
00303     using namespace seed_v2_detail;
00304
00305     if (ends_with_ci(path, ".pkd2"))
00306         return load_seeds_from_pkd2_zlib(path, which_point);
00307
00308     if (ends_with_ci(path, ".csv"))
00309         return load_seeds_from_csv_basic(path);
00310
00311     throw std::runtime_error("Seeds: unsupported file type (expected .pkd2 or .csv): " + path);
00312 }
00313
00314 inline std::vector<SeedRec> load_seed_recs_auto(const std::string& path,
00315                                                SeedPoint which_point = SeedPoint::Last)
00316 {
00317     return load_seeds_auto(path, which_point);
00318 }

```